

Automatización del ciclo post-venta en e-commerce: pipeline de procesamiento de órdenes y atención al cliente con IA, implementado con n8n

Santiago Sordi, Ignacio Odorico, Juan Cruz Ana
Mendoza, Argentina — 2026

PORTADA

Universidad Tecnológica Nacional – Facultad Regional Mendoza

Carrera: Tecnicatura Universitaria en Programación

Materia: Trabajo Integrador

Título: Automatización del ciclo post-venta en e-commerce: pipeline de procesamiento de órdenes y atención al cliente con IA, implementado con n8n

Autores: Santiago Sordi, Ignacio Odorico, Juan Cruz Ana

Director / Profesor: Alberto Cortez

Mendoza – Argentina, 2026

DECLARACIÓN DE ORIGINALIDAD

Quienes suscriben declaran que el presente Trabajo Integrador es de su autoría, que fue elaborado íntegramente para la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, Facultad Regional Mendoza, y que no ha sido presentado con anterioridad para la obtención de ningún otro título o grado académico.

Todas las fuentes consultadas se encuentran debidamente citadas en el cuerpo del texto y consignadas en el Capítulo 8. Los datos experimentales que se reportan en el Capítulo 5 fueron obtenidos por los autores sobre el sistema descrito en el Capítulo 4; los guiones de ejecución, los archivos de resultados crudos y los procedimientos de cálculo se versionan en el repositorio del trabajo y se transcriben en los Anexos I y J, de modo que toda cifra publicada pueda ser recalculada de forma independiente.

Se deja constancia del uso de herramientas de asistencia basadas en modelos de lenguaje durante la redacción y la revisión del documento y durante el desarrollo del software. Su empleo se limitó a tareas de asistencia: el diseño experimental, la ejecución de las mediciones, la interpretación de los resultados y las conclusiones son responsabilidad exclusiva de los autores, que asumen la autoría plena del trabajo.

Santiago Sordi — Ignacio Odorico — Juan Cruz Ana

Mendoza, 2026

RESUMEN

El presente trabajo aborda el problema del procesamiento manual del ciclo post-venta en PyMEs de e-commerce argentinas, cuya gestión artesanal genera demoras, errores y fragmentación en la atención al cliente. Se plantea la siguiente pregunta de investigación: *¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de las métricas MTTD, MTTR y TMR?*

Se diseñó e implementó una solución compuesta por dos flujos orquestados con n8n y desplegados en contenedores Docker. El primer flujo es un pipeline de procesamiento de órdenes (15 nodos) que recibe pedidos vía webhook, verifica stock en PostgreSQL, actualiza estados y notifica al cliente por email. El segundo flujo es un chatbot multicanal implementado y validado sobre dos canales —WhatsApp, con envío simulado a un capturador SMTP local, y Telegram, con envío real de extremo a extremo—, utilizando GPT-4o-mini para clasificar intenciones y generar respuestas automatizadas. La infraestructura incluye PostgreSQL 15 como base de datos, Mailpit como servidor SMTP de pruebas y Grafana para visualizar métricas operativas.

Los resultados obtenidos muestran un MTTD promedio de 0,009 segundos y un MTTR promedio de 0,054 segundos, para un tiempo total end-to-end de 0,063 segundos sobre 50 órdenes medidas. El costo marginal de procesar una orden en el pipeline resulta así unas 780 veces menor que el del proceso manual, cronometrado por el propio equipo sobre diez órdenes en 49,13 segundos por orden (IC 95 %: 43,30 s a 54,95 s; IC 95 % del factor por el teorema de Fieller: $686\times$ a $875\times$). Ambas series están completamente separadas y el contraste es concluyente (U de Mann-Whitney = 0, $p = 2,7 \times 10^{-11}$; t de Welch = 19,05, gl = 9,0, $p = 1,4 \times 10^{-8}$); el factor compara costos marginales de procesamiento y no latencias percibidas por el cliente, y debe leerse como una cota superior por provenir el término automatizado de un entorno de laboratorio. El TMR promedio del chatbot fue de 1,47 segundos sobre un corpus de 150 interacciones y de 3,07 segundos sobre 45 interacciones del canal Telegram real, y la precisión de clasificación alcanzó el 92,7 % (IC 95 % de Wilson: 87,3 % a 95,9 %). Estos valores permiten confirmar la hipótesis de que la automatización reduce de manera sustancial los tiempos operativos del ciclo post-venta respecto del proceso manual de referencia.

Palabras clave: automatización de procesos, e-commerce, post-venta, n8n, chatbot, modelos de lenguaje, Docker, MTTD, MTTR, TMR.

ABSTRACT

This work addresses the manual handling of the post-sale cycle in Argentine e-commerce SMEs, where craft-based management produces delays, errors and fragmented customer service. The research question is stated as follows: to what extent does an automation pipeline orchestrated with n8n reduce the operational times of the post-sale cycle of a simulated e-commerce store, measured through the MTTD, MTTR and TMR metrics?

A solution was designed and implemented as two workflows orchestrated with n8n and deployed in Docker containers. The first is an order-processing pipeline (15 nodes) that receives orders through a webhook, checks stock in PostgreSQL, updates order states and notifies the customer by email. The second is a multichannel chatbot implemented and validated over two channels—a simulated channel that adopts the WhatsApp Cloud API message format and delivers through a local SMTP capture server, and Telegram, with real end-to-end delivery—using GPT-4o-mini to classify intents and generate automated replies. The infrastructure comprises PostgreSQL 15 as the database, Mailpit as the test SMTP server and Grafana for operational dashboards.

The results show a mean MTTD of 0.009 seconds and a mean MTTR of 0.054 seconds, for a total end-to-end time of 0.063 seconds over 50 measured orders. The marginal cost of processing one order in the pipeline is therefore about 780 times lower than that of the manual process, timed by the authors over ten orders at 49.13 seconds per order (95 % CI: 43.30 s to 54.95 s; Fieller 95 % CI of the ratio: 686× to 875×). The two series are completely separated, and the contrast is conclusive (Mann-Whitney $U = 0$, $p = 2.7 \times 10^{-11}$; Welch $t = 19.05$, $df = 9.0$, $p = 1.4 \times 10^{-8}$). The factor compares marginal processing costs and not customer-perceived latencies, and must be read as an upper bound, since the automated term comes from a laboratory environment. The chatbot achieved a mean response time of 1.47 seconds over a corpus of 150 interactions and 3.07 seconds over 45 interactions on the real Telegram channel, and an intent classification accuracy of 92.7 % (Wilson 95 % CI: 87.3 % to 95.9 %) in a zero-shot regime. These values confirm the hypothesis that automation substantially reduces the operational times of the post-sale cycle with respect to the manual reference process.

Keywords: business process automation, e-commerce, post-sale service, n8n, chatbot, large language models, Docker, MTTD, MTTR, mean response time.

ÍNDICE

DECLARACIÓN DE ORIGINALIDAD	2
RESUMEN	3
ABSTRACT	4

Listado de Figuras.....	8
Listado de Tablas principales	10
Glosario de siglas y acrónimos	11
CAPÍTULO 1: INTRODUCCIÓN	14
1.1 Contexto del problema	14
1.2 Planteamiento del problema	14
1.3 Justificación.....	15
1.4 Pregunta de Investigación e Hipótesis.....	15
1.4.1 Pregunta de investigación.....	15
1.4.2 Hipótesis.....	15
1.5 Objetivos	16
1.5.1 Objetivo General	16
1.5.2 Objetivos Específicos.....	16
1.6 Alcance y Limitaciones	17
1.6.1 Alcance.....	17
1.6.2 Limitaciones.....	17
CAPÍTULO 2: MARCO TEÓRICO	18
2.1 Automatización de procesos de negocio.....	18
2.1.1 Definición y antecedentes	18
2.1.2 Herramientas de orquestación de flujos: n8n.....	18
2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce.....	19
2.2 Inteligencia artificial conversacional y modelos de lenguaje.....	20
2.2.1 Evolución de los chatbots: de reglas a LLMs	20
2.2.2 GPT-4o-mini y prompt engineering	20
2.2.3 Precisión en clasificación de intents	21
2.3 Comunicación multicanal en e-commerce	21
2.3.1 Multicanalidad y omnicanalidad: delimitación conceptual	21
2.3.2 TMR como métrica de efectividad	22
2.4 Infraestructura: fundamentos conceptuales y justificación del stack.....	22
2.4.1 Contenedores Docker y reproducibilidad.....	22
2.4.2 PostgreSQL como base de datos de métricas	23
2.4.3 Grafana para observabilidad operativa	23
2.5 Estado del arte	23

2.5.1 Chatbots en la atención al cliente de comercio electrónico	24
2.5.2 Clasificación de intenciones con modelos de lenguaje	24
2.5.3 Automatización de procesos en pequeñas y medianas empresas	25
2.5.4 Antecedentes latinoamericanos y del caso argentino	25
2.5.5 Vacío identificado y contribución de este trabajo	27
2.6 Tratamiento de datos personales y marco legal aplicable	28
CAPÍTULO 3: MARCO METODOLÓGICO.....	31
3.1 Tipo de investigación	31
3.2 Enfoque y diseño: estudio de caso instrumental.....	31
3.3 Metodología de desarrollo	32
3.4 Herramientas y tecnologías	33
3.5 Técnicas de recolección y análisis de datos	34
3.5.1 Recolección automatizada de métricas.....	34
3.5.2 Justificación del tamaño de muestra	34
3.5.3 Criterio de evaluación de la exactitud	35
3.5.4 Análisis estadístico	36
3.5.5 Medición del baseline de atención manual	37
3.6 Amenazas a la validez.....	39
3.6.1 Validez de constructo.....	39
3.6.2 Validez interna	39
3.6.3 Validez externa	39
3.6.4 Confiabilidad	39
CAPÍTULO 4: DESARROLLO DEL ESTUDIO	41
4.1 Arquitectura del sistema.....	41
4.1.1 Servicios Docker	41
4.2 Base de datos	42
4.2.1 Tablas principales.....	42
4.2.2 Diagrama entidad-relación	44
4.2.3 Vistas de métricas.....	44
4.2.4 Datos seed	45
4.3 Flujo 1 — Pipeline de procesamiento de órdenes.....	46
4.3.1 Nodos del workflow.....	46
4.3.2 Diagrama del flujo	48

4.3.3 Métricas MTTD y MTTR.....	49
4.3.4 Flujo de estados de una orden	50
4.4 Flujo 2 — Chatbot con IA.....	50
4.4.1 Nodos del workflow.....	51
4.4.2 Diagrama del flujo	52
4.4.3 Prompt de IA.....	54
4.4.4 Métrica TMR.....	54
4.5 Configuración de Grafana	54
4.5.1 Paneles de los dashboards	55
4.6 Testing.....	57
4.6.1 Pruebas funcionales	58
4.6.2 Pruebas de carga y de concurrencia	59
CAPÍTULO 5: RESULTADOS Y ANÁLISIS.....	61
5.1 Resultados del Flujo 1 — Pipeline de Órdenes	61
5.1.1 Pruebas funcionales	61
5.1.2 Métricas de tiempo	61
5.1.3 Pruebas de carga y de concurrencia	62
5.1.4 Baseline de atención manual	63
5.2 Resultados del Flujo 2 — Chatbot (canal simulado formato WhatsApp y canal Telegram real)	65
5.2.1 Pruebas funcionales del chatbot.....	65
5.2.2 Tiempos de respuesta (TMR)	66
5.2.3 Precisión de clasificación de intents	67
5.3 Contrastación de hipótesis	69
5.4 Análisis comparativo y discusión.....	71
5.4.1 Limitaciones observadas	72
CAPÍTULO 6: CONCLUSIONES.....	74
6.1 Respuesta a la pregunta de investigación.....	74
6.2 Cumplimiento de objetivos específicos	74
6.3 Conclusiones sustantivas	77
6.4 Limitaciones del estudio	78
CAPÍTULO 7: RECOMENDACIONES.....	79
7.1 Recomendaciones para producción	79

7.2 Líneas futuras	81
CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS	83
CAPÍTULO 9: ANEXOS.....	87
Anexo A: Resumen del Schema de Base de Datos	87
Anexo B: Configuración Docker Compose	93
Anexo C: Catálogo de Productos Seed	94
Anexo D: FAQ predefinidas (base de conocimiento completa)	95
Anexo E: Comandos de Testing	98
Anexo F: Lista de Figuras — Implementación de Desarrollo (SIMPLE)	99
Anexo G: Propuesta de Arquitectura de Producción	100
Anexo H: Prompt de sistema del clasificador de intenciones	101
Anexo I: Baseline de atención manual — tiempos cronometrados.....	102
Anexo J: Corpus de evaluación y procedimiento de cálculo estadístico	104

Listado de Figuras

Figura	Descripción	Sección
Figura 1	Arquitectura del sistema. Cuatro servicios en contenedor sobre una red interna, con las dos dependencias externas de las que depende el Flujo 2: la API de OpenAI (GPT-4o-mini) y la API de Telegram.	4.1.1
Figura 2	Diagrama entidad-relación de la base de datos ecommerce_tesis, con las siete tablas, sus claves foráneas y la columna data_source que separa los registros medidos de los datos de carga inicial.	4.2.2
Figura 3	Diagrama del Flujo 1 — Pipeline de procesamiento de órdenes. Elaboración propia a partir del workflow implementado en n8n.	4.3.2

Figura	Descripción	Sección
Figura 4	Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.	4.3.2
Figura 5	Diagrama del Flujo 2 — Chatbot con clasificación de intenciones. Elaboración propia a partir del workflow implementado en n8n.	4.4.2
Figura 6	Canvas del Flujo 2 en n8n, mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por intent y el nodo Router Canal que envía la respuesta por el canal de origen.	4.4.2
Figura 7	Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1. Los paneles de indicadores y de distribución de estados se restringen a los registros medidos; los de «Órdenes totales» y «Órdenes procesadas por día» no lo hacen, de modo que el primer punto de la serie diaria corresponde a las órdenes de carga inicial y no a una corrida experimental (Sección 4.5).	4.5.1
Figura 8	Bandeja de Mailpit con los correos generados por el Flujo 1 durante las pruebas: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com).	4.6.2
Figura 9	Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la precisión de clasificación y la distribución de intenciones sobre el corpus evaluado.	5.2.3

Figura	Descripción	Sección
Figura 10	Canvas del Flujo 1 en su variante de producción, con los nodos de integración con plataformas de e-commerce reales.	7.1
Figura 11	Canvas del Flujo 2 en su variante de producción, con los triggers de Telegram, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025).	7.1

Listado de Tablas principales

Tabla	Descripción	Sección
Tabla 2.1	Posición de este trabajo respecto de los antecedentes revisados	2.5.4
Tabla 3.1	Etapas de desarrollo y entregables	3.3
Tabla 3.2	Herramientas y tecnologías utilizadas	3.4
Tabla 3.3	Datos recolectados y su método de registro	3.5.1
Tabla 4.1	Servicios Docker del sistema	4.1.1
Tabla 4.2	Esquema de tablas de la base de datos	4.2.1
Tabla 4.3	Vistas de métricas en PostgreSQL	4.2.3
Tabla 4.4	Catálogo de productos de prueba	4.2.4
Tabla 4.5	Nodos del workflow del Flujo 1	4.3.1
Tabla 4.6	Estados de una orden y nodo que los asigna	4.3.4
Tabla 4.7	Nodos del workflow del Flujo 2	4.4.1
Tabla 4.8	Paneles de los dashboards de Grafana	4.5.1
Tabla 4.9	Pruebas funcionales del Flujo 1	4.6.1
Tabla 4.10	Pruebas funcionales del Flujo 2	4.6.1
Tabla 5.1	Resultados de las pruebas funcionales	5.1.1
Tabla 5.2	Métricas de tiempo del Flujo 1	5.1.2
Tabla 5.3	Resultados de las pruebas de carga y de concurrencia	5.1.3
Tabla 5.4	Descriptivos del tiempo de procesamiento manual, por fase y total	5.1.4

Tabla	Descripción	Sección
Tabla 5.5	Tiempo medio de respuesta por intención sobre el corpus evaluado	5.2.2
Tabla 5.6	Tiempo medio de respuesta por intención en el canal Telegram	5.2.2
Tabla 5.7	Exhaustividad (recall) de clasificación por intención	5.2.3
Tabla 5.8	Matriz de confusión del clasificador	5.2.3
Tabla 5.9	Precisión, exhaustividad y F1 por clase	5.2.3
Tabla 5.10	Contrastación de hipótesis	5.3
Tabla 6.1	Cumplimiento de objetivos específicos	6.2
Tabla I.1	Tiempos cronometrados del procesamiento manual, por orden y por fase	Anexo I
Tabla I.2	Registros en la base de datos de las órdenes procesadas manualmente	Anexo I

Glosario de siglas y acrónimos

API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
BPA	Business Process Automation (Automatización de Procesos de Negocio)
BPM	Business Process Management (Gestión de Procesos de Negocio)
CACE	Cámara Argentina de Comercio Electrónico
CRM	Customer Relationship Management (Gestión de Relaciones con Clientes)
DDL	Data Definition Language (Lenguaje de Definición de Datos)

ERD	Entity-Relationship Diagram (Diagrama Entidad-Relación)
ERP	Enterprise Resource Planning (Planificación de Recursos Empresariales)
FAQ	Frequently Asked Questions (Preguntas Frecuentes)
FK	Foreign Key (Clave Foránea)
GPT	Generative Pre-trained Transformer
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
ITIL	Information Technology Infrastructure Library
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
MTTD	Mean Time To Detect (Tiempo Medio de Detección)
MTTR	Mean Time To Resolve (Tiempo Medio de Resolución)
PyME	Pequeña y Mediana Empresa
REST	Representational State Transfer
SGBD	Sistema de Gestión de Base de Datos
SKU	Stock Keeping Unit (Unidad de Mantenimiento de Inventario)
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language

I

TMR	Tiempo Medio de Respuesta
URL	Uniform Resource Locator
YAML	YAML Ain't Markup Language
UTN	Universidad Tecnológica Nacional

CAPÍTULO 1: INTRODUCCIÓN

1.1 Contexto del problema

El comercio electrónico en Argentina creció significativamente en los últimos años. Cada vez más PyMEs venden a través de plataformas como Tiendanube, WooCommerce o MercadoShops. Sin embargo, muchas de estas empresas siguen gestionando el ciclo post-venta de forma manual: un operador revisa los pedidos, verifica el stock, envía correos de confirmación uno por uno y responde consultas por WhatsApp, email y redes sociales sin ninguna herramienta que centralice esas interacciones. En efecto, en 2024 la facturación nominal del sector, medida en pesos corrientes, creció un 181 % interanual; dado que ese período transcurrió bajo inflación de tres dígitos, la cifra no es interpretable por sí sola como expansión real del volumen comercializado y se consigna únicamente como indicador de la magnitud nominal del mercado. El dato que sí describe la penetración del canal es de naturaleza no monetaria: para el 54 % de las empresas relevadas el canal online ya representa más del 10 % de sus ventas, frente al 50 % en 2023 (Cámara Argentina de Comercio Electrónico [CACE], 2025).

El ciclo post-venta abarca todo lo que ocurre después de que el cliente confirma su compra: confirmación de la orden, verificación de stock, notificaciones de estado y atención de consultas o reclamos. En una PyME con equipo reducido, estas tareas consumen tiempo y son propensas a errores cuando se hacen manualmente.

1.2 Planteamiento del problema

Se identifican tres problemas concretos en la operación post-venta de PyMEs de e-commerce:

1. **Procesamiento manual de órdenes.** Cada pedido requiere que un operador lo revise, consulte stock, actualice estados y redacte un correo de confirmación. Esto introduce demoras y errores como confirmar pedidos sin stock disponible. Estas demoras son críticas en un contexto donde el 82 % de los clientes espera una resolución inmediata de sus consultas (HubSpot, 2024). Corresponde señalar que esa cifra proviene de un reporte de industria elaborado sobre una encuesta comercial y no de una publicación con revisión por pares, de modo que se la consigna como indicio del contexto de expectativas y no como evidencia arbitrada.
2. **Atención al cliente fragmentada.** Un mismo cliente puede escribir por WhatsApp, mandar un email y dejar un mensaje en Telegram. Sin integración entre canales, el operador pierde contexto, responde tarde o da información inconsistente.

3. **Ausencia de métricas.** Sin indicadores como el tiempo que tarda el sistema en detectar una orden o en responder un mensaje, no hay forma de medir el rendimiento ni identificar cuellos de botella.

1.3 Justificación

La automatización del ciclo post-venta se justifica desde tres perspectivas:

Operativa. Automatizar tareas repetitivas como la verificación de stock y el envío de notificaciones libera al equipo para tareas que requieren criterio humano. Para una PyME que procesa decenas de órdenes por día, la diferencia en horas de trabajo es significativa.

Experiencia del cliente. Un pipeline automatizado permite confirmaciones inmediatas, notificaciones proactivas y respuestas sin dependencia del horario de un operador. El cliente recibe atención consistente sin depender de la disponibilidad de un operador. Diversos estudios señalan que los asistentes conversacionales basados en IA mejoran la calidad de servicio percibida en la atención al cliente (Misischia et al., 2022).

Tecnológica. Herramientas como n8n, PostgreSQL, Docker y los modelos de lenguaje de OpenAI permiten implementar soluciones de automatización sofisticadas a bajo costo, utilizando software de código abierto y APIs accesibles.

1.4 Pregunta de Investigación e Hipótesis

1.4.1 Pregunta de investigación

¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de las métricas MTTD (tiempo de detección y procesamiento de órdenes), MTTR (tiempo de resolución y notificación) y TMR (tiempo de respuesta del chatbot)?

1.4.2 Hipótesis

H1: El pipeline automatizado de procesamiento de órdenes reduce de manera sustancial el tiempo end-to-end del ciclo —desde la recepción del pedido hasta la notificación al cliente— respecto del tiempo que insume procesar la misma tarea de forma manual, medido como baseline propio según el protocolo de la Sección 3.5.5. Como criterio operativo secundario se fija que dicho tiempo se mantenga por debajo de 30 segundos.

H2a: El chatbot basado en GPT-4o-mini responde al cliente en un tiempo medio de respuesta (TMR) inferior a 10 segundos para consultas de tipo FAQ, estado de pedido y consultas generales.

H2b: El chatbot clasifica correctamente la intención del mensaje en al menos el 85 % de los casos. A lo largo del trabajo se reserva «exactitud (accuracy)» para esta métrica global y «precisión» para la métrica por clase de la Tabla 5.9, conforme la terminología de Jurafsky y Martin (2024). Este umbral se fija como criterio del equipo —no deriva de un estándar publicado ni de un requerimiento de negocio externo— y se justifica en la Sección 2.2.3.

Las tres hipótesis se contrastan en el Capítulo 5 a partir de los datos recolectados durante las pruebas funcionales, la corrida de carga secuencial y la prueba de concurrencia. Corresponde señalar de antemano una precisión epistemológica: los umbrales de 30 y de 10 segundos operan como criterios de aceptación fijados por el equipo y no constituyen, por sí mismos, afirmaciones falsables. Dado el diseño del sistema —persistencia sobre una base de datos local y notificación a un capturador SMTP alojado en el mismo host— era previsible que ambos se cumplieran con amplio margen. La afirmación sustantiva y efectivamente refutable de H1 es la comparación contra el baseline de atención manual medido en este trabajo, y es en esos términos que se la somete a prueba en la Sección 5.3.

1.5 Objetivos

1.5.1 Objetivo General

Diseñar e implementar un pipeline automatizado de procesamiento de órdenes y un chatbot con inteligencia artificial para el ciclo post-venta de e-commerce, orquestados con n8n y desplegados en Docker, midiendo su desempeño mediante las métricas MTTD, MTTR y TMR, y contrastando los resultados con la hipótesis de reducción sustancial de los tiempos operativos respecto al proceso manual.

1.5.2 Objetivos Específicos

1. Implementar un pipeline de procesamiento de órdenes (15 nodos en n8n) que reciba pedidos vía webhook, verifique stock en PostgreSQL, actualice estados y notifique al cliente por email, registrando MTTD y MTTR; y verificar que el tiempo end-to-end sea inferior a 30 segundos (H1).
2. Implementar un chatbot multicanal sobre dos canales —WhatsApp, con envío simulado a un capturador SMTP local, y Telegram, con envío real— que normalice el mensaje entrante, clasifique la intención con GPT-4o-mini (FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL) y responda de forma automatizada registrando el TMR; y verificar que ese TMR sea inferior a 10 segundos y que la precisión de clasificación supere el 85 % (H2a y H2b). La extensión a un tercer canal por correo electrónico queda planteada desde el inicio como línea futura, y su diseño se documenta en el Capítulo 7.
3. Diseñar un esquema de base de datos en PostgreSQL con las tablas, vistas de métricas e índices necesarios para instrumentar el ciclo post-venta y sostener el cálculo automático de MTTD, MTTR y TMR.

4. Configurar dashboards en Grafana conectados a PostgreSQL para visualizar MTTD, MTTR y TMR en tiempo real.
5. Validar el funcionamiento del sistema mediante pruebas funcionales con datos simulados, verificando que los flujos operan según lo diseñado y las métricas se registran correctamente.

1.6 Alcance y Limitaciones

1.6.1 Alcance

El proyecto implementa los dos flujos de automatización en un entorno local basado en Docker Compose, con cuatro servicios: n8n, PostgreSQL 15, Mailpit y Grafana. Los datos son simulados: órdenes enviadas por HTTP al webhook y conversaciones iniciadas desde cuentas de prueba.

El alcance funcional incluye:

- Pipeline de órdenes: recepción → verificación de stock → confirmación o rechazo → notificación por email → registro de métricas.
- Chatbot: recepción → normalización → clasificación de intent con IA → respuesta automatizada → registro de métricas.
- Dashboards de métricas operativas en Grafana.

1.6.2 Limitaciones

- **Costo de API de OpenAI:** el modelo GPT-4o-mini tiene un costo por token. En desarrollo se trabaja con volúmenes reducidos de mensajes.
 - **WhatsApp Business API:** requiere una cuenta verificada por Meta, verificación que no se completó dentro del alcance de este trabajo. En consecuencia el canal de WhatsApp no se ejecutó contra la API real ni contra su entorno sandbox: las pruebas del Flujo 2 sobre ese canal se realizaron entregando la respuesta a un capturador SMTP local, de modo que las latencias reportadas para él no incluyen componente alguno de la red de WhatsApp. La validación sobre un canal de mensajería real se resolvió mediante Telegram (Sección 5.2.2).
 - **Entorno local:** el sistema no se despliega en nube. Alta disponibilidad y escalabilidad horizontal quedan fuera del alcance.
 - **Datos simulados:** las pruebas usan datos ficticios que representan escenarios típicos pero no la variabilidad completa de un entorno real.
-

CAPÍTULO 2: MARCO TEÓRICO

2.1 Automatización de procesos de negocio

2.1.1 Definición y antecedentes

La Automatización de Procesos de Negocio (BPA, *Business Process Automation*) se define como el uso de tecnología para ejecutar tareas y flujos de trabajo recurrentes con el fin de aumentar la eficiencia, reducir el error humano y estandarizar los resultados (Dumas et al., 2018). En la literatura de gestión de procesos (*Business Process Management*, BPM), la automatización constituye uno de los cuatro niveles de madurez de proceso, junto con la documentación, la optimización y la gobernanza (van der Aalst, 2016).

En el contexto del comercio electrónico, la automatización de procesos post-venta se enmarca dentro de lo que Turban et al. (2018) denominan *fulfillment automation*: la capacidad de ejecutar el ciclo completo desde la recepción del pedido hasta la notificación al cliente sin intervención humana. En PyMEs de e-commerce con equipo operativo reducido, las tareas manuales del ciclo post-venta —verificación de stock, actualización de estados, envío de notificaciones y atención de consultas— representan una fracción significativa de la carga de trabajo diaria, lo que constituye una oportunidad concreta de mejora mediante automatización (Laudon & Traver, 2021).

2.1.2 Herramientas de orquestación de flujos: n8n

Para la implementación de este proyecto se seleccionó **n8n** como motor de orquestación. n8n es una plataforma de automatización de flujos de trabajo de código abierto y auto-alojable (*self-hosted*) cuya arquitectura se basa en *workflows* compuestos por nodos interconectados, donde cada nodo ejecuta una operación atómica: recibir un webhook, consultar una base de datos, llamar una API externa o enviar un email (n8n GmbH, 2025).

La elección de n8n sobre alternativas comerciales como Zapier o Make.com se fundamenta en tres criterios: (1) **modelo de costos**: n8n es gratuito en modalidad self-hosted, eliminando los costos de licencia que representan una barrera para PyMEs con presupuesto acotado; (2) **soberanía de datos**: al ejecutarse en infraestructura propia, la persistencia de los datos de clientes y la totalidad del procesamiento del Flujo 1 permanecen bajo control del operador, sin transmitirse a servidores de terceros. Corresponde precisar el alcance de este criterio, que no se extiende a la inferencia del Flujo 2: ese flujo remite el texto del mensaje del cliente a la API de OpenAI en cada interacción, dependencia que se discute como limitación en la Sección 5.4.1 y que motiva la recomendación de evaluar modelos de lenguaje de ejecución local del Capítulo 7; (3) **extensibilidad**: n8n permite ejecutar código JavaScript/Python arbitrario en nodos *Function*, lo que posibilita lógica de

transformación compleja sin salir del entorno visual, a diferencia de las plataformas comerciales equivalentes, que operan como servicios gestionados de código cerrado (n8n GmbH, 2025).

2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce

Las métricas **MTTD** (*Mean Time To Detect*) y **MTTR** (*Mean Time To Resolve*) tienen origen en la disciplina de gestión de incidentes de TI (Axelos, 2019) y en el movimiento DevOps (Kim et al., 2016). En el contexto de este trabajo se las adapta al ciclo de procesamiento de órdenes:

- **MTTD** mide el tiempo transcurrido desde la recepción de una orden hasta que el sistema detecta y procesa el stock disponible. Equivale al tiempo de procesamiento interno del pipeline.
- **MTTR** mide el tiempo desde que el stock es procesado hasta que el cliente recibe la notificación de confirmación o rechazo. Equivale al tiempo de resolución y comunicación.

La adaptación de estas métricas a procesos de e-commerce es coherente con la práctica de operational excellence documentada por Womack & Jones (2003) en el contexto de manufactura esbelta, donde la reducción del tiempo de ciclo es un indicador central de eficiencia operativa.

Resulta pertinente discutir la transferibilidad conceptual de MTTD y MTTR desde su dominio de origen — la gestión de incidentes TI — al dominio del procesamiento de órdenes de e-commerce. En ITIL, ambas métricas miden eventos asociados a fallas (un incidente es por definición un evento no deseado). En el contexto de este trabajo, una orden de compra exitosa no constituye una falla sino un flujo nominal del proceso de negocio. La transferencia se sostiene, sin embargo, a nivel estructural: ambas métricas miden intervalos temporales acotados por dos eventos observables y representan etapas secuenciales de un ciclo (detección/procesamiento y resolución/comunicación). Esta analogía estructural — y no semántica estricta — es coherente con la práctica documentada en la literatura de operational excellence, donde indicadores originalmente formulados para entornos de manufactura (lead time, cycle time) se transfieren a procesos administrativos y de servicio sin pérdida de validez operativa (Womack & Jones, 2003). Se reconoce que métricas alternativas más específicas del dominio comercial — como order cycle time u order fulfillment lead time — podrían reemplazar a MTTD y MTTR en versiones futuras del trabajo. La elección de MTTD y MTTR en este estudio se fundamenta en su disponibilidad inmediata como vocabulario de observabilidad ya soportado por las herramientas de monitoreo del stack tecnológico (Grafana), lo que facilita la integración con prácticas de site reliability engineering (Beyer et al., 2016) habituales en equipos de desarrollo.

2.2 Inteligencia artificial conversacional y modelos de lenguaje

2.2.1 Evolución de los chatbots: de reglas a LLMs

Los sistemas de diálogo automatizado evolucionaron desde los chatbots basados en reglas (*rule-based*) de los años 1990 —donde cada posible interacción debe ser explícitamente programada— hacia los sistemas basados en recuperación de información (*retrieval-based*) y, finalmente, hacia los modelos de lenguaje generativos (*generative*) (Jurafsky & Martin, 2024). Los chatbots basados en reglas presentan limitaciones conocidas: frágiles ante variaciones ortográficas, incapaces de manejar intenciones no anticipadas, y costosos de mantener a medida que el catálogo de respuestas crece (McTear, 2020).

Los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) resuelven estas limitaciones al aprender representaciones estadísticas del lenguaje natural a partir de corpus masivos (Brown et al., 2020). Su capacidad de generalización permite comprender consultas variadas y generar respuestas coherentes sin requerir la codificación explícita de cada caso.

2.2.2 GPT-4o-mini y prompt engineering

En este proyecto se utiliza GPT-4o-mini de OpenAI (OpenAI, 2024), un modelo optimizado para inferencia rápida con buen balance entre capacidad y costo. La interacción con el modelo se configura mediante prompt engineering estructurado: el prompt de sistema define el rol del modelo (agente de atención al cliente de un e-commerce), las categorías de intención válidas (FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL), el formato de respuesta esperado (un objeto JSON estricto, sin texto ni marcado alrededor) y las reglas de idioma y de tono.

Corresponde precisar el régimen de prompting empleado, porque la distinción tiene consecuencias sobre la interpretación de los resultados. Brown et al. (2020) diferencian tres regímenes según la cantidad de ejemplos etiquetados que se incluyen en el prompt: zero-shot, donde el modelo recibe únicamente la instrucción y la definición de las clases; one-shot, con un ejemplo; y few-shot, con varios ejemplos de entrada y salida. Liu et al. (2023) sistematizan el conjunto de estas técnicas. El clasificador de este trabajo opera en régimen zero-shot: el prompt de sistema —que se transcribe íntegramente en el Anexo H— enumera las cuatro categorías y fija el formato de salida, pero no incluye ningún ejemplo etiquetado. La elección no fue deliberada en su origen, sino que se constató al auditar la implementación; se la reporta como tal porque condiciona la lectura del resultado: el accuracy obtenido corresponde al desempeño del modelo sin ejemplos ni ajuste fino, lo que constituye un piso y no un techo para la tarea.

2.2.3 Precisión en clasificación de intents

La evaluación de la precisión de clasificación de intenciones en chatbots se realiza típicamente mediante métricas de clasificación estándar: accuracy, precision, recall y F1-score (Jurafsky & Martin, 2024). En este trabajo se emplea accuracy como métrica principal por su interpretabilidad directa, y se la acompaña —siguiendo la práctica habitual cuando las clases no son estrictamente equiprobables— del desglose por clase de precisión, exhaustividad y F1, además de la matriz de confusión completa (véase §5.2.3). El conjunto de prueba no está perfectamente balanceado: la razón entre la clase más numerosa y la menos numerosa es de 1,92 a 1 (48 y 25 mensajes sobre un total de 150), un desbalance moderado que se reporta explícitamente para que el lector pueda ponderar el accuracy global a la luz del desempeño por clase. Se adopta un umbral mínimo del 85 % de accuracy como criterio de aceptación definido por el equipo. Ese valor no deriva de un estándar publicado, y conviene explicitar el razonamiento que lo sustenta. La literatura revisada en la Sección 2.5 muestra que la clasificación de intenciones sin ajuste fino alcanza desempeño útil: Parikh et al. (2023) documentan que el prompting zero-shot basado en descripciones de intención resulta efectivo, y Luo et al. (2023) obtienen desempeño competitivo con cantidades mínimas de datos. Esos trabajos evalúan sobre corpus normalizados con decenas de intenciones, mientras que el dominio de este trabajo se limita a cuatro categorías, lo que vuelve la tarea comparativamente más sencilla. Por esa razón se fija un umbral exigente para el nivel del prototipo, sin pretender que sea equiparable a los valores reportados sobre aquellos corpus.

2.3 Comunicación multicanal en e-commerce

2.3.1 Multicanalidad y omnicanalidad: delimitación conceptual

Conviene distinguir con precisión dos estrategias que suelen emplearse como sinónimos y no lo son. La estrategia multicanal consiste en poner a disposición del cliente varios canales de comunicación —WhatsApp, Telegram, correo electrónico, redes sociales— atendidos por un mismo sistema de procesamiento, pero donde cada conversación se resuelve de forma independiente y no se conserva contexto entre canales. La estrategia omnicanal (omnichannel) va un paso más allá: integra esos canales en una experiencia unificada, de manera que el historial y el contexto del cliente se preserven con independencia del canal utilizado (Verhoef et al., 2015), en oposición al enfoque en el que cada canal opera en silos. La diferencia operativa entre ambas es concreta y verificable: la omnicanalidad exige identidad unificada de cliente entre canales y recuperación del historial conversacional previo.

Corresponde declarar con precisión el alcance de lo implementado en este trabajo. El sistema desarrollado es multicanal: recibe y responde por WhatsApp y por Telegram a través de un mismo flujo de procesamiento, con un único clasificador y una única base de datos. Sin embargo, la tabla interactions no registra identificador de conversación ni identidad unificada de cliente entre canales, y ningún nodo del

Flujo 2 recupera interacciones previas al construir la respuesta: cada mensaje se atiende sin memoria del anterior. La omnicanalidad en sentido estricto —identidad unificada y memoria conversacional entre canales— no fue implementada, y se declara como línea futura en el Capítulo 7. En el segmento de PyMEs de e-commerce argentinas, donde la atención suele distribuirse entre WhatsApp y correo electrónico, esa evolución constituye el paso natural del prototipo.

2.3.2 TMR como métrica de efectividad

El TMR (Tiempo Medio de Respuesta) es la métrica operativa central para evaluar la efectividad de un sistema de atención. En la literatura de calidad de servicio, la capacidad de respuesta (responsiveness) es una de las cinco dimensiones del modelo SERVQUAL (Zeithaml et al., 1990); si bien ese modelo es anterior a la existencia de los canales digitales, la dimensión se traslada a este contexto como el tiempo que el cliente espera hasta recibir una respuesta. Reportes de la industria señalan que los clientes esperan respuestas cada vez más rápidas —el 82% espera una resolución inmediata de sus consultas (HubSpot, 2024)—, muy por debajo de los tiempos que caracterizan a la atención manual. Un sistema automatizado con TMR de segundos representa, por tanto, una mejora de varios órdenes de magnitud respecto al estándar manual.

2.4 Infraestructura: fundamentos conceptuales y justificación del stack

Corresponde declarar de antemano la función de esta sección, porque difiere de la de las tres anteriores. Las Secciones 2.1 a 2.3 construyen los conceptos que el trabajo necesita para formular sus hipótesis; esta, en cambio, cumple una función doble: expone las nociones conceptuales que gobiernan la infraestructura —reproducibilidad computacional, modelo relacional y observabilidad operativa— y, junto a cada una, la decisión de herramienta que el trabajo tomó apoyándose en ella. Esa segunda mitad es de naturaleza metodológica antes que teórica, y podría haberse ubicado en el Capítulo 3. Se la mantiene aquí para que cada elección quede junto al concepto que la fundamenta; el detalle operativo del stack se consigna en la Sección 3.4 y su configuración concreta en el Capítulo 4.

2.4.1 Contenedores Docker y reproducibilidad

Docker es una plataforma de virtualización a nivel de sistema operativo que empaqueta aplicaciones y sus dependencias en unidades portables denominadas *contenedores* (Merkel, 2014). Docker Compose extiende este concepto permitiendo definir y orquestar múltiples contenedores mediante un único archivo de configuración YAML (Docker Inc., 2025). La adopción de contenedores en proyectos de investigación aplicada es una práctica recomendada para garantizar la reproducibilidad del entorno experimental (Boettiger, 2015): cualquier evaluador

puede levantar el sistema completo con un único comando, eliminando la variabilidad del entorno como fuente de error.

2.4.2 PostgreSQL como base de datos de métricas

PostgreSQL 15 se seleccionó como sistema de gestión de base de datos relacional por tres razones: (1) soporte completo de SQL estándar incluyendo window functions necesarias para el cálculo de métricas temporales; (2) soporte nativo de tipos JSON para almacenar payloads sin procesar de los webhooks; (3) capacidad de crear vistas (views) que Grafana consume directamente sin necesidad de ETL adicional (PostgreSQL Global Development Group, 2025).

2.4.3 Grafana para observabilidad operativa

Grafana es una plataforma de visualización y observabilidad de código abierto que permite crear dashboards interactivos conectados directamente a fuentes de datos como PostgreSQL, InfluxDB o Prometheus (Grafana Labs, 2025). En el contexto de este trabajo, Grafana actúa como capa de observabilidad del pipeline: los paneles de métricas MTTD, MTTR y TMR permiten verificar en tiempo real que el sistema opera dentro de los rangos esperados, alineándose con las prácticas de site reliability engineering documentadas por Beyer et al. (2016).

2.5 Estado del arte

Para situar la contribución de este trabajo corresponde revisar qué se ha publicado sobre el problema abordado. Se declara en primer término el procedimiento seguido, de modo que tanto el alcance de la revisión como sus omisiones resulten atribuibles.

Procedimiento de búsqueda. Se consultaron ACL Anthology, arXiv, Crossref y los catálogos en línea de ScienceDirect y Springer Nature para la literatura internacional, y SciELO, Redalyc, Dialnet y los repositorios institucionales de universidades nacionales argentinas —entre ellos el repositorio del CONICET— para la literatura regional arbitrada. Las cadenas combinaron los términos chatbot, customer service, e-commerce, intent classification, large language model, zero-shot, business process automation, robotic process automation, SME y order fulfillment, en inglés, con sus equivalentes en español para las búsquedas de alcance regional. La ventana temporal se fijó entre 2018 y 2026, con dos excepciones declaradas: las obras canónicas ya incorporadas al marco teórico, y el estudio de Alderete et al. (2017) sobre adopción de comercio electrónico en pymes de Córdoba, que se incluye por ser el trabajo empírico de referencia sobre el caso nacional específico que este trabajo sitúa como contexto. La identificación bibliográfica de cada antecedente —autores, revista, volumen, páginas y DOI— fue verificada contra el registro de Crossref o contra el texto publicado. Se incluyeron trabajos empíricos que reportaran métricas cuantitativas, revisiones sistemáticas y preprints de laboratorios o conferencias reconocidas; se excluyeron material de divulgación, documentación de producto y publicaciones sin proceso de revisión.

Limitación declarada. La revisión que sigue no constituye una revisión sistemática en el sentido de un protocolo formal de cribado con doble evaluador: es una revisión de antecedentes acotada, proporcionada al nivel de un trabajo integrador de tecnicatura. Se la presenta como tal y no como hallazgo exhaustivo del campo.

2.5.1 Chatbots en la atención al cliente de comercio electrónico

Ngai et al. (2021) presentan un chatbot basado en conocimiento para atención al cliente, construido sobre una representación explícita del dominio antes de la difusión de los modelos de lenguaje de gran escala. Su trabajo resulta pertinente como línea de base arquitectónica: muestra el costo de construir y mantener la base de conocimiento que un sistema de reglas exige, costo que es precisamente el que un modelo de lenguaje permite evitar. Misischia et al. (2022), por su parte, abordan la cuestión desde la calidad de servicio antes que desde la técnica: identifican las funciones del chatbot relevantes para el cliente y sostienen que atributos como un estilo de interacción socialmente orientado y un trato empático inciden sobre la calidad percibida de la atención. Ambos trabajos coinciden en un punto que este trabajo asume como supuesto de diseño: la utilidad del chatbot no se agota en la exactitud de su clasificación, sino que depende también de la calidad de la respuesta que entrega.

2.5.2 Clasificación de intenciones con modelos de lenguaje

Parikh et al. (2023) evalúan cuatro estrategias de bajo consumo de datos para clasificación de intenciones —adaptación de dominio, aumento de datos, prompting zero-shot con modelos de lenguaje y ajuste fino de parámetros eficientes— y concluyen que el ajuste fino eficiente sobre Flan-T5 obtiene el mejor desempeño incluso con un único ejemplo por intención, al tiempo que confirman que el prompting zero-shot basado en descripciones de las intenciones resulta efectivo para la tarea. Luo et al. (2023) llegan a una conclusión convergente por otro camino: muestran que el aprendizaje basado en prompts permite a modelos de lenguaje pequeños alcanzar un desempeño competitivo en reconocimiento de intenciones de cliente con una cantidad mínima de datos de entrenamiento, y que un prompt bien diseñado habilita desempeño útil en escenarios zero-shot.

Estos antecedentes son los que dan sentido al régimen zero-shot que emplea este trabajo (Sección 2.2.2) y permiten interpretar su resultado: el accuracy obtenido no es un valor aislado sino un punto dentro de un rango que la literatura ya documenta como alcanzable sin ajuste fino. Corresponde señalar, sin embargo, una diferencia de alcance que impide la comparación directa de cifras: los trabajos citados evalúan sobre corpus públicos normalizados —del tipo de CLINC150, Banking77 o SNIPS—, con decenas o centenas de intenciones, mientras que este trabajo opera sobre cuatro categorías de un dominio deliberadamente acotado. Un accuracy más alto sobre menos clases no constituye un mejor resultado, y este trabajo no lo presenta como tal.

2.5.3 Automatización de procesos en pequeñas y medianas empresas

Pyłacz y Žukovskis (2023) estudian la implementación de automatización robótica de procesos en pequeñas y medianas empresas y sus implicancias organizacionales, y documentan que la adopción efectiva permanece muy por debajo del interés declarado. Su diagnóstico sobre las barreras —infraestructura digital limitada, brechas de competencias y procedimientos poco formalizados— es coherente con el problema que motiva este trabajo y respalda empíricamente la elección de una herramienta de bajo costo y auto-alojable. En el extremo opuesto de la escala organizacional, Zhang et al. (2021) analizan mediante estudio de caso la orquestación de recursos de inteligencia artificial en el centro logístico de Alibaba, y muestran cómo la capacidad emerge de la articulación entre los recursos técnicos y los humanos y organizacionales, y no del componente tecnológico en aislamiento. Ese hallazgo es pertinente aquí en sentido admonitorio: sugiere que las mejoras de tiempo que un prototipo mide en condiciones de laboratorio no se trasladan automáticamente a una organización real.

2.5.4 Antecedentes latinoamericanos y del caso argentino

Los antecedentes revisados hasta aquí describen el estado internacional de cada componente, pero ninguno se sitúa en el contexto que este trabajo declara como problema: el de una pequeña o mediana empresa de comercio electrónico argentina. Corresponde por lo tanto revisar la literatura arbitrada de la región, tanto la que caracteriza ese contexto como la que reporta implementaciones de asistentes conversacionales en él.

Sobre la adopción de comercio electrónico en pymes argentinas, Alderete, Jones y Motta (2017) estiman un modelo probit ordenado sobre 119 pymes de Córdoba y encuentran que la preparación digital —tanto objetiva como percibida—, la educación de los empleados, los beneficios esperados, la calidad de la conexión de banda ancha y el grado de internacionalización inciden significativamente en la probabilidad de adoptar el canal. Alderete y Porris (2023) retoman la cuestión sobre 154 pymes de Bahía Blanca y la vinculan al entorno institucional: las empresas asociadas a la cámara sectorial alcanzan un nivel medio de adopción de 2,11 frente a 1,46 en las radicadas en el parque industrial. Ambos trabajos importan aquí por una razón precisa: sostienen con evidencia local el supuesto que este trabajo asume sobre su destinatario, esto es que la barrera de adopción en el segmento no es de voluntad sino de preparación técnica y de acompañamiento. Es exactamente el supuesto que justifica elegir una herramienta de bajo costo, auto-alojable y de configuración visual.

Sobre la automatización de procesos en la región, Aguirre Mayorga (2022) propone un marco metodológico de cuatro fases que integra gestión de procesos de negocio con design thinking, y lo valida sobre un proceso de certificación del sector eléctrico colombiano, donde documenta reducción de tiempos y digitalización del flujo de

trabajo. Su aporte a este trabajo es metodológico antes que técnico: muestra que en la región la automatización de un proceso se aborda habitualmente desde el rediseño del proceso y no desde la instrumentación de su desempeño, que es el ángulo que aquí se adopta.

Sobre asistentes conversacionales con datos regionales, Ramos De Santis (2024) analiza 1.250 usuarios de chatbots de diez empresas líderes de logística en Colombia, Perú y Ecuador y, mediante regresión múltiple, identifica cinco predictores de la satisfacción del cliente —capacidad de resolver el problema, conocimiento del producto, autonomía en la resolución, corrección gramatical y disposición a recomendar el servicio—, con un coeficiente de determinación ajustado de 0,7512. Ese resultado es directamente pertinente para la lectura del Capítulo 5 de este trabajo, y en un sentido incómodo: cuatro de los cinco predictores que allí explican la satisfacción refieren a la calidad del contenido de la respuesta, que es precisamente la dimensión que este trabajo no midió. Fondevila-Gascón, Huamanchumo, Martín-Guait y Gutiérrez-Aragón (2024), publicados en una revista arbitrada peruana aunque con datos relevados en España —condición que corresponde consignar—, llegan por vía de encuesta a una conclusión convergente: los clientes están dispuestos a interactuar con agentes conversacionales, pero la disposición se revierte cuando la calidad de la respuesta y de la comprensión es pobre.

Sobre implementaciones concretas, Pachas-Santos, Calderón-Vilca y Cárdenas-Mariño (2023) construyen un chatbot de recomendación de productos con una arquitectura multimodal basada en ViLBERT, entrenada sobre Flickr30K y evaluada sobre un conjunto de 20.000 productos, con una exactitud del 80,6 % sobre consultas por imagen y del 75 % sobre consultas por texto. Bravo Maruri, Ramírez Reina, Orozco Lara y Espinoza Martínez (2025) documentan, mediante estudio de caso en una empresa tecnológica ecuatoriana, la integración de un chatbot con inteligencia artificial al sistema de gestión de clientes para automatizar el soporte de primer nivel. Este último es el antecedente regional más próximo a lo que aquí se hace —automatizar la atención de primer nivel de una empresa concreta— y por eso conviene marcar la diferencia de alcance: se trata de un estudio de caso descriptivo, sin baseline cronometrado ni contraste contra un umbral fijado de antemano.

Del conjunto regional se desprenden dos observaciones que ordenan el resto del capítulo. La primera es que la literatura latinoamericana sobre chatbots está fuertemente orientada a la percepción del usuario y a la satisfacción declarada, y mucho menos a la instrumentación del desempeño: se mide qué opina el cliente del asistente, no cuánto tarda el sistema ni con qué exactitud clasifica. La segunda es que los trabajos que sí describen implementaciones lo hacen en clave de estudio de caso descriptivo, sin término de comparación medido. Esa combinación deja disponible el hueco que este trabajo ocupa, y también explica por qué su contribución es de método antes que de magnitud.

2.5.5 Vacío identificado y contribución de este trabajo

La revisión permite delimitar la contribución con precisión, y también su modestia. Los trabajos sobre clasificación de intenciones miden el componente de inteligencia artificial de manera aislada, sobre corpus normalizados y sin integración con un sistema operativo real; los trabajos sobre automatización de procesos en pequeñas y medianas empresas describen la adopción y sus barreras, pero no instrumentan métricas de tiempo extremo a extremo sobre un artefacto concreto; el estudio de caso de fulfillment opera a una escala organizacional que no es la del segmento aquí considerado; y la literatura regional, que sí sitúa el problema en el contexto correcto, mide percepción del usuario o describe implementaciones sin instrumentar tiempos ni contrastarlos contra un término de comparación medido. No se identificaron trabajos que midan, sobre un mismo artefacto y con instrumentación embebida, tanto el tiempo de procesamiento de órdenes como el de respuesta conversacional, y que además comparen ese resultado contra un baseline manual cronometrado por los propios autores.

La Tabla 2.1 sitúa este trabajo frente a los antecedentes revisados. Su contribución no reside en superar a ninguno de ellos en su propia métrica, cosa que no hace ni pretende, sino en articular dos componentes que la literatura suele tratar por separado dentro de un artefacto reproducible, y en medirlos con instrumentación propia contra un término de comparación medido y no supuesto.

Tabla 2.1: Posición de este trabajo respecto de los antecedentes revisados.

Antecedente	Dominio	Enfoque técnico	Qué mide	Entorno
Ngai et al. (2021)	Atención al cliente	Chatbot basado en conocimiento explícito	Desempeño del asistente sobre consultas de clientes	Organización real
Mischia et al. (2022)	Atención al cliente	Revisión de funciones del chatbot	Calidad de servicio percibida	Revisión
Parikh et al. (2023)	Clasificación de intenciones	Zero-shot, aumento de datos y ajuste fino eficiente	Accuracy sobre corpus públicos	Corpus normalizados
Luo et al. (2023)	Intenciones de cliente	Aprendizaje basado en prompts	Accuracy con datos escasos	Corpus de industria
Pyłacz y Žukovskis (2023)	Procesos de PyME	Automatización robótica de procesos	Adopción, barreras e implicancias organizacionales	Estudio de campo

			s	
Zhang et al. (2021)	Fulfillment de e-commerce	Orquestación de recursos con IA	Eficiencia operativa y exactitud de órdenes	Estudio de caso (Alibaba)
Bravo Maruri et al. (2025) · EC	Soporte al cliente de nivel 1	Chatbot con IA integrado al CRM	Descripción de la implementación	Estudio de caso (empresa real)
Pachas-Santos et al. (2023) · MX/PE	Recomendación en e-commerce	Modelo multimodal ViLBERT	Exactitud sobre consultas por imagen y texto	Corpus públicos
Fondevila-Gascón et al. (2024) · PE/ES	Chatbots en atención al cliente	Encuesta cuantitativa	Percepción y actitud del cliente	Encuesta (datos de España)
Ramos De Santis (2024) · CO/PE/EC	Chatbots en logística	Regresión múltiple sobre 1.250 usuarios	Predictores de la satisfacción del cliente	Encuesta a usuarios B2C
Aguirre Mayorga (2022) · CO	Procesos de negocio	BPM integrado con design thinking	Rediseño del proceso y tiempos de ciclo	Estudio de caso (sector eléctrico)
Alderete y Porris (2023) · AR	Adopción de e-commerce en pymes	Análisis descriptivo y ANOVA sobre 154 pymes	Nivel de adopción según vínculo institucional	Relevamiento (Bahía Blanca)
Alderete et al. (2017) · AR	Adopción de e-commerce en pymes	Probit ordenado sobre 119 pymes	Determinantes de la adopción del canal	Encuesta (Córdoba)
Este trabajo	Ciclo post-venta de PyME	Orquestación con n8n y clasificación zero-shot con LLM	MTTD, MTTR, TMR y accuracy, contra baseline manual medido	Prototipo local instrumentado

2.6 Tratamiento de datos personales y marco legal aplicable

El dominio de aplicación de este trabajo es la atención al cliente, y en consecuencia el sistema manipula datos personales: nombre, dirección de correo electrónico y número de teléfono de quien realiza una compra, además del contenido íntegro de los mensajes que esa persona envía. Corresponde por lo tanto explicitar el marco

legal aplicable, aun cuando el prototipo opere sobre datos ficticios y en un entorno local.

En la República Argentina el tratamiento de datos personales se rige por la Ley 25.326 de Protección de los Datos Personales, sancionada en el año 2000. Cinco de sus disposiciones resultan directamente pertinentes al sistema construido. El artículo 5 exige el consentimiento libre, expreso e informado del titular como condición de licitud del tratamiento. El artículo 6 impone el deber de informarle la finalidad de la recolección y los derechos que le asisten. Los artículos 9 y 10 establecen, respectivamente, la obligación de adoptar medidas técnicas y organizativas que garanticen la seguridad de los datos y el deber de confidencialidad de quienes los tratan. Los artículos 14 y 16 reconocen al titular los derechos de acceso, rectificación, actualización y supresión. Y el artículo 12 regula la transferencia internacional de datos, prohibiéndola hacia países que no proporcionen niveles adecuados de protección.

Esta última disposición interpela directamente a la arquitectura del Flujo 2. El sistema remite el texto completo del mensaje del cliente a la interfaz de programación de OpenAI, alojada fuera del territorio nacional, en cada interacción. Se trata de una transferencia internacional de datos personales en el sentido del artículo 12, y un despliegue productivo debería resolverla mediante alguna de las vías que la propia norma y su reglamentación contemplan, o bien evitarla ejecutando el modelo de lenguaje en infraestructura propia, alternativa que este trabajo recomienda evaluar en el Capítulo 7. La contradicción entre esta dependencia y el criterio de soberanía de datos invocado en la Sección 2.1.2 se declara allí de manera explícita.

Existe además un deber que no deriva de la Ley 25.326 sino de la buena fe en la relación de consumo, y que un sistema conversacional automatizado debe atender: informar al cliente que está interactuando con un sistema y no con una persona. El prompt del asistente construido en este trabajo instruye al modelo a adoptar un tono humano y a no presentarse como un chatbot genérico (Anexo H), decisión que optimiza la calidad percibida de la interacción pero que, en un despliegue real, debería acompañarse de una identificación inequívoca del carácter automatizado del canal.

Qué implementa el prototipo y qué faltaría. En su estado actual el sistema opera exclusivamente sobre datos ficticios, generados para las pruebas sobre dominios reservados para documentación, y persiste la totalidad de la información en una base de datos local, de modo que no existe tratamiento de datos personales reales ni riesgo asociado. Un despliegue productivo requeriría, como mínimo: obtener y registrar el consentimiento del titular en el momento de la compra; publicar la finalidad del tratamiento y el procedimiento para ejercer los derechos de acceso, rectificación y supresión; cifrar los datos en tránsito y en reposo; definir un plazo de retención y un procedimiento de supresión que hoy no existen, dado que el esquema

I

conserva las interacciones de forma indefinida; minimizar los datos remitidos al proveedor de inferencia, que actualmente recibe el mensaje íntegro sin depuración previa; y resolver la base legal de la transferencia internacional. Estas obligaciones se incorporan a las recomendaciones del Capítulo 7.

CAPÍTULO 3: MARCO METODOLÓGICO

3.1 Tipo de investigación

La presente investigación es de tipo **aplicada y tecnológica** (Hernández Sampieri et al., 2014). Se orienta a resolver un problema concreto del ciclo post-venta en e-commerce —la gestión manual de órdenes y consultas de clientes— mediante el diseño e implementación de un pipeline automatizado. El objetivo no es generar teoría nueva sino construir un artefacto tecnológico funcional y medir su desempeño mediante métricas operativas cuantificables (MTTD, MTTR, TMR), verificando las hipótesis formuladas en la Sección 1.4.

3.2 Enfoque y diseño: estudio de caso instrumental

El trabajo adopta la metodología de **estudio de caso** según la definición de Yin (2018): una investigación empírica que examina un fenómeno contemporáneo dentro de su contexto real, especialmente cuando los límites entre el fenómeno y el contexto no son claramente evidentes. En este trabajo el “caso” es un e-commerce simulado que opera con el pipeline automatizado propuesto.

Siguiendo la tipología de Yin (2018), se trata de un **estudio de caso instrumental único**: el e-commerce simulado no es un fin en sí mismo sino el vehículo para comprender el comportamiento del sistema de automatización bajo condiciones controladas. El diseño es de tipo **exploratorio-confirmatorio**: exploratorio porque la revisión de antecedentes de la Sección 2.5 no identificó benchmarks publicados de n8n aplicados al ciclo post-venta de PyMEs argentinas; confirmatorio porque se contrasta el desempeño observado contra las hipótesis H1 y H2.

El e-commerce simulado incluye un catálogo de 20 productos de tecnología y mobiliario distribuidos en 11 categorías (Notebooks, Periféricos, Monitores, Audio, Almacenamiento, Tablets, Redes, Accesorios, Componentes, Mobiliario e Impresoras), un flujo completo de procesamiento de órdenes y un chatbot que atiende consultas por WhatsApp y Telegram.

El diseño de medición es cuantitativo: MTTD, MTTR y TMR sobre marcas temporales registradas automáticamente, y exactitud de clasificación contra un conjunto de etiquetas de referencia. Corresponde declarar con precisión qué se evaluó y qué no, porque el alcance de lo medido condiciona la lectura de todo el Capítulo 5. No se evaluó la calidad del contenido de las respuestas que el chatbot entrega al cliente: no se definió una rúbrica de coherencia ni de corrección factual, no se seleccionó una muestra de respuestas para juzgar y no se reporta ningún resultado sobre esa dimensión. La omisión es consciente y su consecuencia se declara en las Secciones 5.2.2 y 6.4: el trabajo puede afirmar que el sistema clasifica la intención con determinada exactitud y que responde en determinado tiempo, pero no puede afirmar que la respuesta entregada sea correcta, y la afirmación es especialmente

frágil en las consultas de tipo FAQ, donde el contexto de la base de conocimiento no llega al modelo (Sección 4.4.3). La evaluación de la corrección del contenido queda planteada como línea futura en el Capítulo 7, con la rúbrica y el procedimiento que requeriría.

Lo que el trabajo sí practica es triangulación de fuentes de evidencia, en el sentido en que Yin (2018) la recomienda para el estudio de caso: que un dato relevante no descansa sobre un único instrumento. Se aplica en dos puntos, ambos verificables en el Capítulo 5. El primero es el baseline de atención manual, medido simultáneamente por dos vías independientes —un cronómetro por fases operado sobre la tarea, que arroja 49,13 s por orden, y los incrementos entre notificaciones consecutivas registrados por la propia base de datos, que arrojan 51,28 s—, cuya convergencia dentro del 4 % es la que sostiene la validez de la cifra que después se compara contra el pipeline. El segundo es el conjunto de etiquetas de referencia del clasificador, construido por el equipo y contrastado a ciegas por un evaluador independiente sobre una submuestra aleatoria, con un κ de Cohen de 0,919 (Sección 3.5.3). En ambos casos la segunda fuente pudo haber refutado a la primera y no lo hizo; ese es el valor del procedimiento y también su límite, porque ninguna de las dos triangula la dimensión cualitativa que este trabajo dejó sin medir.

3.3 Metodología de desarrollo

El desarrollo se organizó en 6 etapas secuenciales, cada una con entregables concretos (ver Tabla 3.1):

Tabla 3.1: Etapas de desarrollo y entregables.

Etapa	Descripción	Entregable
1. Diseño del schema de BD	Modelado de 7 tablas y 6 vistas de métricas en PostgreSQL	Script SQL con DDL completo
2. Infraestructura Docker	Configuración de 4 servicios con Docker Compose	<code>docker-compose.yml</code> funcional
3. Flujo 1 — Pipeline de órdenes	Workflow de 15 nodos en n8n: recepción, stock, emails	Workflow exportable + pruebas
4. Flujo 2 — Chatbot	Workflow de 18 nodos funcionales: dos canales (WhatsApp simulado y Telegram real), IA, tickets	Workflow exportable + pruebas
5. Dashboards Grafana	13 paneles con métricas MTTR, MTTR y TMR en dos tableros	Dashboard JSON exportable
6. Testing	10 pruebas funcionales, 1 corrida de carga y 1 prueba de	Reporte de resultados

	conurrencia	
--	-------------	--

Esta secuencia permite validar cada capa antes de avanzar a la siguiente: primero la base de datos, luego la infraestructura, después los flujos, y finalmente la visualización y el testing. Este enfoque incremental reduce el riesgo de defectos acumulados y facilita el aislamiento de causas ante fallas.

3.4 Herramientas y tecnologías

Las herramientas empleadas y la justificación de cada elección se detallan en la Tabla 3.2.

Tabla 3.2: Herramientas y tecnologías utilizadas.

Herramienta	Versión	Justificación
n8n	n8nio/n8n:2.12.2	Plataforma de automatización low-code/no-code auto-alojable. Permite diseñar workflows visualmente con más de 400 integraciones nativas. Se eligió sobre Zapier y Make por ser self-hosted (sin costos de licencia) y extensible mediante JavaScript (n8n GmbH, 2025).
PostgreSQL	postgres:15.13-alpine	SGBD relacional de código abierto con soporte completo de SQL, <i>window functions</i> y tipos JSON. Elegido por robustez, disponibilidad en Docker y compatibilidad nativa con Grafana (PostgreSQL Global Development Group, 2025).
Docker Compose	Formato de archivo Compose v2 (especificación vigente)	Permite levantar los 4 servicios con un solo comando, garantizando reproducibilidad del entorno experimental (Boettiger, 2015).
GPT-4o-mini	gpt-4o-mini (API de OpenAI)	Modelo de OpenAI optimizado para tareas de clasificación y generación de texto. Menor costo que GPT-4o con rendimiento suficiente para clasificar intents y generar respuestas de soporte al cliente (OpenAI, 2024).

Mailpit	axllent/mailpit:v1.29.6	Servidor SMTP de desarrollo que captura emails sin enviarlos realmente. Ideal para testing sin riesgo de spam ni costos de mensajería (Axllent, 2025).
Grafana	grafana/grafana:13.0.7	Plataforma de visualización con conexión nativa a PostgreSQL. Permite crear dashboards interactivos sin código adicional (Grafana Labs, 2025).
curl	—	Herramienta de línea de comandos para disparar webhooks y simular órdenes entrantes en las pruebas.

3.5 Técnicas de recolección y análisis de datos

3.5.1 Recolección automatizada de métricas

La recolección de datos se realizó de forma automatizada mediante el propio sistema. Cada flujo registra timestamps en PostgreSQL al inicio y fin de cada operación, lo que permite calcular las métricas sin intervención manual ni sesgo del observador (ver Tabla 3.3).

Tabla 3.3: Datos recolectados y su método de registro.

Dato	Fuente	Método de recolección
MTTD y MTTR	Tabla orders	Diferencia de timestamps: processed_at - received_at y notified_at - processed_at
TMR	Tabla interactions	Diferencia entre received_at y responded_at
Precisión de clasificación	Tabla interactions	Comparación de la intención asignada por el modelo contra la intención de referencia para los 150 mensajes del corpus etiquetado
Tasa de resolución autónoma	Tabla interactions + tickets	Proporción de interacciones resueltas sin escalado a operador humano

3.5.2 Justificación del tamaño de muestra

Flujo 1 — Corridas de medición: se ejecutaron dos ensayos con propósitos distintos. El primero envió 50 órdenes secuenciales espaciadas 2 segundos, tamaño que provee las estimaciones de MTTD, MTTR y tiempo end-to-end con un intervalo de

confianza acotado. El segundo disparó 20 solicitudes simultáneas contra un stock deliberadamente insuficiente, repetidas durante 6 rondas (120 órdenes), lo que representa un volumen de concurrencia razonable para una PyME de escala pequeña. Este segundo tamaño no pretende representatividad estadística sino verificar la integridad transaccional del sistema bajo concurrencia, condición de diseño crítica para cualquier sistema de procesamiento de órdenes.

Flujo 2 — Clasificación de intents: se enviaron 150 mensajes de prueba distribuidos entre los cuatro intents del sistema. El criterio de distribución fue proporcional a la frecuencia esperada de cada tipo de consulta en un e-commerce típico —mayor volumen de FAQ y RECLAMO, menor volumen de ESTADO_PEDIDO y GENERAL—, criterio que la distribución efectivamente ejecutada respeta: FAQ 48 mensajes (32,0%), RECLAMO 42 (28,0%), ESTADO_PEDIDO 35 (23,3 %) y GENERAL 25 (16,7 %). Corresponde precisar el origen de ese criterio: la frecuencia esperada por tipo de consulta es una estimación del propio equipo, construida a partir del catálogo de FAQ definido para el caso y de los escenarios de uso previstos. No proviene de datos de tráfico observados en un e-commerce real, a los que este trabajo no tuvo acceso, y por lo tanto se declara como supuesto de diseño y no como dato empírico. Sobre el tamaño de la muestra: 150 observaciones superan el mínimo de 96 que arroja la fórmula de Cochran (1977) para estimar una proporción con un margen de ± 10 puntos porcentuales al 95% de confianza. Ese margen de referencia, sin embargo, no sería suficiente para discriminar el desempeño observado del umbral de aceptación, que distan 7,7 puntos. Lo que habilita el contraste no es el margen nominal sino la precisión efectivamente alcanzada: con 150 observaciones y una proporción de aciertos de 0,927, el intervalo de confianza de Wilson al 95 % resulta [87,3 %; 95,9 %], es decir un margen de aproximadamente ± 4 puntos, cuyo límite inferior se ubica por encima del umbral del 85% (véase §5.3). El tamaño muestral es, por lo tanto, suficiente para el contraste que el trabajo se propone, y así se reporta.

3.5.3 Criterio de evaluación de la exactitud

La precisión de clasificación del chatbot se evalúa mediante accuracy (proporción de predicciones correctas sobre el total), calculada comparando el intent asignado por GPT-4o-mini con el intent de referencia (ground truth) de cada mensaje. El umbral de aceptación es 85% (H2), criterio definido por el equipo para un prototipo en dominio acotado. El ground truth se construyó siguiendo el protocolo que se detalla a continuación, diseñado para que el etiquetado sea verificablemente anterior e independiente de la salida del modelo.

Construcción y congelamiento del corpus. Los 150 mensajes de prueba se redactaron incluyendo deliberadamente errores ortográficos, abreviaciones y casos de frontera semántica. El corpus se versionó en el repositorio del proyecto antes de asignar ninguna etiqueta, lo que fija su contenido con sello temporal verificable.

Etiquetado de referencia ciego. Un anotador del equipo asignó a cada mensaje una de las cuatro categorías (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL) sin haber

ejecutado el corpus contra el modelo. El carácter ciego del etiquetado no se afirma: se demuestra por la secuencia de versionado, en la que el ground truth queda congelado en el repositorio con anterioridad al registro de las clasificaciones producidas por GPT-4o-mini.

Descarte de un primer intento inválido. Un primer etiquetado fue descartado por el propio equipo tras un control de tiempos que evidenció que no había habido lectura real de los mensajes (mediana de 0,25 segundos por ítem). Se documenta aquí por transparencia metodológica: no se utilizó como ground truth bajo ningún concepto.

Control de estabilidad (test-retest). El mismo anotador re-etiquetó una submuestra de 50 mensajes en orden aleatorizado. Este control verifica la consistencia interna del criterio, pero no constituye una medida de acuerdo entre evaluadores, ya que un mismo anotador puede reproducir sus decisiones previas por memoria.

Acuerdo entre anotadores con evaluador independiente. Para descartar que el ground truth respondiera a un criterio idiosincrásico del equipo, un evaluador independiente —ajeno al trabajo, sin participación en el diseño del prompt ni en la construcción del corpus— etiquetó una submuestra aleatoria de 50 mensajes. El etiquetado se realizó sobre una planilla que presenta únicamente el texto del mensaje, en orden aleatorizado, sin exhibir las etiquetas del equipo ni las predicciones del modelo. La planilla registra el tiempo empleado en cada ítem (mediana de 4,0 segundos; ningún ítem por debajo de 1 segundo), dato que se reporta para permitir el mismo control de validez aplicado en el punto anterior. El acuerdo se cuantifica con el coeficiente κ de Cohen, que descuenta el acuerdo atribuible al azar.

3.5.4 Análisis estadístico

Los tiempos de procesamiento (MTTD, MTTR y TMR) se reportan como promedios aritméticos acompañados de su desvío estándar y de los valores mínimo y máximo observados, criterio que se mantiene de forma uniforme en todo el capítulo de resultados. No se asume normalidad en la distribución de las latencias: la literatura sobre tiempos de respuesta en sistemas distribuidos describe distribuciones asimétricas con cola derecha, y los datos medidos son consistentes con ese patrón. Para el contraste de la hipótesis sobre precisión de clasificación (H2b) se calcula el intervalo de confianza del 95% por el método de Wilson, apropiado para proporciones con tamaños de muestra moderados, y el acuerdo entre anotadores mediante el coeficiente κ de Cohen.

Para el contraste de H1 —que compara la serie de tiempos del pipeline contra la del procesamiento manual— se adopta como prueba principal la U de Mann-Whitney, elegida por lo que acaba de declararse sobre la distribución: al no asumirse normalidad, la prueba de rangos es la que corresponde. Se la acompaña de la t de Welch, que no supone igualdad de varianzas, a título de comprobación complementaria, y del tamaño del efecto en ambas escalas —correlación rango-

biserial y d de Cohen—, porque con muestras de este tamaño el valor p por sí solo dice poco. El nivel de significación se fija en 0,05 y todos los contrastes son bilaterales. El intervalo de confianza del factor de mejora, que es un cociente de dos medias, se calcula por el teorema de Fieller y no por propagación simplificada de la incertidumbre del numerador. Se anticipa aquí el límite que el Capítulo 5 retoma: una prueba de contraste establece si la diferencia observada es atribuible al azar del muestreo, pero no convierte el diseño en experimental, porque no hay asignación al azar a condiciones ni grupo de control.

3.5.5 Medición del baseline de atención manual

La pregunta de investigación y el objetivo general están formulados en términos comparativos: se busca establecer en qué medida la automatización reduce los tiempos operativos respecto del proceso manual. Ese término de comparación exige una medición propia y no una estimación tomada de la experiencia informal del equipo. Por ese motivo se diseñó un experimento específico, destinado exclusivamente a cronometrar el procesamiento manual de una orden bajo condiciones equivalentes a las del pipeline automatizado.

Tarea cronometrada. Un operador procesa una orden completa contra la misma base de datos y el mismo catálogo que utiliza el Flujo 1, sin asistencia del sistema automatizado. La tarea se descompone en tres fases que se miden por separado: T1, lectura de los datos de la orden y consulta del stock disponible del producto mediante una sentencia SQL; T2, descuento del stock y actualización del estado de la orden a confirmada, o bien marcado como sin stock cuando corresponde; y T3, redacción de la notificación al cliente completando los campos variables de una plantilla fija. El tiempo total por orden es la suma de las tres fases. El desglose importa tanto como el total, porque permite identificar dónde se concentra efectivamente el esfuerzo manual.

Decisiones de diseño y su sentido. Dos decisiones condicionan el valor obtenido y se declaran de manera explícita, porque ambas se resolvieron eligiendo deliberadamente el escenario que menos favorece a la hipótesis. La primera concierne a la redacción del correo: se optó por una plantilla fija a completar y no por la redacción desde cero, ya que la primera representa a una PyME con un proceso mínimo establecido, que es el caso realista. Esta elección reduce el tiempo del baseline y, en consecuencia, reduce el factor de mejora que el trabajo podrá reportar. La segunda concierne al alcance del cronómetro, que cubre únicamente el procesamiento: desde que el operador toma conocimiento de la orden hasta que termina de redactar la notificación. La latencia de detección, es decir cada cuánto un operador revisa efectivamente la bandeja de entrada, no se incluye dentro del valor medido, porque no resulta observable sin convertirla en un parámetro elegido por los autores; se trata por separado y como supuesto declarado en la Sección 5.1.4. La regla que ordena todo el procedimiento es que lo medido y lo supuesto no se combinan nunca dentro de una misma cifra.

Muestra e instrumento. Se procesaron doce órdenes. De las diez que resultaron válidas, siete correspondieron a la rama con stock disponible y tres a la rama sin stock —una proporción de 70 % y 30 % que reproduce la distribución observada en la corrida automatizada (35 y 15 sobre 50)—; de las dos descartadas por interrupción del operador, una (ORD-E4-002) pertenecía a la rama sin stock y la otra (ORD-E4-001) se interrumpió antes de que su rama quedara consignada. Las dos ramas demandan trabajo manual distinto y la muestra debe reflejar ambas. La composición completa, orden por orden y con su marca de descarte, se transcribe en la Tabla I.1 del Anexo I. Las órdenes se crearon en la base con un identificador de procedencia propio, distinto del que llevan las órdenes medidas del pipeline, de modo que sus marcas temporales a escala humana no contaminen el cálculo de MTTD y MTTR; todas las consultas de resultados del Capítulo 5 filtran por ese identificador. La medición se realizó con un cronómetro por fases desarrollado para este fin, que registra cada transición y exporta los tiempos a un archivo separado por comas sin intervención manual sobre los valores.

Reglas fijadas antes de comenzar. Se estableció por anticipado que el operador trabajaría a ritmo normal, sin preparar consultas de antemano, y que toda orden durante cuya ejecución se produjera una interrupción sería descartada y consignada como tal. Dos de las doce órdenes se descartaron por aplicación de esa regla; quedan registradas en el archivo de resultados con su marca de descarte y no se eliminaron del archivo. El resultado primario se calcula sobre las diez órdenes restantes.

Criterio de análisis. Se reportan media, mediana, desvío estándar, mínimo y máximo del tiempo total y de cada fase. Dado el tamaño reducido de la muestra, el intervalo de confianza de la media se calcula mediante la distribución t de Student con nueve grados de libertad y no por aproximación normal, que subestimaría su amplitud. Se controla además el efecto de aprendizaje mediante el coeficiente de correlación de Pearson entre el orden de ejecución y el tiempo empleado, cuyo resultado se reporta junto con los descriptivos en la Sección 5.1.4. Los tiempos crudos, las etiquetas de descarte y el texto de las notificaciones redactadas se transcriben en el Anexo I.

Qué magnitud mide el valor obtenido. Esta precisión es determinante para la comparación del Capítulo 5 y por eso se declara aquí. El cronómetro registra el tiempo de trabajo efectivo que el operador dedica a una orden, es decir el costo marginal de procesar una orden adicional. No mide la latencia que percibe el cliente, que en un proceso manual depende de cuántas órdenes haya por delante en la cola y no de la velocidad del operador. Ambas magnitudes son legítimas y responden a preguntas distintas: la primera describe la capacidad de proceso, la segunda la experiencia de espera.

Las doce órdenes se insertaron en la base mediante una única sentencia de preparación, de modo que todas comparten la misma marca de recepción. En consecuencia, la diferencia entre `notified_at` y `received_at` que queda registrada para

cada una no es su latencia individual sino su tiempo de permanencia en la cola, que crece de manera monótona a medida que el operador avanza. Ese registro se conserva y se reporta en la Sección 5.1.4 porque describe un fenómeno real —el encolamiento—, pero no es el valor que se contrasta contra el pipeline automatizado. El término de comparación es el tiempo marginal cronometrado.

3.6 Amenazas a la validez

Siguiendo el marco de Yin (2018) para estudios de caso, se identifican las siguientes amenazas a la validez:

3.6.1 Validez de constructo

Amenaza: Las métricas MTTD, MTTR y TMR fueron adaptadas desde el dominio de gestión de incidentes TI (ITIL) al dominio de procesamiento de e-commerce. Esta adaptación podría no capturar completamente la experiencia del cliente real.

Mitigación: Se operacionalizaron explícitamente cada métrica mediante timestamps observables en la base de datos (Sección 4.3.3 y 4.4.4), y se documentó la correspondencia con las definiciones de origen.

3.6.2 Validez interna

Amenaza: Los resultados de tiempo (MTTD, MTTR, TMR) corresponden a un entorno Docker en una notebook local. Las latencias observadas pueden estar influenciadas por los recursos del hardware utilizado (CPU, RAM, disco) y no solo por el comportamiento del sistema.

Mitigación: Se especifican las condiciones del entorno de prueba en la Sección 4.6 y los resultados se presentan como benchmarks de referencia bajo las condiciones descritas, no como valores absolutos.

3.6.3 Validez externa

Amenaza: Al tratarse de un e-commerce simulado con datos ficticios, los resultados podrían no generalizarse directamente a un e-commerce real con variabilidad de productos, clientes y patrones de compra.

Mitigación: El sistema y su metodología de prueba son reproducibles (Docker Compose + scripts de testing incluidos en el repositorio), lo que permite replicar el estudio con datos reales como trabajo futuro. Los escenarios de prueba fueron diseñados para cubrir los casos límite más relevantes (órdenes con stock, sin stock, payloads inválidos, mensajes ambiguos).

3.6.4 Confiabilidad

Amenaza: Los resultados de TMR dependen de la latencia de la API de OpenAI, que puede variar según la carga del servicio externo.

|

Mitigación: Todas las pruebas se realizaron en una ventana de tiempo acotada para minimizar la variabilidad temporal del servicio externo. Los tiempos se reportan como promedios aritméticos acompañados de su desvío estándar, criterio uniforme con el establecido en la Sección 3.5.4; la mediana se informa además de forma complementaria en los casos en que la distribución resulta asimétrica.

CAPÍTULO 4: DESARROLLO DEL ESTUDIO

4.1 Arquitectura del sistema

La arquitectura se basa en **4 servicios Docker orquestados con Docker Compose**, conectados a través de una red bridge interna. n8n actúa como nodo central de orquestación, conectándose a PostgreSQL para persistencia, a Mailpit —capturador SMTP local que intercepta los correos salientes sin entregarlos a destinatarios reales (Axllent, 2025)— para envío de emails y a Grafana para visualización.

4.1.1 Servicios Docker

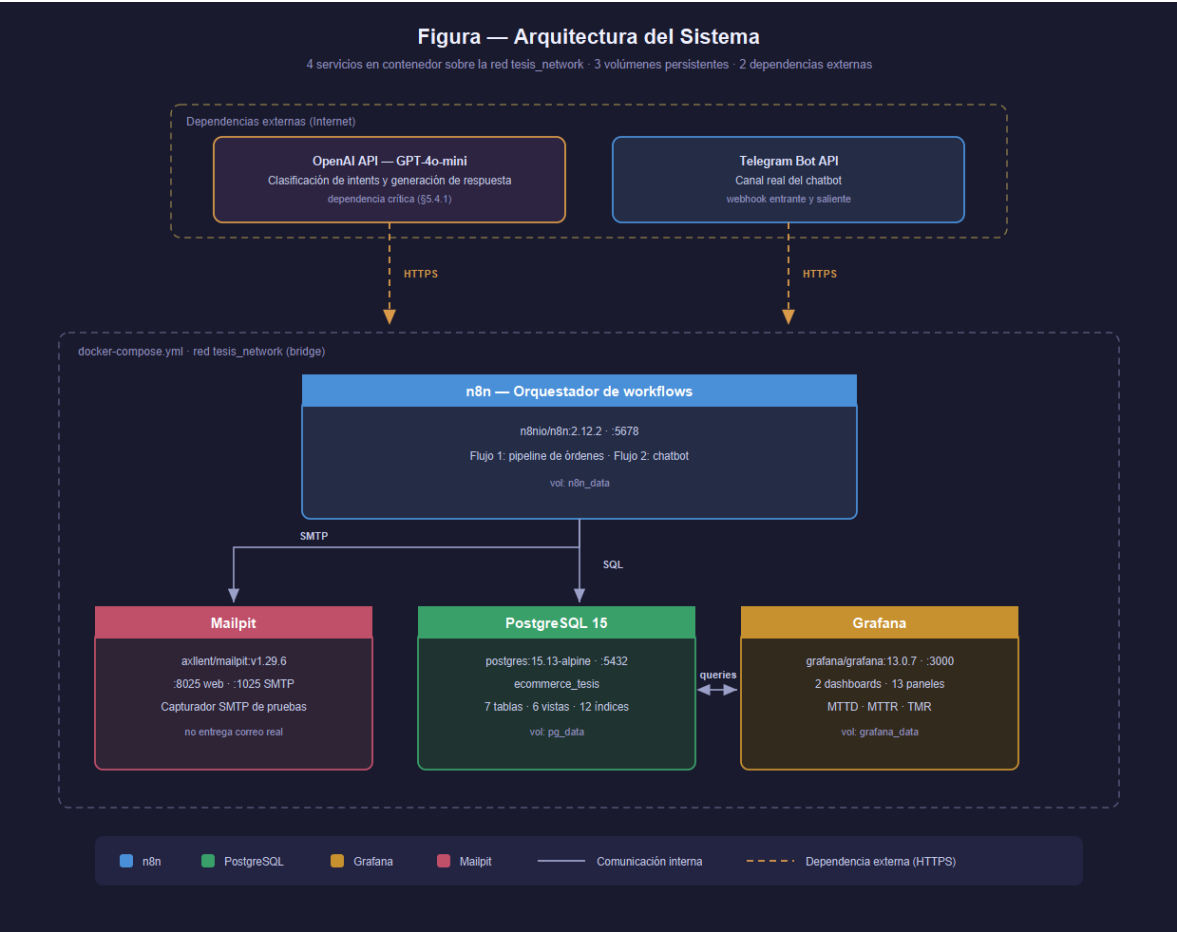


Figura 1: Arquitectura del sistema. Cuatro servicios en contenedor sobre una red interna, con las dos dependencias externas de las que depende el Flujo 2: la API de OpenAI (GPT-4o-mini) y la API de Telegram.

Los servicios que componen el despliegue se resumen en la Tabla 4.1.

Tabla 4.1: Servicios Docker del sistema.

Servicio	Imagen	Puerto(s)	Función
n8n	n8nio/n8n:2.12.2	5678	Motor de workflows. Ejecuta los Flujos 1 y 2
PostgreSQL	postgres:15.13-alpine	5433 → 5432 (host → contenedor)	Base de datos principal (ecommerce_tesis)
Mailpit	axllent/mailpit:v1.29.6	8025 (web), 1025 (SMTP)	Captura emails de confirmación y alertas
Grafana	grafana/grafana:13.0.7	3000	Dashboard de métricas en tiempo real

Volúmenes persistentes: n8n_data, pg_data, grafana_data — garantizan que los datos sobrevivan al reinicio de los contenedores.

Red: tesis_network (driver bridge) — permite que los servicios se comuniquen entre sí por nombre de contenedor (ej. postgres:5432 desde n8n).

4.2 Base de datos

4.2.1 Tablas principales

La base de datos ecommerce_tesis contiene 7 tablas, 6 vistas de métricas y 12 índices de rendimiento sobre las columnas usadas en filtros y joins (ver Tabla 4.2):

Corresponde una salvedad sobre order_items, porque el esquema y el sistema implementado no coinciden en este punto. La tabla existe, tiene sus claves foráneas y sus índices, y normaliza correctamente la relación entre una orden y sus productos. Pero ninguno de los quince nodos del Flujo 1 (Tabla 4.5) inserta en ella: el pipeline implementado carga órdenes de un solo producto, y conserva a tal efecto los campos de producto y cantidad en la propia tabla de órdenes. El esquema resuelve por lo tanto el modelado de órdenes multiproducto, y el sistema todavía no lo ejercita. Se declara aquí y no solo en el comentario del script del Anexo A, porque es en el cuerpo del capítulo donde el lector lo busca.

Tabla 4.2: Esquema de tablas de la base de datos.

Tabla	Columnas principales	Descripción
products	sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock mínimo

|

orders	order_number, customer_name, customer_email, customer_phone, product_id (FK), quantity, total_amount, status, received_at, processed_at, notified_at, raw_payload	Órdenes con 8 estados posibles y timestamps de cada etapa; los campos processed_at y notified_at permiten calcular MTTD y MTTR directamente
order_items	order_id (FK), product_id (FK), quantity, unit_price, subtotal	Ítems de cada orden: permite N productos por orden
stock_alerts	product_id (FK), order_id (FK), sku, stock_actual, stock_min	Alertas de reposición cuando el stock cruza su umbral
interactions	channel, user_id, message, intent, ai_response, order_id (FK), is_urgent, received_at, responded_at	Registro de interacciones del chatbot por canal
tickets	interaction_id (FK), order_id (FK), channel, user_id, subject, status, priority	Tickets de reclamos generados automáticamente
faq_responses	question, answer, category, enabled	Base de conocimiento para respuestas frecuentes

Estados admitidos por la restricción de integridad del campo status: pending, processing, confirmed, no_stock, error, cancelled, shipped y delivered.

Nota: stock_verified y stock_updated no son estados del campo status sino pasos internos del pipeline, observables en el historial de ejecuciones que el propio motor de flujos conserva. El esquema de este trabajo no incorpora una tabla de bitácora de eventos: el campo status refleja únicamente los estados de negocio visibles al cliente y al operador, y las marcas temporales received_at, processed_at y notified_at registran los instantes del procesamiento.

Intents del chatbot: FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL

4.2.2 Diagrama entidad-relación

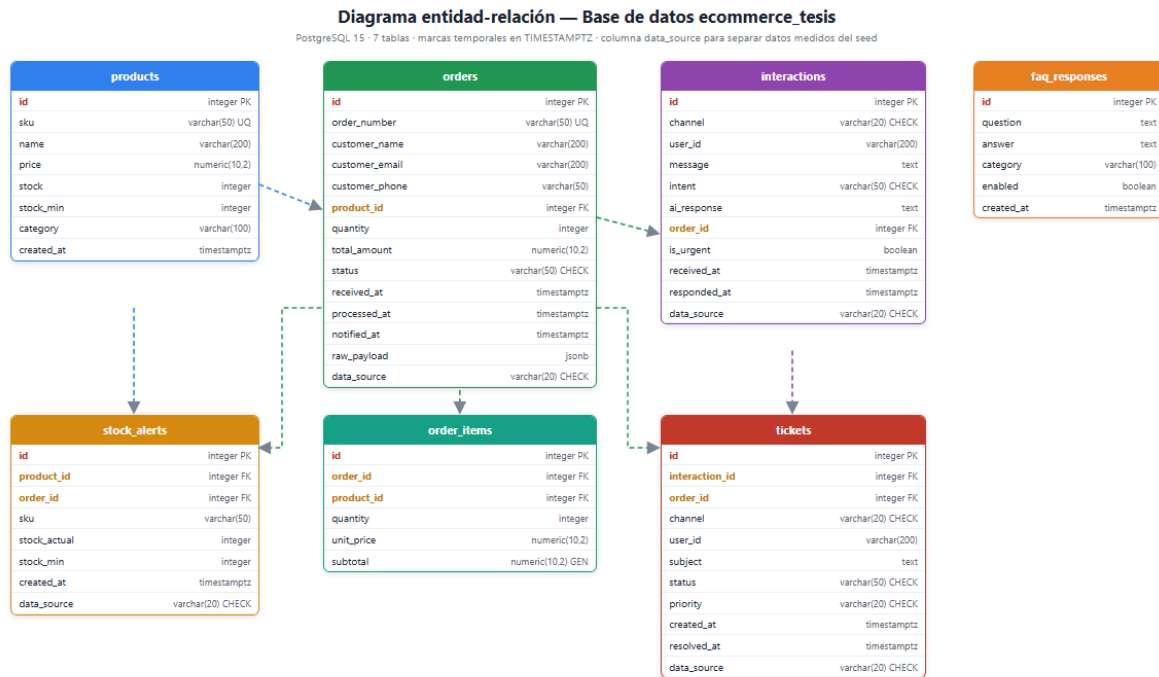


Figura 2: Diagrama entidad-relación de la base de datos ecommerce_tesis, con las siete tablas, sus claves foráneas y la columna data_source que separa los registros medidos de los datos de carga inicial.

4.2.3 Vistas de métricas

Las vistas de métricas definidas sobre el esquema se describen en la Tabla 4.3.

Tabla 4.3: Vistas de métricas en PostgreSQL.

Vista	Descripción	Métricas
v_order_processing_time	Tiempos de procesamiento por orden usando timestamps de orders	MTTD (received→processed), MTTR (processed→notified) por orden
v_daily_order_summary	Resumen diario de órdenes	Total, confirmadas, sin stock, errores por día
v_chatbot_response_time	Tiempo de respuesta del chatbot	TMR por interacción
v_daily_chatbot_summary	Resumen diario del chatbot	Interacciones por canal e intent por día

v_metrics_summary	Resumen ejecutivo unificado	Promedios globales de MTTD, MTTR, TMR
v_chatbot_corpuses	Aísla el corpus de 150 mensajes con etiquetas de referencia, por ventana temporal	TMR e intención predicha sobre el corpus evaluado

4.2.4 Datos seed

Productos (20):

El catálogo de productos utilizado en las pruebas se lista en la Tabla 4.4.

Tabla 4.4: Catálogo de productos de prueba.

SKU	Producto	Precio	Stock	Stock Mín.	Categoría
PROD-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PROD-002	Mouse Inalámbrico Logitech	\$29.99	50	5	Periféricos
PROD-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PROD-004	Monitor Samsung 24” FHD	\$249.99	8	2	Monitores
PROD-005	Auriculares Sony WH- 1000XM5	\$349.99	4	2	Audio
PROD-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PROD-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PROD-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PROD-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PROD-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras

PROD-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PROD-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PROD-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks
PROD-014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores
PROD-015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PROD-016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PROD-017	Disco Rígido Seagate 2TB HDD	\$64.99	20	4	Almacenamiento
PROD-018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamiento
PROD-019	Pad Mouse XL Antideslizante	\$14.99	60	8	Accesorios
PROD-020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

FAQ (23 entradas, 8 categorías): Pagos, Envíos, Devoluciones, Garantía, Soporte, Facturación, Productos y Pedidos. El contenido íntegro se transcribe en el Anexo D.

4.3 Flujo 1 — Pipeline de procesamiento de órdenes

El Flujo 1 automatiza el ciclo completo de una orden: desde su recepción por webhook hasta el envío de email de confirmación (o rechazo por falta de stock). Los timestamps `received_at`, `processed_at` y `notified_at` en la tabla `orders` permiten calcular los tiempos MTTD y MTTR directamente.

4.3.1 Nodos del workflow

Los nodos que componen el Flujo 1 y su función se detallan en la Tabla 4.5.

Tabla 4.5: Nodos del workflow del Flujo 1.

#	Nodo	Tipo n8n	Descripción
1	Webhook - Recibir Orden	Webhook	Escucha POST en /webhook/orden-nueva y recibe el pedido
2	Registrar Orden	PostgreSQL	INSERT en orders con estado inicial y marca de received_at
3	Verificar Stock	PostgreSQL	SELECT del producto por SKU; devuelve stock actual y stock_min
4	IF Stock Disponible	IF	Evalúa si el stock alcanza para la cantidad solicitada
5	Actualizar Stock	PostgreSQL	UPDATE que descuenta la cantidad y devuelve el stock resultante
6	IF Stock Bajo	IF	Evalúa si el stock resultante quedó por debajo de stock_min
7	Registrar Alerta Stock Bajo	PostgreSQL	INSERT en stock_alerts; el flujo continúa hacia la confirmación
8	Confirmar Orden	PostgreSQL	UPDATE de orders a estado confirmed y marca de processed_at
9	Enviar Email Confirmación	Email (SMTP)	Envía la confirmación al cliente a través de Mailpit
10	Registrar Notificación	PostgreSQL	UPDATE de notified_at, que cierra la medición del MTTR
11	Respuesta Confirmada	Respond to Webhook	Devuelve al emisor el resultado de la orden confirmada
12	Marcar Sin Stock	PostgreSQL	UPDATE de orders a estado no_stock cuando no alcanza el stock
13	Enviar Email Sin Stock	Email (SMTP)	Envía el aviso de stock insuficiente al cliente
14	Registrar Notificación Sin Stock	PostgreSQL	UPDATE de notified_at en la rama sin stock
15	Respuesta Sin Stock	Respond to Webhook	Devuelve al emisor el resultado de la orden sin stock

4.3.2 Diagrama del flujo

La Figura 3 representa el flujo lógico del pipeline de procesamiento de órdenes, desde la recepción del pedido hasta la notificación al cliente.

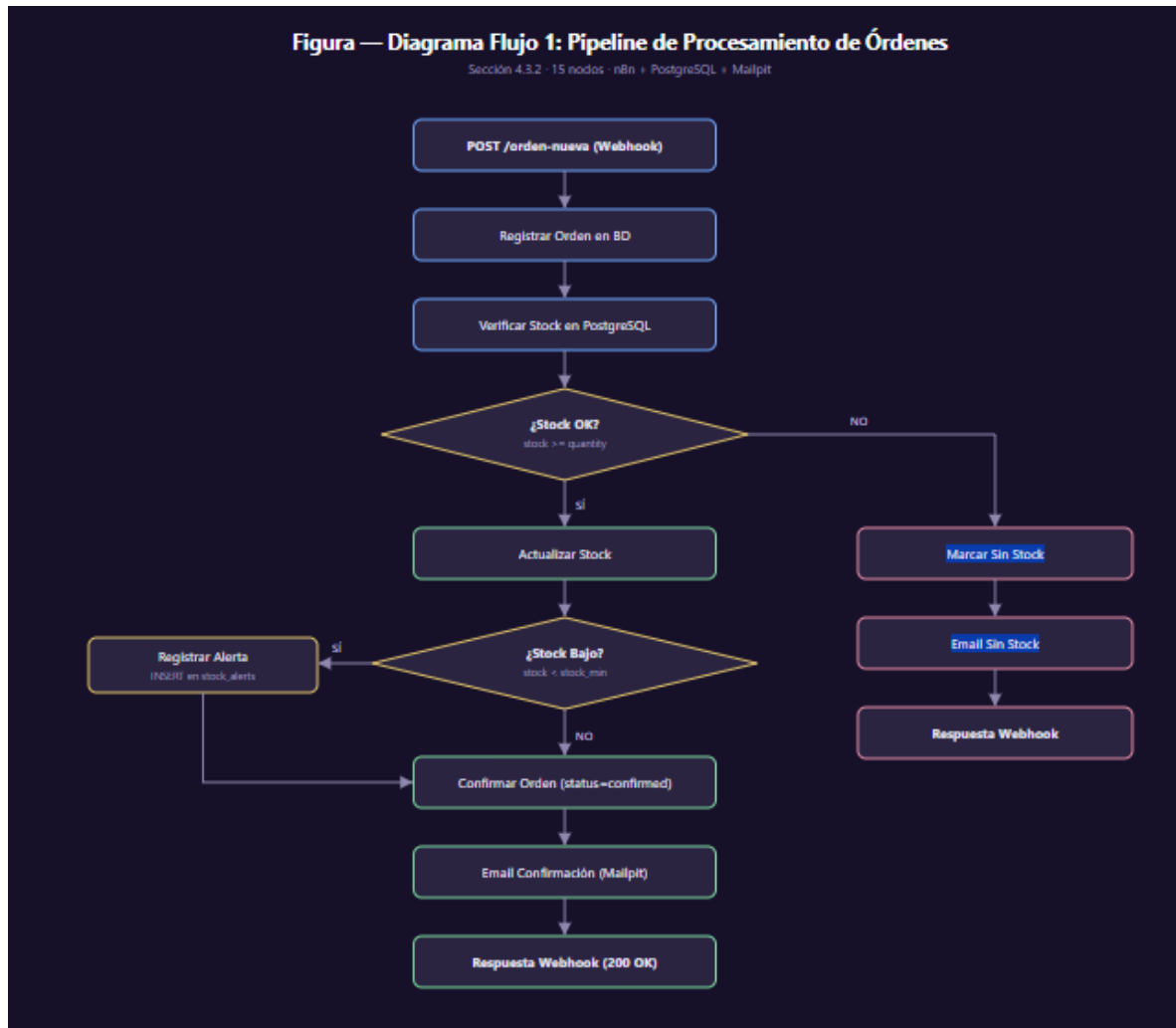


Figura 3: Diagrama del Flujo 1 — Pipeline de procesamiento de órdenes. Elaboración propia a partir del workflow implementado en n8n.

La Figura 4 muestra ese mismo flujo tal como quedó implementado en el canvas de n8n.

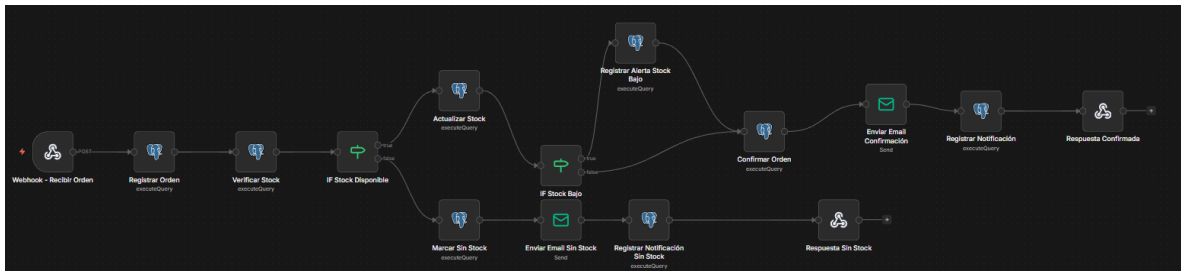


Figura 4: Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.

4.3.3 Métricas MTTD y MTTR

MTTD (Mean Time To Detect): Tiempo promedio entre la recepción de la orden (received_at) y el procesamiento del stock (processed_at). Mide la velocidad de detección y validación del pipeline.

MTTR (Mean Time To Resolve): Tiempo entre el procesamiento (processed_at) y la notificación al cliente (notified_at). Mide el ciclo de resolución y despacho de la orden.

Ambas métricas se calculan en la vista v_order_processing_time a partir de las tres marcas temporales de la tabla orders. Para evitar ambigüedad en su operacionalización se precisa dónde escribe cada marca el workflow: received_at lo escribe el nodo Registrar Orden al ingresar el pedido; processed_at lo escribe Confirmar Orden, una vez verificado y descontado el stock; y notified_at lo escribe el nodo Registrar Notificación, que se ejecuta DESPUÉS del nodo de envío del correo (ver Tabla 4.5, nodos 9 y 10). El MTTR mide, por lo tanto, el tiempo hasta que la notificación fue efectivamente despachada por el sistema. Cabe señalar que el destinatario es un capturador SMTP local: el tiempo de tránsito hasta la casilla real del cliente queda fuera de lo observable en este entorno y no forma parte de la métrica. Sobre received_at corresponde una precisión que acota el alcance del MTTD. Esa marca la escribe el propio pipeline, ya en posesión del pedido, y no la capa HTTP que lo recibe: el intervalo medido no es el que media entre el arribo del pedido al sistema y su detección, sino el que separa dos escrituras consecutivas sobre la base local. El nombre heredado de la práctica ITIL promete por lo tanto más de lo que el instrumento entrega, y así se declara. La comparación contra el baseline manual conserva no obstante la simetría, porque ese baseline también excluye la latencia de detección y así se establece en la Sección 3.5.5. Capturar la marca de arribo en la capa HTTP, anterior a la escritura en base, queda planteado en el Capítulo 7 como corrección instrumental de una futura versión.

```

mttd_seconds = EXTRACT(EPOCH FROM (processed_at - received_at))
mttr_seconds = EXTRACT(EPOCH FROM (notified_at - processed_at))
  
```

4.3.4 Flujo de estados de una orden

Una orden transita por los estados que admite la restricción de integridad del esquema (ver Tabla 4.6). Conviene distinguirlos de las marcas temporales: `received_at`, `processed_at` y `notified_at` son timestamps que registran instantes del procesamiento y no estados de la orden; en particular, la notificación al cliente no constituye un estado sino la escritura de `notified_at`. De los ocho estados admitidos, el pipeline implementado asigna tres (`pending`, `confirmed` y `no_stock`); los restantes corresponden a etapas posteriores del ciclo de vida comercial que exceden el alcance de este trabajo.

Tabla 4.6: Estados de una orden y nodo que los asigna.

Estado	Descripción	Nodo que lo asigna
<code>pending</code>	Orden recibida y registrada; aún no se verificó el stock	Registrar Orden (valor por defecto)
<code>processing</code>	Estado intermedio previsto para procesamiento asincrónico	No lo asigna el pipeline actual
<code>confirmed</code>	Stock descontado y orden aprobada	Confirmar Orden
<code>no_stock</code>	Stock insuficiente para la cantidad solicitada	Marcar Sin Stock
<code>shipped</code>	Orden despachada	Gestión posterior (fuera del pipeline)
<code>delivered</code>	Orden entregada al cliente	Gestión posterior (fuera del pipeline)
<code>cancelled</code>	Orden cancelada	Gestión posterior (fuera del pipeline)
<code>error</code>	Error inesperado durante el procesamiento	No lo asigna el pipeline actual

El MTDD se calcula sobre el intervalo `received_at` → `processed_at` y el MTTR sobre el intervalo `processed_at` → `notified_at`; ambas son diferencias entre marcas temporales y no entre estados, conforme la distinción establecida en el párrafo anterior. En el caso de `no_stock`, el MTTR refleja el tiempo transcurrido hasta el envío de la notificación de rechazo.

4.4 Flujo 2 — Chatbot con IA

El Flujo 2 implementa un chatbot multicanal sobre dos canales efectivamente medidos: WhatsApp, con envío simulado a un capturador SMTP local, y Telegram, con envío real. La extensión al correo electrónico (Gmail) queda como línea futura y no se ejecutó, por el piso de latencia que impone su mecanismo de consulta

periódica. El chatbot normaliza cada mensaje a un formato común según su canal de origen, consulta la base de conocimiento FAQ y usa GPT-4o-mini para clasificar el intent y generar una respuesta contextualizada. Si detecta un reclamo, crea un ticket automáticamente; luego un nodo de ruteo (Router Canal) envía la respuesta por el mismo canal por el que llegó el mensaje.

4.4.1 Nodos del workflow

Los nodos que componen el Flujo 2 y su función se detallan en la Tabla 4.7.

Tabla 4.7: Nodos del workflow del Flujo 2.

#	Nodo	Tipo n8n	Descripción
1	Trigger WhatsApp Business	Webhook	Recibe mensajes por webhook (formato WhatsApp Cloud API)
2	Trigger Telegram	Telegram Trigger	Recibe mensajes reales del canal Telegram
3	Trigger Gmail	Gmail Trigger	Diseñado, no activado (el polling impone un piso de latencia ~1 min)
4	Normalizar Mensaje	Function	Detecta el canal y unifica a formato común (usuario, mensaje, canal)
5	Buscar FAQ	PostgreSQL	SELECT de faq_responses
6	Preparar Contexto FAQ	Function	Arma el contexto de FAQ para el prompt
7	IA - Motor Decisión	LangChain / GPT-4o-mini	Clasifica el intent y genera la respuesta
8	OpenAI Chat Model	OpenAI	Modelo de lenguaje GPT-4o-mini
9	Parse JSON	Function	Extrae el JSON estructurado de la salida del modelo
10	Switch Intent	Switch	Rutea según intent (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL)
11	Buscar Pedido	PostgreSQL	SELECT de orders (rama ESTADO_PEDIDO)
12	Preparar Respuesta Pedido	Function	Arma la respuesta con el detalle del pedido

13	Crear Ticket	PostgreSQL	INSERT en tickets (rama RECLAMO)
14	Preparar Respuesta Ticket	Function	Arma la respuesta con el n° de ticket
15	Router Canal	IF	Rutea la respuesta según el canal de origen
16	Enviar Telegram	Telegram	Envía la respuesta por Telegram (canal real)
17	Enviar Respuesta (canal simulado)	Email (SMTP)	Envía la respuesta por email: canal simulado en formato WhatsApp Cloud API con entrega SMTP
18	Registrar Interacción	PostgreSQL	INSERT en interactions con received_at/responded_at (TMR)

4.4.2 Diagrama del flujo

La Figura 5 representa el flujo lógico del chatbot, desde la recepción del mensaje hasta el envío de la respuesta por el canal de origen.

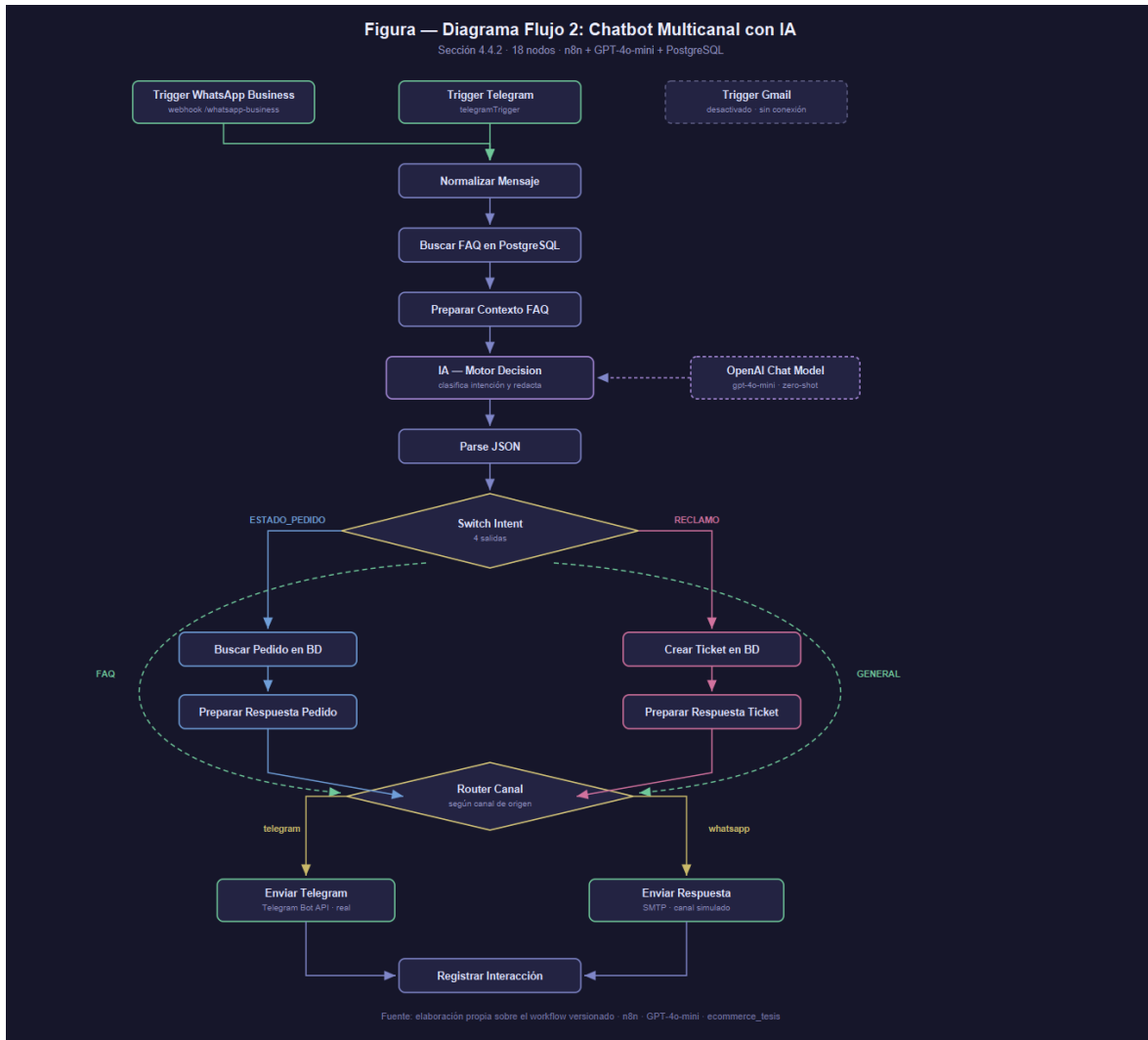


Figura 5: Diagrama del Flujo 2 — Chatbot con clasificación de intenciones. Elaboración propia a partir del workflow implementado en n8n.

La Figura 6 muestra su implementación en el canvas de n8n.



Figura 6: Canvas del Flujo 2 en n8n, mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por intent y el nodo Router Canal que envía la respuesta por el canal de origen.

4.4.3 Prompt de IA

El nodo de GPT-4o-mini recibe un prompt de sistema que define la identidad del asistente, la tarea de clasificación, las cuatro categorías de intención admitidas — enumeradas dentro del esquema de salida como FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL— y el formato estricto de la respuesta: un objeto JSON con los campos intent, order_id, urgente y respuesta. Conviene precisar qué no contiene, porque condiciona la lectura del resultado: no incluye criterios de decisión que definan cuándo corresponde cada categoría, ni reglas que establezcan cuándo un mensaje debe marcarse como urgente. El modelo resuelve ambas cosas a partir del significado de las etiquetas. El texto completo se transcribe en el Anexo H, de modo que el resultado de clasificación del Capítulo 5 sea auditable.

Se eligió GPT-4o-mini por su balance entre costo y capacidad: para la clasificación de intenciones en un dominio acotado y la generación de respuestas de soporte no se requiere la potencia completa de GPT-4o. El prompt no incorpora ejemplos etiquetados, de modo que opera en régimen zero-shot (véase la Sección 2.2.2). Cabe señalar una diferencia entre el diseño y la implementación que se detectó al auditar el workflow y que se declara antes que corregirse, porque corregirla invalidaría la medición ya realizada: el flujo incluye un nodo que recupera las FAQ de la base de datos y arma un bloque de contexto, pero esa variable no llega a referenciarse desde el prompt del nodo de inferencia. En consecuencia, el accuracy del 92,7 % reportado en el Capítulo 5 se obtuvo sin inyección de FAQ en el prompt. El cableado efectivo de ese contexto queda como línea futura en el Capítulo 7. De ello se sigue una salvedad sobre el recuento de nodos que se consigna aquí para que no induzca a error: los nodos 5 y 6 del Flujo 2 —Buscar FAQ en PostgreSQL y Preparar Contexto FAQ— integran los dieciocho nodos funcionales del workflow, pero no inciden en la salida del sistema en la configuración efectivamente medida. El recuento describe el workflow tal como está construido, no la cadena causal que produce la respuesta.

4.4.4 Métrica TMR

TMR (Tiempo Medio de Respuesta): Tiempo entre la recepción del mensaje del cliente (received_at) y el envío de la respuesta (responded_at). Se calcula automáticamente en la vista v_chatbot_response_time. Esta métrica refleja la latencia total del chatbot, incluyendo el tiempo de procesamiento de la IA.

4.5 Configuración de Grafana

Se configuraron dos dashboards en Grafana con conexión directa a PostgreSQL, uno por cada flujo: “Pipeline Post-Venta — Overview” (6 paneles) y “Chatbot Multicanal — Métricas” (7 paneles). Los paneles se alimentan de las vistas de métricas y, en los casos en que se requiere una agregación específica del panel, de una consulta directa sobre las tablas base. El filtrado por procedencia no es uniforme entre paneles, y corresponde declarar qué panel aplica cuál criterio, porque de ello depende cómo se lee cada tablero. Diez de los trece paneles se restringen a los

registros medidos (`data_source = 'measured'`): cuatro del tablero del Flujo 1 —los tres indicadores de tiempo y la distribución de estados— lo hacen mediante una cláusula explícita en la consulta del propio panel, y seis del tablero del Flujo 2 lo heredan de la vista `v_chatbot_corpus`, que además acota la población a la ventana temporal de la corrida evaluada. El panel de exactitud es un valor constante, según se declara en la Tabla 4.8. Los dos paneles restantes, ambos del tablero del Flujo 1, no aplican ese filtro: «Órdenes totales» consume la vista `v_metrics_summary` y «Órdenes procesadas por día» la vista `v_daily_order_summary`, y ninguna de las dos restringe por procedencia —sus definiciones, transcritas en el Anexo A, agregan sobre la totalidad de la tabla de órdenes—. En consecuencia, esos dos paneles incluyen también las órdenes de carga inicial. La consecuencia sobre los resultados de este trabajo es nula, y conviene decir por qué: ninguna cifra del Capítulo 5 proviene de esas dos vistas. Las métricas reportadas se calculan sobre las corridas identificadas por su propio prefijo de número de orden, según se detalla en la Sección 3.5.1. Los dos paneles cumplen una función de contexto operativo y no de evidencia. Homogeneizar el filtro en las cuatro vistas que hoy no lo aplican queda planteado en el Capítulo 7.

4.5.1 Paneles de los dashboards

Los paneles configurados en el dashboard y su fuente de datos se listan en la Tabla 4.8.

Tabla 4.8: Paneles de los dashboards de Grafana (Flujo 1: paneles 1 a 6; Flujo 2: paneles 7 a 13).

#	Panel	Tipo	Fuente de datos	Descripción
1	MTTD — Detección (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo promedio de detección de una orden
2	MTTR — Resolución (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo promedio hasta la notificación al cliente
3	End-to- end (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo total de extremo a extremo
4	Órdenes totales	Stat	<code>v_metrics_summary</code>	Volumen de órdenes medidas
5	Distribución de estados de órdenes	Pie chart	orders	Proporción de órdenes por estado
6	Órdenes procesada	Time series	<code>v_daily_order_summary</code>	Evolución diaria del volumen procesado

	s por día			
7	TMR promedio (s)	Stat	v_chatbot_corpus	Tiempo medio de respuesta del chatbot
8	Interacciones totales	Stat	v_chatbot_corpus	Volumen de interacciones del corpus evaluado
9	Precisión (accuracy)	Stat	Valor constante	Accuracy de clasificación del corpus. No se deriva de una consulta SQL: su cálculo requiere las etiquetas de referencia, que no residen en la base de datos operativa. El panel muestra el valor obtenido en el Anexo J.
10	Tickets abiertos	Stat	tickets	Tickets generados por la rama de reclamos
11	TMR promedio por intent (s)	Bar chart	v_chatbot_corpus	Desglose del TMR por intención
12	Distribución de intents	Pie chart	v_chatbot_corpus	Proporción de mensajes por intención
13	Interacciones por día y canal	Time series	v_chatbot_corpus	Evolución diaria por canal de entrada

La Figura 7 presenta el dashboard del Flujo 1 con los valores medidos.



Figura 7: Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1. Los paneles de indicadores y de distribución de estados se restringen a los registros medidos; los de «Órdenes totales» y «Órdenes procesadas por día» no lo hacen, de modo que el primer punto de la serie diaria corresponde a las órdenes de carga inicial y no a una corrida experimental (Sección 4.5).

4.6 Testing

Las pruebas se ejecutaron sobre una notebook con procesador Intel Core i5 de 11^a generación, 16 GB de RAM DDR4, almacenamiento SSD NVMe de 512 GB y sistema operativo Windows 11 con WSL2 (Ubuntu 22.04). Los cuatro contenedores Docker (n8n, PostgreSQL, Mailpit, Grafana) se levantaron con la configuración por defecto del archivo docker-compose.yml, sin asignación explícita de límites de CPU o memoria. La conectividad a la API de OpenAI se realizó sobre una conexión de fibra óptica de 100 Mbps simétricos. Estas condiciones del entorno deben tenerse en cuenta al interpretar los valores absolutos de MTTD, MTTR y TMR reportados en el Capítulo 5: los tiempos de procesamiento del Flujo 1 dependen del rendimiento del hardware local y del SGBD; el TMR del Flujo 2 está dominado por la latencia de la API externa y no por el entorno local. El TMR observado se ubica entre 1,47 s sobre el canal con entrega local y 3,07 s sobre el canal Telegram real. Conviene precisar qué se puede y qué no se puede concluir de esa diferencia de 1,6 s. No es una acotación del componente atribuible a la red del canal, porque las dos series no son apareadas: provienen de conjuntos distintos —150 mensajes del corpus etiquetado y 45 interacciones de Telegram declaradas como muestra adicional y no como subconjunto— que difieren en composición de intenciones, en longitud de los mensajes y en momento de ejecución, y los tres factores inciden sobre el tiempo de inferencia. La diferencia se reporta por lo tanto como observación entre dos condiciones de medición, no como descomposición del tiempo. Aislar el componente de red exigiría enviar un mismo subconjunto de mensajes por ambos canales, lo que queda planteado en el Capítulo 7.

4.6.1 Pruebas funcionales

Siguiendo el criterio de diseño de casos de prueba orientado a cubrir tanto los caminos nominales como las condiciones de borde (Myers et al., 2011), se ejecutaron 10 pruebas funcionales sobre los escenarios principales de ambos flujos: (ver Tabla 4.9) (ver Tabla 4.10)

Flujo 1 — Pipeline de órdenes:

Tabla 4.9: Pruebas funcionales del Flujo 1.

#	Prueba	Entrada	Resultado esperado
F1	Orden con stock suficiente	POST con producto en stock	Orden confirmada + email enviado
F2	Orden sin stock	POST con stock = 0	Orden marcada no_stock + email de rechazo
F3	Orden que genera stock bajo	POST que deja stock < stock_min	Alerta low_stock_alert registrada
F4	SKU inválido	POST con product_sku inexistente en el catálogo	Rechazo con error controlado
F5	Orden duplicada	POST con un order_number ya registrado	Rechazo por restricción de unicidad

Flujo 2 — Chatbot:

Tabla 4.10: Pruebas funcionales del Flujo 2.

#	Prueba	Entrada	Resultado esperado
C1	Consulta FAQ	“¿Cuáles son los métodos de pago?”	Respuesta de FAQ + intent=FAQ
C2	Estado de pedido	“¿Dónde está mi pedido ORD-001?”	Consulta a orders + respuesta con estado
C3	Reclamo	“Quiero hacer un reclamo, el producto llegó roto”	Ticket creado + intent=RECLAMO

C4	Reclamo urgente	“URGENTE: el pedido nunca llegó y necesito solución YA”	Ticket creado + is_urgent=true + respuesta al cliente por el canal de origen
C5	Consulta general	“¿Tienen tiendas físicas?”	Respuesta de IA + intent=GENERAL

4.6.2 Pruebas de carga y de concurrencia

Se diseñaron dos ensayos distintos sobre el Flujo 1, con objetivos diferentes y poblaciones separadas. El primero (identificado como E1.a en el repositorio) evalúa el comportamiento del pipeline bajo un régimen de llegada sostenido pero no simultáneo: se envían 50 órdenes secuenciales al webhook, espaciadas 2 segundos entre sí, con una composición de 35 órdenes con stock suficiente y 15 que fuerzan deliberadamente la rama de stock insuficiente. Esta corrida es la que provee las mediciones de MTTD, MTTR y tiempo end-to-end reportadas en la Sección 5.1.

El segundo ensayo (E1.b) evalúa una propiedad distinta y no trivial: la atomicidad de la transacción de descuento de stock bajo concurrencia real. Se disparan 20 solicitudes HTTP simultáneas contra un producto cuyo stock inicial se fija deliberadamente en 5 unidades, y el ensayo se repite durante 6 rondas, totalizando 120 órdenes. El criterio de falla está definido de antemano y es binario: si en alguna ronda el sistema confirma más de 5 órdenes, existe sobreventa y por lo tanto una condición de carrera en el descuento de inventario.

Ambas corridas se ejecutan mediante scripts versionados que dejan un manifiesto JSON con la semilla utilizada, el número de nodos del flujo al momento de medir, el commit de Git y las marcas temporales en UTC. El plan de órdenes es determinístico: la misma semilla sobre el mismo stock inicial reproduce exactamente la misma secuencia de SKU y cantidades.

Las órdenes generadas por ambos ensayos se registran con `data_source = 'measured'`, lo que las separa tanto de los datos seed precargados como del baseline manual de la Sección 3.5.5. Las vistas que alimentan los tableros de resultados filtran por ese valor, de modo que ninguna cifra reportada en el Capítulo 5 mezcla datos medidos con datos sintéticos.

Los correos de confirmación y de aviso de stock insuficiente generados durante ambas corridas se verifican en la bandeja del capturador SMTP (ver Figura 8).

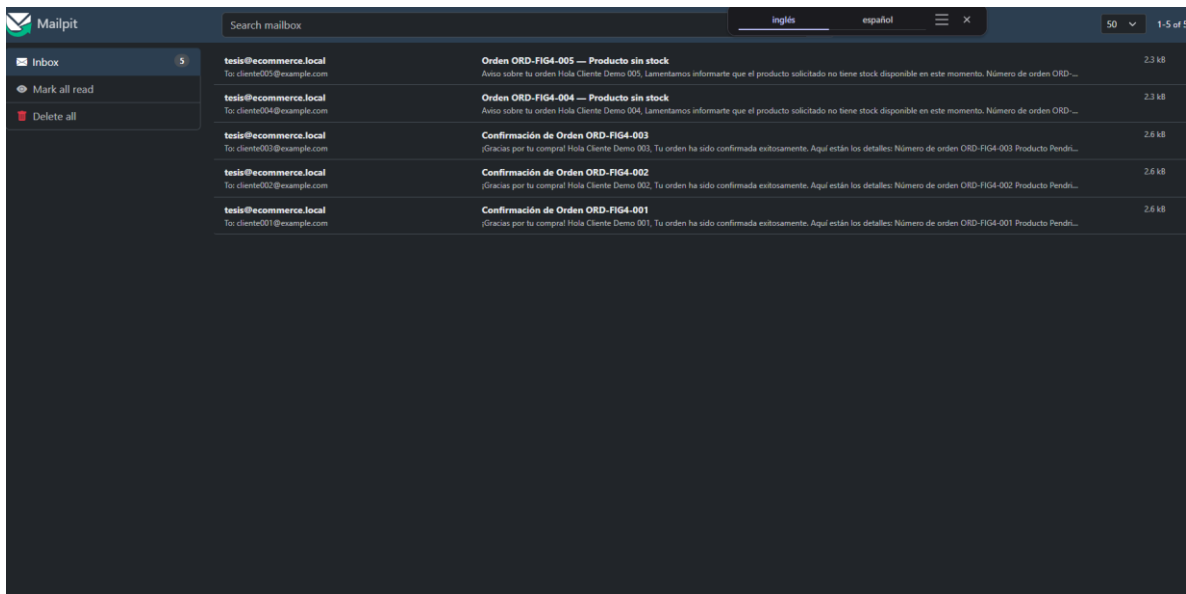


Figura 8: Bandeja de Mailpit con los correos generados por el Flujo 1 durante una corrida de verificación de cinco órdenes (ORD-FIG4-001 a 005), preparada para exhibir ambas ramas de notificación y distinta de la corrida de carga de cincuenta órdenes: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com).

CAPÍTULO 5: RESULTADOS Y ANÁLISIS

5.1 Resultados del Flujo 1 — Pipeline de Órdenes

5.1.1 Pruebas funcionales

Se ejecutaron cinco pruebas funcionales sobre el pipeline de procesamiento de órdenes. En cada caso se verificó la respuesta del webhook, el estado de la base de datos y los artefactos generados (emails, eventos) (ver Tabla 5.1).

Tabla 5.1: Resultados de las pruebas funcionales.

Prueba	Escenario	Resultado
PF-01	Orden exitosa (stock suficiente)	PASS — orden confirmada, stock decrementado, email enviado
PF-02	Orden sin stock	PASS — orden rechazada con status no_stock, email de aviso
PF-03	Stock bajo (bajo umbral)	PASS — orden confirmada + alerta low_stock_alert registrada
PF-04	SKU inválido	PASS — orden rechazada con error controlado
PF-05	Orden duplicada	PASS — orden rechazada

Las cinco pruebas resultaron exitosas (100% de aprobación). En todos los casos el pipeline actualizó correctamente los timestamps en orders y las notificaciones se enviaron al destinatario esperado.

5.1.2 Métricas de tiempo

Las métricas de tiempo obtenidas para el Flujo 1 se resumen en la Tabla 5.2.

Tabla 5.2: Métricas de tiempo del Flujo 1.

Métrica	Valor sobre las 50 órdenes medidas
MTTD (tiempo de detección)	media 0,009 s · desvío 0,003 s · mediana 0,009 s · IC 95 % [0,008; 0,009] s
MTTR (tiempo de resolución)	media 0,054 s · desvío 0,003 s · mediana 0,054 s · IC 95 % [0,053; 0,055] s
Tiempo total end-to-end	media 0,063 s · desvío 0,005 s · mediana 0,062 s · IC 95 % [0,061; 0,064] s

El tiempo total end-to-end medido fue de 0,063 s en promedio (desvío estándar 0,005 s; mediana 0,062 s; mínimo 0,057 s; máximo 0,095 s; $n = 50$), lo que significa que desde la recepción de una orden hasta la escritura de la marca de notificación al cliente transcurren menos de siete centésimas de segundo, incluyendo la verificación de stock, la actualización del inventario y el envío del correo al servidor SMTP. El valor máximo observado (0,095 s) corresponde a la primera orden de la corrida y se atribuye al arranque en frío del motor de flujos; se reporta sin excluirlo de la muestra.

5.1.3 Pruebas de carga y de concurrencia

La corrida de carga secuencial (E1.a) procesó las 50 órdenes enviadas sin un solo error de pipeline y con cobertura completa de las tres marcas temporales: las 50 órdenes registraron `received_at`, `processed_at` y `notified_at`, incluidas las 15 que siguieron la rama de stock insuficiente. Sobre esa población se calculan las métricas de la Tabla 5.2.

Tabla 5.3: Resultados de las pruebas de carga y de concurrencia.

Métrica	Resultado
E1.a — Órdenes enviadas (secuenciales, espaciadas 2 s)	50
E1.a — Órdenes procesadas	50/50 (100 %)
E1.a — Órdenes confirmadas	35 (70 %)
E1.a — Órdenes sin stock	15 (30 %)
E1.a — Errores de pipeline	0
E1.b — Rondas × solicitudes simultáneas	$6 \times 20 = 120$ órdenes
E1.b — Stock inicial por ronda	5 unidades (PROD-005)
E1.b — Órdenes confirmadas por ronda	5 en las 6 rondas (ninguna sobreventa)
E1.b — Productos con stock negativo	0
E1.b — Órdenes sin procesar bajo concurrencia	49 de 120 (40,8 %)
E1.b — MTTD medio bajo concurrencia	0,092 s ($\pm 0,025$ s; máx. 0,157 s)
E1.b — MTTR medio bajo concurrencia	0,100 s (derivado: end-to-end – MTTD; el desvío no es derivable de las cifras publicadas)
E1.b — End-to-end medio bajo concurrencia	0,192 s ($\pm 0,024$ s; máx. 0,260 s)

Total de órdenes con data_source = 'measured'	173 (50 de E1.a + 120 de E1.b + 3 de verificación)
---	--

La prueba de concurrencia (E1.b) arroja dos resultados que conviene reportar por separado, porque apuntan en direcciones distintas. El primero es concluyente y favorable: en las seis rondas ejecutadas, contra un stock inicial de 5 unidades y con 20 solicitudes simultáneas en cada una, el sistema confirmó exactamente 5 órdenes en todas y cada una de las rondas. No se registró sobreventa en ningún caso, ni quedó ningún producto con stock negativo. La restricción de integridad a nivel de esquema y el carácter transaccional de la sentencia de descuento resultaron suficientes para impedir la condición de carrera, que era exactamente la propiedad que el ensayo se proponía refutar.

El segundo resultado matiza al primero y corresponde declararlo con la misma claridad: bajo concurrencia el pipeline no perdió integridad, pero sí perdió capacidad de proceso. De las 120 órdenes generadas, 49 (el 40,8 %) quedaron registradas sin marca de processed_at, es decir que el motor de flujos aceptó la solicitud pero no completó su ejecución. Ninguna quedó en estado de error ni produjo un descuento de stock incorrecto: simplemente no avanzaron. El comportamiento observado es, en consecuencia, seguro pero no elástico. Las métricas también se degradan de forma apreciable respecto de la corrida secuencial: el MTTD medio pasa de 0,009 s a 0,092 s —un factor de diez— y el tiempo end-to-end de 0,063 s a 0,192 s, valores que de todos modos se mantienen tres órdenes de magnitud por debajo del criterio operativo de 30 segundos. Esta limitación de capacidad motiva las recomendaciones de encolado y control de admisión del Capítulo 7.

5.1.4 Baseline de atención manual

Para dotar de un término de comparación medido, y no estimado, a la afirmación central del trabajo, se cronometró el procesamiento manual de una orden siguiendo el protocolo descrito en la Sección 3.5.5. La Tabla 5.4 presenta los descriptivos por fase y del tiempo total sobre las diez órdenes válidas.

Tabla 5.4: Descriptivos del tiempo de procesamiento manual, por fase y total.

Fase	n	Media	Mediana	Desvío	Mín.	Máx.
T1 — Lectura y verificación	10	7,05 s	7,12 s	2,00 s	4,37 s	11,39 s
T2 — Actualización en la base	10	5,80 s	5,27 s	1,67 s	4,45 s	9,09 s
T3 — Redacción de la notificación	10	36,27 s	34,91 s	6,55 s	30,45 s	53,67 s
Total por	10	49,13 s	46,37 s	8,15 s	42,75 s	69,80 s

orden						
-------	--	--	--	--	--	--

El tiempo marginal de procesamiento manual de una orden resultó de 49,13 s, con un intervalo de confianza al 95 % de [43,30 s; 54,95 s] calculado por la distribución t de Student con nueve grados de libertad. El coeficiente de variación fue del 16,6 %, dispersión baja que indica que el procedimiento se ejecutó de manera estable a lo largo de la serie. Conforme se estableció en la Sección 3.5.5, este valor mide el costo de procesar una orden adicional y no la latencia percibida por el cliente.

El reparto del esfuerzo entre fases es en sí mismo un hallazgo. La redacción de la notificación al cliente concentra el 73,8 % del tiempo total (36,27 s de 49,13 s), mientras que la lectura y verificación de stock insume el 14,3 % (7,05 s) y la actualización de la base el 11,8 % restante (5,80 s). Es decir que el grueso del costo del procesamiento manual no está en la consulta ni en la escritura de datos, que son las operaciones que un operador humano ejecuta con rapidez, sino en la comunicación con el cliente. Desagregado por rama, el tiempo fue de 50,96 s en las siete órdenes con stock disponible y de 44,84 s en las tres sin stock, diferencia atribuible a que la plantilla de confirmación exige cargar el importe total y la de rechazo no.

El valor admite una validación cruzada que conviene explicitar, porque proviene de un instrumento independiente del cronómetro. Las marcas temporales que el operador fue escribiendo en la base durante la medición permiten calcular el intervalo entre notificaciones de órdenes consecutivas, que es la misma magnitud marginal medida por otra vía: ese cálculo arroja 51,28 s de media (desvío 5,22 s). La diferencia de 2,15 s respecto de los 49,13 s cronometrados corresponde a los intervalos muertos entre terminar una orden y comenzar la siguiente, que el cronómetro no contabiliza y el reloj de la base sí. Dos instrumentos independientes coinciden dentro del 4 %, lo que respalda la validez de la medición.

El registro en la base aporta además un resultado complementario sobre el encolamiento. Como las doce órdenes ingresaron en un mismo lote (Sección 3.5.5), el tiempo transcurrido entre la recepción del lote y la notificación de cada orden crece de forma monótona: 72,4 s para la primera atendida y 533,9 s —ocho minutos y catorce segundos— para la última, con una media de 313,37 s y un desvío de 154,57 s (Tabla I.2). Corresponde subrayar que esta cifra no es una propiedad del proceso manual sino del tamaño del lote elegido, y que por lo tanto no se la utiliza como término de comparación: un lote mayor la aumentaría sin que el operador trabajase más lento. Su valor es ilustrativo y apunta a la razón de fondo por la que la automatización importa en este dominio: el pipeline automatizado procesó 50 órdenes consecutivas sin que el tiempo de la última se degradara respecto de la primera, mientras que en el proceso manual la espera del último cliente creció un factor de siete frente a la del primero.

Corresponden dos advertencias sobre la interpretación de este valor. La primera surge de un control previsto en el protocolo: la correlación de Pearson entre el orden

de ejecución y el tiempo empleado fue de $r = -0,693$, magnitud que indica que el efecto de aprendizaje siguió actuando dentro de la serie válida. En consecuencia, la media de 49,13 s subestima el tiempo que emplearía un operador no entrenado, y el baseline debe leerse como un piso y no como un valor central. La segunda es que la medición se realizó sobre un único operador, lo que impide estimar la variabilidad entre personas; se declara como limitación en la Sección 5.4.1.

Por último, este valor no incluye la latencia de detección, esto es, el tiempo que una orden permanece a la espera antes de que un operador advierta su llegada. Esa magnitud no es observable sin convertirla en un parámetro elegido por los autores, y por lo tanto no se suma al número medido. A título ilustrativo puede señalarse que, si un operador revisara la bandeja cada quince minutos, la espera media esperable sería de 7,5 minutos, valor que por sí solo excede en dos órdenes de magnitud al tiempo de procesamiento cronometrado. Se consigna como escenario declarado y no como medición.

5.2 Resultados del Flujo 2 — Chatbot (canal simulado formato WhatsApp y canal Telegram real)

5.2.1 Pruebas funcionales del chatbot

Se ejecutaron cinco pruebas funcionales sobre el Flujo 2, con el mismo criterio con que se ejecutaron las del Flujo 1: se verificó la respuesta entregada por el canal, el estado de la base de datos y los artefactos generados. Su diseño se detalla en la Tabla 4.10; aquí se reporta su ejecución. Las cinco resultaron aprobadas. C1, sobre una consulta de tipo frecuente, fue clasificada como FAQ y respondida por el canal de origen. C2, sobre estado de pedido, fue clasificada como ESTADO_PEDIDO y disparó la consulta a la tabla de órdenes, devolviendo el estado del pedido referido en el mensaje. C3, sobre un reclamo, fue clasificada como RECLAMO y generó el ticket correspondiente. C4, sobre un reclamo redactado en términos de urgencia, fue clasificada como RECLAMO, generó ticket y quedó registrada con la marca de urgencia en alto. C5, sobre una consulta abierta, fue clasificada como GENERAL y respondida por el modelo sin escalar. En los cinco casos la interacción quedó registrada con sus dos marcas temporales, de modo que el TMR de cada una es reconstruible.

Corresponde una precisión sobre el alcance de estas pruebas, para que no se las confunda con la evaluación del clasificador. Son pruebas funcionales de un caso por intención: verifican que el flujo se recorra completo y que cada rama se active, no que la clasificación sea correcta en general ni que el contenido de la respuesta sea adecuado. Lo primero se mide sobre el corpus de 150 mensajes en la Sección 5.2.3; lo segundo no se midió en este trabajo, según se declara en la Sección 3.2.

5.2.2 Tiempos de respuesta (TMR)

Se enviaron 150 mensajes de prueba distribuidos entre los cuatro intents posibles, simulando consultas reales de clientes con errores ortográficos, abreviaciones y mensajes ambiguos (ver Tabla 5.5).

Tabla 5.5: Tiempo medio de respuesta por intención sobre el corpus evaluado.

Intent	Mensajes clasificados	TMR promedio	TMR mín.	TMR máx.	Desvío est.	No escalada
FAQ	45	1,54 s	1,19 s	2,91 s	±0,33 s	100%
RECLAMO	40	1,54 s	1,31 s	1,94 s	±0,15 s	0% (escala a ticket por diseño)
ESTADO_PEDIDO	36	1,45 s	1,22 s	1,91 s	±0,15 s	100%
GENERAL	29	1,28 s	1,04 s	1,78 s	±0,18 s	100%
Total	150	1,47 s	1,04 s	2,91 s	±0,25 s	~73%

TMR promedio: 1,47 s sobre los 150 mensajes del corpus. Este valor confirma H2a (TMR < 10 s) con amplio margen.

Tasa de no escalada: 73,3 % (110 de 150 mensajes). Si se excluyen los reclamos —que escalan a ticket por diseño del sistema y no por falla del clasificador—, la tasa asciende al 100 %: el chatbot respondió sin derivar a un operador humano la totalidad de los mensajes que no eran reclamos. Corresponde precisar qué mide este indicador y qué no mide. Mide que el flujo se completó sin escalar: el sistema clasificó el mensaje, generó una respuesta y la entregó por el canal de origen. No mide que esa respuesta fuera correcta. La corrección del contenido entregado al cliente no fue evaluada en este trabajo, y la omisión es material en las consultas de tipo FAQ: como se documenta en la Sección 4.4.3, el contexto de la base de conocimiento no llega al prompt, de modo que esas respuestas provienen del conocimiento general del modelo y no de las políticas efectivamente vigentes en el comercio. Llamar «tasa de resolución» a este indicador sobreestimaría lo medido; se lo denomina en consecuencia tasa de no escalada.

Tickets generados: 40 tickets de soporte, generados a partir de los mensajes clasificados como RECLAMO por el modelo (40 predicciones), no de los 42 reclamos reales; el enrutamiento opera sobre la predicción del clasificador.

Validación del segundo canal (Telegram): para verificar que la arquitectura multicanal es real y no solo declarada, el canal de Telegram se implementó de extremo a extremo —recepción por webhook, clasificación con GPT-4o-mini,

consulta a la base de datos y respuesta por el mismo canal— y se validó sobre una muestra de 45 interacciones reales. A diferencia del canal simulado —que adopta el formato de la WhatsApp Cloud API en la recepción y entrega por SMTP a un capturador local, sin tocar la red de WhatsApp en ningún extremo—, Telegram opera contra la API real, por lo que su TMR incluye la latencia de red efectiva (ver Tabla 5.6).

Tabla 5.6: Tiempo medio de respuesta por intención en el canal Telegram.

Intent	Mensajes	TMR promedio	TMR mín.	TMR máx.
ESTADO_PEDIDO	15	3,32 s	2,29 s	5,44 s
FAQ	8	3,38 s	2,52 s	5,75 s
GENERAL	12	2,84 s	2,18 s	5,96 s
RECLAMO	10	2,72 s	2,45 s	2,93 s
Total	45	3,07 s	2,18 s	5,96 s

El TMR global del canal Telegram fue de 3,07 s —superior al del canal simulado (1,47 s) por la latencia de red real, pero holgadamente por debajo del umbral de 10 s—, lo que confirma que H2a se sostiene también sobre un canal en producción. Corresponde precisar el alcance de esta validación: las 45 interacciones de Telegram son adicionales al corpus de 150 mensajes y no un subconjunto suyo, y no cuentan con etiquetas de referencia. En consecuencia, la validación del segundo canal cubre la latencia de respuesta pero no la exactitud de clasificación, que se reporta únicamente sobre el corpus etiquetado. El canal de correo (Gmail) se mantiene como diseño (Capítulo 7): su mecanismo de consulta periódica (polling) impone un piso de latencia de aproximadamente un minuto, incompatible con la métrica de segundos.

5.2.3 Precisión de clasificación de intents

Se evaluó la precisión de clasificación comparando el intent asignado por GPT-4o-mini contra el intent esperado para los 150 mensajes de prueba. El intent esperado se etiquetó de forma ciega respecto a la salida del modelo (ver Sección 3.5.3) (ver Tabla 5.7).

Tabla 5.7: Exhaustividad (recall) de clasificación por intención.

Intent	Mensajes	Clasificados correctamente	Clasificados incorrectamente	Exhaustividad (recall)
FAQ	48	44	4	91,7%
RECLAMO	42	38	4	90,5%
ESTA	35	32	3	91,4%

DO_P EDID O				
GENE RAL	25	25	0	100%
Total	150	139	11	92,7%

Matriz de confusión (intent real vs. predicho por el modelo) (ver Tabla 5.8):

Tabla 5.8: Matriz de confusión del clasificador.

Real \ Predicho	ESTADO_PEDIDO	FAQ	GENERAL	RECLAMO	Total real
ESTADO_PEDIDO	32	1	1	1	35
FAQ	1	44	2	1	48
GENERAL	0	0	25	0	25
RECLAMO	3	0	1	38	42
Total predicho	36	45	29	40	150

Precisión, exhaustividad (recall) y F1 por clase (ver Tabla 5.9):

Tabla 5.9: Precisión, exhaustividad y F1 por clase.

Clase	Precisión	Recall	F1
ESTADO_PEDIDO	88,9%	91,4%	0,901
FAQ	97,8%	91,7%	0,946
GENERAL	86,2%	100%	0,926
RECLAMO	95,0%	90,5%	0,927
Global	—	92,7%	—

Accuracy global: 92,7% (139/150), lo que supera el umbral mínimo de 85% establecido en H2. Los 11 errores se concentraron en fronteras semánticas entre categorías —principalmente RECLAMO asignado a ESTADO_PEDIDO y FAQ/GENERAL entre sí—, coherente con mensajes cortos y ambiguos.

Acuerdo entre anotadores: sobre la submuestra de 50 mensajes etiquetada por el evaluador independiente (protocolo en §3.5.3), el acuerdo observado con el ground truth fue de 47 coincidencias sobre 50 ($P_o = 0,940$), con un coeficiente κ de Cohen de 0,919, valor que corresponde a un acuerdo casi perfecto en la escala de Landis y Koch (1977). Los tres desacuerdos se concentran en fronteras semánticas reales entre categorías —“me pueden ayudar?” (GENERAL/FAQ), “mi pedido no llega y ya pasaron los 5 días” (RECLAMO/FAQ) y una consulta de stock referida a un pedido ya realizado (FAQ/ESTADO_PEDIDO)—, es decir en los mismos casos ambiguos donde se concentran los errores del clasificador. Este resultado indica que el ground truth no responde a un criterio idiosincrásico del equipo.

|

La categoría con mayor precisión fue FAQ (97,8 %), seguida de RECLAMO (95,0 %), ESTADO_PEDIDO (88,9 %) y GENERAL (86,2 %). El patrón se invierte al observar la exhaustividad: GENERAL alcanza el 100 % (25 de 25), es decir que el modelo no dejó sin identificar ningún mensaje que perteneciera realmente a esa categoría, pero incorporó a ella cuatro mensajes provenientes de otras clases. Esa asimetría — exhaustividad máxima junto con la precisión más baja del conjunto— es consistente con el carácter abierto de la categoría: al no estar acotada por patrones específicos, GENERAL opera de hecho como clase residual y absorbe los mensajes que el modelo no consigue asignar con confianza a las otras tres (ver Figura 9).

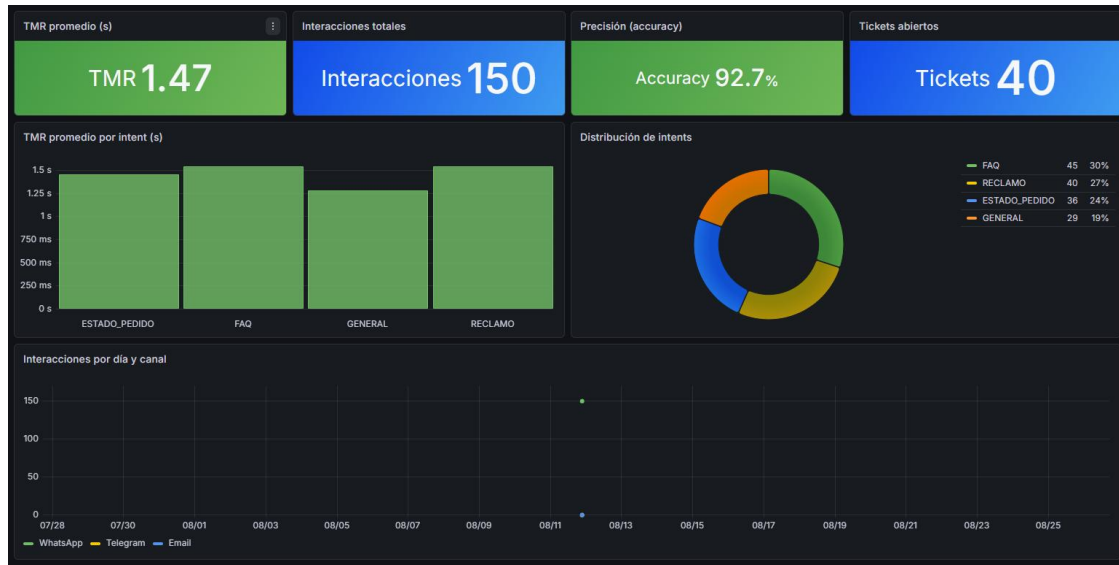


Figura 9: Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la precisión de clasificación y la distribución de intenciones sobre el corpus evaluado. El panel de interacciones por día y canal se alimenta de la vista del corpus, que está acotada a la ventana temporal de esa corrida: por eso muestra únicamente las 150 interacciones del canal simulado y no las 45 de Telegram, que se ejecutaron fuera de esa ventana y se reportan en la Tabla 5.6. La serie de correo aparece en cero porque el canal está previsto en el esquema y no implementado.

5.3 Contrastación de hipótesis

El contraste de cada hipótesis contra su criterio de aceptación se sintetiza en la Tabla 5.10.

Tabla 5.10: Contrastación de hipótesis.

Hipótesis	Criterio	Resultado obtenido	Veredicto
H1: Pipeline sustancialment e más rápido que el proceso	End-to-end menor que el baseline manual; < 30 s	0,063 s frente a 49,13 s del baseline manual (factor $\approx 780\times$; IC 95 % por el teorema de Fieller: $686\times$ a $875\times$). U de	CONFIRMADA

manual	como criterio operativo	Mann-Whitney = 0 con separación completa de ambas series ($p = 2,7 \times 10^{-11}$); t de Welch = 19,05, gl = 9,0, $p = 1,4 \times 10^{-8}$. Muy por debajo del criterio operativo de 30 s.	
H2a: TMR < 10 s	TMR promedio < 10 s	1,47 s promedio	CONFIRMADA
H2b: Exactitud (accuracy) de clasificación $\geq$ 85 %	Accuracy $\geq$ 85%	92,7 % (139/150); IC 95 % de Wilson [87,3 %; 95,9 %]	CONFIRMADA

Las tres hipótesis fueron confirmadas por los datos experimentales. En particular:

- H1 se confirma en sus dos términos. En el término comparativo, que es el sustantivo: el costo marginal de procesar una orden en el pipeline (0,063 s; $n = 50$) resulta unas 780 veces menor que el del procesamiento manual, medido en 49,13 s por orden (IC 95 % del factor: $686\times$ a $875\times$, por el teorema de Fieller). Ambos términos miden la misma magnitud —el tiempo que insume una orden adicional— conforme se precisa en la Sección 3.5.5. En el criterio operativo secundario: ese mismo tiempo representa el 0,2 % del umbral de 30 segundos. La Sección 5.4 precisa por qué el factor debe leerse como una cota superior.
- H2a se confirma con amplio margen: el TMR del corpus (1,47 s) representa el 14,7 % del umbral de 10 s, y el del canal Telegram real (3,07 s) el 30,7 %.
- H2b se confirma con un accuracy global de 92,7 % (139/150), cuyo intervalo de confianza de Wilson al 95 % —[87,3 %; 95,9 %]— tiene el límite inferior por encima del umbral del 85 %. El procedimiento de cálculo se detalla en el Anexo J.

Sobre la confirmación de H2b corresponde una precisión metodológica adicional. El umbral del 85% se fija como criterio del equipo sobre el desempeño global del clasificador, y el accuracy total del 92,7% (139/150) lo supera. Ninguna subcategoría queda por debajo del umbral: por exhaustividad (recall) las cuatro clases se ubican entre 90,5% y 100%, y por precisión entre 86,2% y 97,8%. Se reporta además el intervalo de confianza de Wilson al 95% del accuracy global, [87,3 %; 95,9 %], cuyo límite inferior supera el umbral. La asimetría entre precisión y exhaustividad de GENERAL es coherente con la naturaleza abierta de GENERAL, que actúa como categoría residual para mensajes que no encajan claramente en FAQ, ESTADO_PEDIDO ni RECLAMO, y donde la frontera con FAQ es semánticamente difusa (consultas como '¿tienen descuentos?' admiten ambas clasificaciones). Se considera que el promedio global es la métrica adecuada para contrastar H2b dado

que la hipótesis no fue formulada por subcategoría; no obstante, se reconoce esta asimetría como una limitación del clasificador que se aborda en §5.4.1 y se incorpora a las líneas futuras del Capítulo 7.

Contraste estadístico de H1. Afirmar que una serie es sustancialmente más rápida que otra describe una diferencia de magnitud, no una inferencia: para sostener que esa diferencia no proviene del azar del muestreo hace falta una prueba de contraste, y se la reporta a continuación. Las dos series están completamente separadas: el máximo de la serie automatizada (0,095 s sobre $n = 50$) es inferior al mínimo de la serie manual (42,754 s sobre $n = 10$), de modo que ningún par de observaciones se cruza. La U de Mann-Whitney vale por lo tanto 0, con un valor p exacto bilateral de $2/C(60, 10) = 2,7 \times 10^{-11}$ y un tamaño del efecto rango-biserial de 1,000, el máximo posible. La t de Welch sobre los descriptivos publicados arroja $t = 19,05$ con 9,0 grados de libertad y $p = 1,4 \times 10^{-8}$ (d de Cohen = 15,3). Ambos contrastes se calculan íntegramente a partir de las cifras de las Tablas 5.2, 5.4 y I.1, de modo que son reproducibles sin acceso a los datos crudos. Corresponde una salvedad de alcance: la prueba establece que la diferencia observada no es atribuible al azar del muestreo, pero no resuelve la asimetría de constructo entre ambos términos —el alcance instrumentado de cada uno— que se discute en la Sección 5.4. La significación estadística confirma que la diferencia existe; su interpretación sigue estando acotada por lo que cada instrumento midió.

5.4 Análisis comparativo y discusión

Los resultados confirman la premisa central del trabajo: la automatización reduce de manera sustancial los tiempos operativos del ciclo post-venta. La reducción del costo marginal de procesamiento de una orden es de un factor aproximado de 780 veces respecto del baseline manual medido en la Sección 5.1.4 (49,13 s $\rightarrow$ 0,063 s), con un intervalo de confianza al 95 % de $686\times$ a $875\times$. Ese intervalo se obtiene por el teorema de Fieller, que es el procedimiento exacto para el cociente de dos medias y propaga la incertidumbre de ambos términos. Se consigna, por transparencia del cálculo, que la propagación simplificada —tratar el denominador automatizado como constante y arrastrar solo la incertidumbre del baseline— arroja $687\times$ a $872\times$: la diferencia entre ambos procedimientos es inferior al 0,4 % porque el desvío del término automatizado (0,005 s sobre $n = 50$) es cuatro órdenes de magnitud menor que el del término manual. Corresponde precisar el alcance de esa comparación en tres puntos. Primero, ambos términos miden la misma magnitud: el tiempo que insume procesar una orden adicional. No se comparan contra el tiempo de espera observado en la cola manual (313,37 s de media), porque esa cifra depende del tamaño del lote y no del proceso, y utilizarla habría inflado el factor hasta cerca de $5.000\times$. Segundo, el factor debe leerse como una cota superior y no como el valor esperable en un despliegue productivo: el término automatizado proviene de un entorno de laboratorio en el que la notificación se entrega a un capturador SMTP alojado en el mismo host, sin tránsito de correo real ni latencia de servicios de

terceros, mientras que el término manual se midió sobre un operador que ya conocía el procedimiento y trabajaba con una plantilla fija, dos decisiones deliberadamente conservadoras que reducen el numerador. Tercero, el alcance instrumentado no es idéntico: los 0,063 s cubren el intervalo `received_at` → `notified_at`, es decir escrituras sobre la base local, mientras que el baseline manual cubre la tarea humana completa de lectura, verificación, actualización y redacción. En atención al cliente, el chatbot responde en 1,47 s en promedio sobre el corpus evaluado y en 3,07 s sobre el canal Telegram real.

El modelo GPT-4o-mini demostró ser adecuado para la clasificación de intenciones en dominios acotados, manejando correctamente mensajes con errores ortográficos, abreviaciones y varias preguntas dentro de un mismo mensaje. El dato relevante es que ese 92,7 % de precisión se alcanzó en régimen zero-shot: sin ajuste fino, sin ejemplos etiquetados en el prompt y sin inyección de la base de FAQ, con un prompt de sistema de poco más de mil caracteres que se limita a enumerar las cuatro categorías y a fijar el formato de salida (Anexo H). El resultado debe leerse por lo tanto como un piso del desempeño alcanzable en la tarea y no como su techo: las técnicas de prompting sistematizadas por Liu et al. (2023), así como el cableado del contexto de FAQ que el flujo ya construye, constituyen margen de mejora todavía sin explotar.

5.4.1 Limitaciones observadas

(a) Dependencia de servicios externos y limitaciones de los LLMs: el Flujo 2 depende de la API de OpenAI; una caída del servicio inhabilita la clasificación de intents y la generación de respuestas. Adicionalmente, los modelos de lenguaje como GPT-4o-mini presentan limitaciones conocidas en entornos de producción: (i) **alucinación** — el modelo puede generar respuestas plausibles pero factualmente incorrectas, especialmente ante consultas fuera del dominio de entrenamiento (Huang et al., 2023); (ii) **prompt injection** — un usuario malicioso podría formular un mensaje diseñado para alterar las instrucciones del sistema y obtener respuestas no autorizadas (Perez & Ribeiro, 2022). Estas limitaciones justifican la supervisión humana periódica del chatbot en entornos de producción y la implementación de filtros de validación sobre las respuestas generadas. El Flujo 1 opera de forma independiente y no se ve afectado por estas limitaciones.

(a-bis) Desempeño por subcategoría: el accuracy global del 92,7% se distribuye de forma razonablemente homogénea. Por exhaustividad (recall): GENERAL 100% (25/25), FAQ 91,7%, ESTADO_PEDIDO 91,4% y RECLAMO 90,5%. Por precisión: FAQ 97,8%, RECLAMO 95,0%, ESTADO_PEDIDO 88,9% y GENERAL 86,2%. Todas las clases superan el umbral del 85% en ambas métricas. Este patrón sugiere que la categoría GENERAL — definida operacionalmente como categoría residual — concentra los casos de frontera semántica con FAQ. Una mitigación posible para trabajos futuros consiste en redefinir GENERAL mediante subcategorías más específicas (saludo, despedida, consulta abierta no clasificable) o en incorporar al

prompt ejemplos etiquetados de esta clase (few-shot), que hoy no tiene ninguno, sin requerir fine-tuning del modelo.

(b) Entorno de prueba local: las métricas de tiempo se obtuvieron en un entorno Docker sobre hardware de desarrollo (notebook). En un servidor de producción con mayor capacidad de cómputo y menor latencia de red, los valores de MTTD y MTTR podrían ser inferiores; sin embargo, el TMR está dominado por la latencia de la llamada de inferencia a la API de OpenAI, que no depende del hardware local: el rango observado en este trabajo va de 1,47 s a 3,07 s según el canal.

(c) Tamaño del conjunto de prueba: el conjunto de 150 mensajes cubre los escenarios más comunes pero no la variabilidad completa de un entorno real. La precisión del 92,7% podría variar con mensajes de dominio diferente al entrenado.

(c-bis) Ventana temporal de la corrida del chatbot: las 150 llamadas del corpus se ejecutaron dentro de un intervalo de sesenta minutos de un mismo día, según acota la propia vista que aísla la población (Anexo A). La decisión es coherente con la mitigación declarada en la Sección 3.6.4 —acotar la ventana reduce la variabilidad del servicio externo dentro de la medición— pero tiene un costo que corresponde consignar: el desvío de $\pm 0,25$ s que acompaña al tiempo medio de respuesta describe la dispersión dentro de esa hora y no la variabilidad del proveedor de inferencia entre días y franjas horarias, que es justamente la fuente de incertidumbre que la amenaza a la validez identifica. El valor reportado debe leerse en consecuencia como el desempeño en una ventana favorable y no como el esperable a lo largo del día.

(d) Ausencia de grupo de control y de diseño experimental comparativo: el contraste de H1 se ejecutó y se reporta al cierre de la Sección 5.3 —U de Mann-Whitney sobre las dos series y t de Welch sobre sus descriptivos—, de modo que la diferencia observada no es atribuible al azar del muestreo. Lo que ese contraste no resuelve es de otro orden y conviene separarlo en tres puntos. Primero, el baseline manual se midió sobre un único operador: la prueba compara dos series de tiempos, no dos poblaciones de operadores, y por lo tanto no permite estimar la variabilidad entre personas ni descartar el efecto del evaluador. Segundo, no hubo asignación al azar a condiciones ni grupo de control; las dos series provienen de dos procedimientos distintos aplicados a una tarea equivalente, no de un experimento controlado, de modo que la atribución causal de la diferencia a la automatización descansa en el diseño del caso y no en la aleatorización. Tercero, y como se precisa en la propia Sección 5.3, la significación estadística confirma que la diferencia existe pero no resuelve la asimetría de constructo entre ambos términos: cada instrumento cubre un alcance distinto. Una comparación inferencial en sentido pleno exigiría varios operadores independientes y un diseño experimental que excede el alcance declarado de este trabajo, y queda planteada como línea futura en el Capítulo 7.

CAPÍTULO 6: CONCLUSIONES

6.1 Respuesta a la pregunta de investigación

La presente investigación planteó la siguiente pregunta: *¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de MTTD, MTTR y TMR?*

Los resultados demuestran que la automatización reduce los tiempos de forma sustancial: el costo marginal de procesar una orden en el pipeline (0,063 s end-to-end; n = 50) es aproximadamente 780 veces menor que el del procesamiento manual, cronometrado en 49,13 s por orden sobre diez órdenes (Sección 3.5.5) y validado de forma independiente contra las marcas temporales de la base, que arrojan 51,28 s para la misma magnitud. La Sección 5.4 acota explícitamente ese factor como cota superior. El TMR del chatbot es de 1,47 s en promedio sobre el corpus evaluado. La disponibilidad ininterrumpida es una propiedad esperable de la arquitectura —el sistema no depende de un operador humano para responder— pero no fue medida: no se ejecutó ensayo de disponibilidad ni se observó el servicio durante una ventana prolongada, de modo que no se la reporta como resultado. Las tres hipótesis de trabajo (H1, H2a y H2b) fueron confirmadas por los datos experimentales (ver Sección 5.3).

6.2 Cumplimiento de objetivos específicos

El grado de cumplimiento de cada objetivo específico se detalla en la Tabla 6.1.

Tabla 6.1: Cumplimiento de objetivos específicos.

Obj.	Enunciado	Resultado	Estado
OE1	Pipeline de órdenes: MTTD y MTTR < 30 s end-to-end	MTTD: 0,009 s / MTTR: 0,054 s / Total: 0,063 s (n = 50, corrida E1.a). 50/50 órdenes procesadas sin errores. Bajo concurrencia (E1.b, 120 órdenes) el pipeline sostuvo la atomicidad del descuento de stock sin sobreventa, pero perdió capacidad de proceso: 49 de 120 órdenes (40,8 %) quedaron sin marca de	CUMPLIDO (régimen secuencial)

		processed_at y las métricas se degradaron: el tiempo de detección por un factor de diez (0,009 s → 0,092 s) y el extremo a extremo por un factor de tres (0,063 s → 0,192 s), según la Sección 5.1.3. El objetivo se da por cumplido en régimen secuencial; en régimen concurrente el comportamiento es seguro pero no elástico.	
OE2	Chatbot: TMR < 10 s y exactitud (accuracy) ≥ 85 %	TMR: 1,47 s en el canal simulado vía SMTP local (n = 150) y 3,07 s en el canal Telegram real (n = 45). Accuracy de clasificación: 92,7 % sobre el corpus etiquetado. Dos canales implementados y validados; Gmail queda como diseño (§7).	CUMPLIDO (dos canales)
OE3	Schema PostgreSQL con las tablas, vistas e índices necesarios	7 tablas (products, orders, order_items, interactions, tickets, faq_responses y stock_alerts), 6 vistas de métricas y 12 índices de rendimiento sobre las columnas de filtrado y de unión. El objetivo se formuló en términos cualitativos —las tablas, vistas e índices necesarios para instrumentar el ciclo— y el esquema desplegado los provee: MTDD, MTTR y TMR se calculan	CUMPLIDO

		íntegramente desde las vistas, sin cómputo externo. Se consigna que order_items está en el esquema pero ningún nodo del pipeline la escribe: el sistema implementado carga órdenes mono-producto (Sección 4.2.1).	
OE4	Dashboards Grafana con MTTD, MTTR, TMR en tiempo real	13 paneles distribuidos en dos tableros: 6 para el pipeline de órdenes y 7 para el chatbot, con conexión directa a las vistas de PostgreSQL.	CUMPLIDO
OE5	Validación mediante pruebas funcionales con datos simulados	10 pruebas funcionales (5 por flujo), con su diseño en las Tablas 4.9 y 4.10 y sus resultados en las Secciones 5.1.1 y 5.2.1; 1 corrida de carga secuencial de 50 órdenes y 1 prueba de concurrencia de 6 rondas × 20 solicitudes simultáneas, con datos crudos y manifiesto de ejecución versionados.	CUMPLIDO

Los cinco objetivos específicos fueron cumplidos, con dos precisiones sobre el alcance de ese cumplimiento. OE3 y OE4 se formularon en términos cualitativos en la Sección 1.5.2 —las tablas, vistas de métricas e índices necesarios para instrumentar el ciclo, y los dashboards para visualizar MTTD, MTTR y TMR— y el sistema desplegado los satisface: siete tablas, seis vistas y doce índices sostienen el cálculo de las tres métricas sin cómputo externo, y trece paneles repartidos en dos tableros las visualizan en tiempo real. No se los contrasta contra una cifra comprometida de antemano porque los objetivos no la fijaron. OE1 se cumple en régimen secuencial: bajo concurrencia el pipeline conserva la integridad transaccional pero pierde el 40,8 % de las órdenes, según se consigna en su propia celda y se analiza en la Sección 5.1.3. OE2 se cumplió sobre los dos canales efectivamente implementados y medidos, quedando el canal de correo como diseño documentado en el Capítulo 7.

6.3 Conclusiones sustantivas

Sobre la viabilidad de n8n como orquestador: n8n demostró ser una plataforma viable para la automatización de procesos de negocio de complejidad media en el contexto de PyMEs. La capacidad de diseñar workflows visualmente, combinada con la posibilidad de ejecutar JavaScript en nodos *Function* y la integración nativa con PostgreSQL, SMTP y la API de OpenAI, permitió implementar ambos flujos sin escribir código de aplicación. Esto reduce la barrera técnica de adopción para empresas sin equipos de desarrollo dedicados.

Sobre la efectividad de GPT-4o-mini en atención al cliente: el modelo alcanzó un **accuracy del 92,7 % en la clasificación de intenciones sin ajuste fino y en régimen zero-shot**, es decir guiado exclusivamente por una instrucción que enumera las cuatro categorías y fija el formato de salida, sin ejemplos etiquetados. Que un modelo de bajo costo alcance ese desempeño con un **prompt de mil caracteres** es, a juicio de los autores, el hallazgo más transferible del trabajo para una PyME: reduce la barrera de entrada de la clasificación automática de consultas a la redacción cuidadosa de una instrucción. El margen de mejora disponible —incorporar ejemplos al prompt y cablear el contexto de FAQ que el flujo ya recupera— queda documentado como línea futura.

Sobre la arquitectura Docker Compose: la infraestructura basada en cuatro contenedores está diseñada para ser reproducible. El entorno se levanta con un único comando (`docker compose up -d`), las imágenes están fijadas por versión explícita, los datos persisten entre reinicios mediante volúmenes nombrados, y el esquema, los flujos y los scripts de medición están versionados en el repositorio. Corresponde aclarar, no obstante, que no se realizó un ensayo de reproducción con un evaluador externo: el tiempo efectivo de puesta en marcha por un tercero no fue medido y por lo tanto no se afirma.

Sobre las métricas MTTD/MTTR/TMR: la adaptación de estas métricas del dominio ITIL al ciclo post-venta de e-commerce resultó operativamente válida: los timestamps registrados automáticamente en PostgreSQL permiten calcular y visualizar las métricas sin intervención manual, y los valores obtenidos son coherentes con el comportamiento esperado del sistema.

Sobre el retorno al marco teórico: las tres nociones que el Capítulo 2 puso a trabajar admiten una lectura a la luz de lo medido. La primera es la escala de madurez de proceso de van der Aalst (2016), donde la automatización es un nivel intermedio entre la documentación y la gobernanza: el sistema construido alcanza el nivel de automatización —el ciclo se ejecuta sin intervención humana y queda instrumentado— pero no el de gobernanza, porque no incorpora control de admisión, política de reintentos ni gestión del caso de excepción, y la prueba de concurrencia lo mostró con precisión: 40,8 % de las órdenes quedaron sin procesar sin que ningún mecanismo lo advirtiera. La segunda es la dimensión de capacidad de respuesta

(responsiveness) del modelo SERVQUAL (Zeithaml et al., 1990), que el TMR operacionaliza: los 1,47 s del canal simulado y los 3,07 s de Telegram saturan esa dimensión, pero SERVQUAL mide cinco y este trabajo instrumentó una sola. La fiabilidad y la empatía —que dependen de que la respuesta sea correcta y pertinente, no de que sea rápida— quedaron fuera de la medición, y es exactamente allí donde se ubica el riesgo declarado en la Sección 5.2.2 sobre el contenido de las respuestas de tipo FAQ. La tercera es la transferibilidad de MTTD y MTTR discutida en la Sección 2.1.3: la transferencia se sostuvo a nivel estructural, como se anticipó, pero la medición confirmó también su límite semántico. Un MTTD de 0,009 segundos no informa sobre detección de nada, porque no hay falla que detectar en un flujo nominal; informa sobre la distancia entre dos escrituras. Métricas propias del dominio comercial —order cycle time, order fulfillment lead time— describirían el mismo intervalo sin arrastrar una promesa que el fenómeno no tiene.

6.4 Limitaciones del estudio

Las limitaciones del trabajo se documentan en detalle en la Sección 5.4.1 y en el análisis de amenazas a la validez del Capítulo 3. Se enuncian aquí, para que el lector que llega a las conclusiones las encuentre y no deba reconstruirlas: (i) el entorno es de laboratorio local y los datos son simulados, de modo que el término automatizado de toda comparación proviene de condiciones más favorables que las de un despliegue productivo; (ii) el baseline manual se midió sobre un único operador que ya conocía el procedimiento, lo que acota su representatividad aun cuando la decisión reduzca el factor reportado; (iii) el contraste entre ambas series se ejecutó sobre poblaciones que miden costo marginal de procesamiento y no latencia percibida, asimetría de constructo que la significación estadística no resuelve; (iv) el corpus de evaluación del clasificador es de 150 mensajes sobre cuatro categorías, construido por el propio equipo, y su distribución responde a un supuesto de diseño y no a tráfico observado; (v) no se evaluó la corrección del contenido de las respuestas entregadas al cliente, omisión que resulta material en las consultas de tipo FAQ porque el contexto de la base de conocimiento no llega al prompt; (vi) bajo concurrencia el sistema pierde el 40,8 % de las órdenes sin perder integridad, de modo que las cifras de desempeño valen para régimen secuencial; y (vii) el Flujo 2 depende de un proveedor externo de inferencia, lo que introduce a la vez un punto único de falla y la transferencia internacional de datos analizada en la Sección 2.6. Estas limitaciones no invalidan las conclusiones dentro del alcance declarado (prototipo funcional en entorno controlado), pero deben tenerse en cuenta al extrapolar los resultados a entornos de producción reales.

CAPÍTULO 7: RECOMENDACIONES

7.1 Recomendaciones para producción

El sistema implementado en este trabajo corresponde a una versión de desarrollo (denominada “SIMPLE”) que opera sobre un entorno local con datos simulados. Como parte del trabajo se diseñaron también versiones de producción de ambos flujos, disponibles en el repositorio como archivos JSON importables en n8n:

- **Flujo 1 PRODUCCIÓN** (workflows/Flujo 1 – Pipeline de Procesamiento de Órdenes PRODUCCION.json): extiende el webhook de entrada para soportar triggers nativos de WooCommerce, Shopify y MercadoLibre, con un nodo de normalización que unifica el formato de cada plataforma antes de ingresar al pipeline de stock.
- **Flujo 2 PRODUCCIÓN** (workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json): reemplaza el webhook simulado por triggers reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025), con un nodo Merge que unifica los tres canales en un flujo único de procesamiento.

Estas versiones no fueron ejecutadas en producción real durante el desarrollo de este trabajo integrador —quedan marcadas como active: false en n8n— pero constituyen la hoja de ruta técnica para una implementación productiva (Figuras 10 y 11).

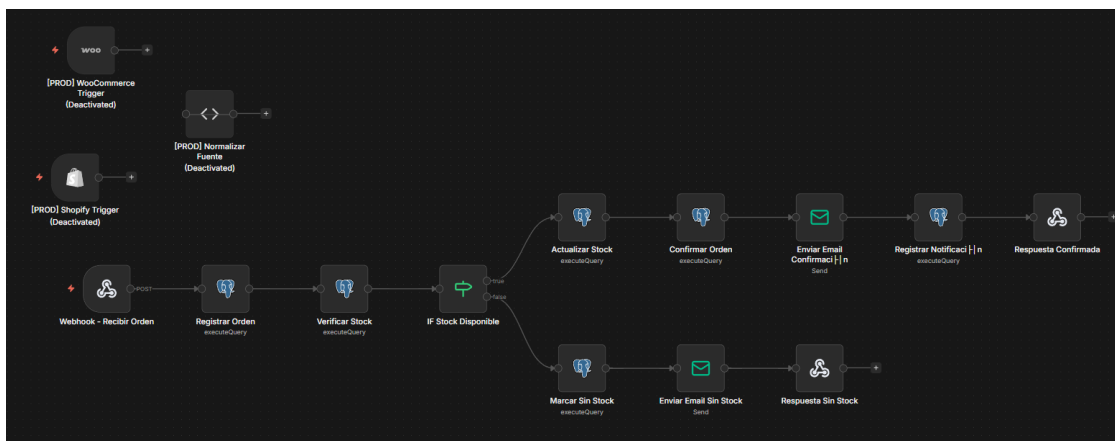


Figura 10: Canvas del Flujo 1 en su variante de producción, con los nodos de integración con plataformas de e-commerce reales.

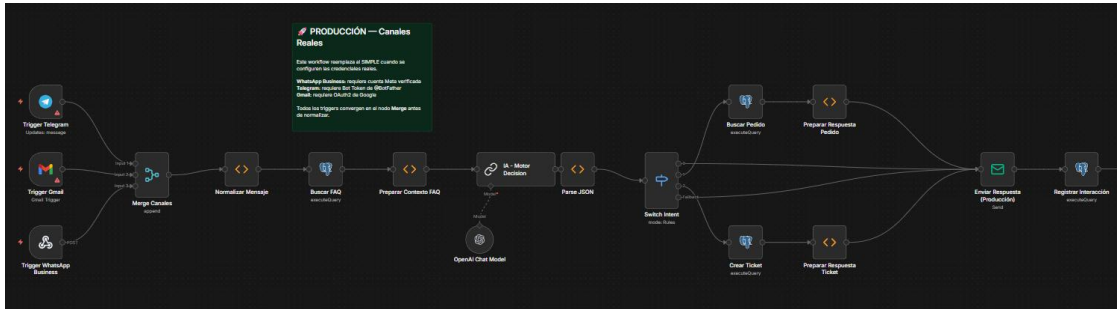


Figura 11: Canvas del Flujo 2 en su variante de producción, con los triggers de Telegram, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025).

Para llevar el sistema a producción se recomienda adicionalmente:

- Implementar autenticación en los webhooks (API keys o JWT) para evitar accesos no autorizados.
- Agregar rate limiting y circuit breakers para proteger el sistema ante picos de tráfico o caídas de la API de OpenAI.
- Completar la verificación de la cuenta de WhatsApp Business API con Meta antes de activar el Flujo 2 PRODUCCIÓN.
- Implementar caché Redis para las respuestas FAQ más frecuentes y reducir invocaciones innecesarias a la IA.
- Incorporar tests de smoke automatizados (scripts que se ejecuten periódicamente) para validar la salud del pipeline.
- Evaluar modelos de IA locales (Llama 3, Mistral) como alternativa a OpenAI para reducir costos y dependencia de servicios externos en escenarios de alto volumen.
- Control de admisión y encolado de solicitudes: la prueba de concurrencia de la Sección 5.1 mostró que, si bien el pipeline no pierde integridad transaccional bajo carga simultánea, el 40,8 % de las órdenes queda sin procesar. Un despliegue productivo debería interponer una cola de mensajes entre el webhook y el pipeline, de modo que las solicitudes se encolen en lugar de descartarse, e incorporar métricas de profundidad de cola y de reintentos.
- Capturar la marca de recepción en la capa HTTP: hoy `received_at` la escribe el nodo Registrar Orden, ya dentro del pipeline, de modo que el MTTD mide el intervalo entre dos escrituras sobre la base local y no el que media entre el arribo del pedido y su detección (Sección 4.3.3). Tomar la marca en el punto de entrada —desde el propio webhook, antes de la primera escritura— corrige la operacionalización sin alterar la definición de la métrica, y permite además separar la latencia de red de la de procesamiento. Es una corrección instrumental de bajo costo que vuelve al MTTD comparable con el de un despliegue real.

- Homogeneizar el filtro de procedencia en las vistas de métricas: cuatro de las seis vistas agregan sobre la totalidad de la tabla de órdenes o de interacciones, sin restringir por `data_source` (Anexo A). Dos paneles del tablero del Flujo 1 consumen esas vistas y por lo tanto muestran también los datos de carga inicial (Sección 4.5). Incorporar la restricción a la definición de las vistas —y no a la consulta de cada panel— evita que la propiedad dependa de quién construya el tablero, y hace que la separación entre lo medido y lo precargado sea una garantía del esquema y no una convención de uso.
- Aislar el componente de red del tiempo de respuesta: enviar un mismo subconjunto de mensajes por el canal simulado y por Telegram, de modo que las dos series sean apareadas y la diferencia entre ambas sea atribuible al trayecto de entrega y no a la composición de la muestra (Sección 4.6).

7.2 Líneas futuras

- Fine-tuning del modelo de IA con datos reales de conversaciones de clientes para mejorar la precisión de clasificación de intents.
- Análisis de sentimiento sobre los mensajes de RECLAMO para priorización automática de tickets según tono emocional del cliente.
- Integración con sistemas ERP/CRM existentes en el e-commerce (Odoo, HubSpot) para sincronización bidireccional de datos.
- Extensión del pipeline a envíos y logística: notificaciones automáticas de despacho y tracking en tiempo real.
- Medición del baseline manual con un diseño de mayor alcance: replicar el protocolo de la Sección 3.5.5 sobre varios operadores independientes y sin entrenamiento previo. El contraste de H1 ya está ejecutado (Sección 5.3), de modo que lo que ese diseño agregaría no es la prueba sino precisamente aquello que la prueba no puede dar: una estimación de la variabilidad entre operadores, la posibilidad de un grupo de control asignado al azar y, con ello, la atribución de la diferencia al procedimiento y no al caso particular medido.
- Implementación efectiva de la omnicanalidad: incorporar un identificador de cliente unificado y un identificador de conversación a la tabla `interactions`, y agregar al Flujo 2 un nodo de recuperación del historial previo, de modo que el contexto se preserve entre canales según la definición de la Sección 2.3.1.
- Cumplimiento del régimen de protección de datos personales: implementar el registro del consentimiento del titular, la política de retención y supresión, la minimización de los datos remitidos al proveedor de inferencia y la resolución de la base legal de la transferencia internacional, conforme la Ley 25.326 analizada en la Sección 2.6.
- Incorporación de ejemplos etiquetados al prompt y cableado del contexto de FAQ que el flujo ya construye, para cuantificar la mejora sobre el desempeño zero-shot medido en este trabajo. Ese cableado debe medirse junto con lo que hoy falta: una evaluación de la corrección del contenido de las respuestas,

que este trabajo no ejecutó (Sección 3.2). El procedimiento que se propone es el siguiente: extraer de la tabla interactions las respuestas efectivamente entregadas, restringir la muestra a los mensajes clasificados como FAQ, y hacerlas juzgar por dos evaluadores independientes contra las veintitrés entradas de la base de conocimiento transcritas en el Anexo D, sobre una rúbrica de tres niveles —respuesta consistente con la política de la tienda, respuesta genérica pero no contradictoria, y respuesta que contradice la política vigente—, reportando el acuerdo entre ambos evaluadores con el mismo coeficiente κ que se empleó para el conjunto de etiquetas de referencia. La comparación de esa medición antes y después del cableado del contexto es la que permitiría atribuirle una mejora, y no la mera inspección del prompt.

CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS

- Aguirre Mayorga, H. S. (2022). Aproximación metodológica para la innovación y transformación digital de los procesos de negocio. Un caso de estudio. *Cuadernos de Administración*, 35, 1–22.
<https://doi.org/10.11144/Javeriana.cao35.amitd>
- Alderete, M. V., Jones, C., & Motta, J. J. (2017). Los factores organizacionales y del entorno en la adopción del comercio electrónico en pymes de Córdoba, Argentina. *Redes. Revista de Estudios Sociales de la Ciencia y la Tecnología*, 23(45), 63–95. <https://doi.org/10.48160/18517072re45.111>
- Alderete, M. V., & Porris, M. S. (2023). Análisis de la adopción del comercio electrónico en Pymes y su vínculo con instituciones locales. *Ciencias Administrativas*, (22), e122. <https://doi.org/10.24215/23143738e122>
- Axelos. (2019). *ITIL Foundation: ITIL 4 edition*. TSO (The Stationery Office).
- Axllent. (2025). *Mailpit — Email & SMTP testing tool*.
<https://github.com/axllent/mailpit>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.
- Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79.
<https://doi.org/10.1145/2723872.2723882>
- Bravo Maruri, N. R., Ramírez Reina, Á. G., Orozco Lara, F. R., & Espinoza Martínez, M. P. (2025). Automatización del soporte al cliente mediante un chatbot con IA integrado al CRM: caso de estudio empresa EDITRATECH. *GADE: Revista Científica*, 5(3), 206–219. <https://doi.org/10.63549/rg.v5i3.705>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Cámara Argentina de Comercio Electrónico. (2025). *Estudio Anual de Comercio Electrónico 2024*. CACE. Recuperado el 28 de agosto de 2026, de <https://cace.org.ar/pages/biblioteca-de-estudios>
- Cochran, W. G. (1977). *Sampling techniques* (3.^a ed.). John Wiley & Sons.
- Docker Inc. (2025). *Docker Compose documentation*.
<https://docs.docker.com/compose/>

- Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018). *Fundamentals of business process management* (2.^a ed.). Springer.
<https://doi.org/10.1007/978-3-662-56509-4>
- Fondevila-Gascón, J.-F., Huamanchumo, A., Martín-Guart, R., & Gutiérrez-Aragón, Ó. (2024). El chatbot como factor de éxito comunicativo, de marketing y empresarial: análisis empírico. *Correspondencias & Análisis*, (19), 47–70.
<https://doi.org/10.24265/cian.2024.n19.02>
- Grafana Labs. (2025). *Grafana documentation*.
<https://grafana.com/docs/grafana/latest/>
- Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). *Metodología de la investigación* (6.^a ed.). McGraw-Hill.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv*.
<https://arxiv.org/abs/2311.05232>
- HubSpot. (2024). *Annual state of service report*. HubSpot.
<https://www.hubspot.com/hubfs/2024%20HubSpot%20State%20of%20Service.pdf>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3.^a ed., borrador en revisión permanente). Stanford University. Recuperado el 28 de agosto de 2026, de
<https://web.stanford.edu/~jurafsky/slp3/>
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174. <https://doi.org/10.2307/2529310>
- Laudon, K. C., & Traver, C. G. (2021). *E-commerce 2021: Business, technology, society* (16.^a ed.). Pearson.
- Ley 25.326 de Protección de los Datos Personales. (2000). Boletín Oficial de la República Argentina.
<https://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/norma.htm>
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1–35.
<https://doi.org/10.1145/3560815>

- Luo, H., Liu, P., & Esping, S. (2023). Towards data-efficient customer intent recognition with prompt-based learning paradigm. *arXiv*.
<https://arxiv.org/abs/2309.14779>
- McTear, M. (2020). *Conversational AI: Dialogue systems, conversational agents, and chatbots*. Morgan & Claypool Publishers.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.
- Misischia, C. V., Pöcze, F., & Strauss, C. (2022). Chatbots in customer service: Their relevance and impact on service quality. *Procedia Computer Science*, 201, 421–428. <https://doi.org/10.1016/j.procs.2022.03.055>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3.^a ed.). John Wiley & Sons.
- n8n GmbH. (2025). *n8n documentation*. <https://docs.n8n.io/>
- Ngai, E. W. T., Lee, M. C. M., Luo, M., Chan, P. S. L., & Liang, T. (2021). An intelligent knowledge-based chatbot for customer service. *Electronic Commerce Research and Applications*, 50, 101098.
<https://doi.org/10.1016/j.elerap.2021.101098>
- OpenAI. (2024). *GPT-4o mini — Model documentation*.
<https://platform.openai.com/docs/models/gpt-4o-mini>
- Pachas-Santos, L. A., Calderón-Vilca, H. D., & Cárdenas-Mariño, F. C. (2023). Chatbot basado en el aprendizaje profundo para recomendar productos relevantes. *Computación y Sistemas*, 27(2), 4119.
<https://doi.org/10.13053/cys-27-2-4119>
- Parikh, S., Tiwari, M., Tumbade, P., & Vohra, Q. (2023). Exploring zero and few-shot techniques for intent classification. En *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (Vol. 5: Industry Track) (pp. 744–751). Association for Computational Linguistics.
<https://doi.org/10.18653/v1/2023.acl-industry.71>
- Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv*. <https://arxiv.org/abs/2211.09527>
- PostgreSQL Global Development Group. (2025). *PostgreSQL 15 documentation*.
<https://www.postgresql.org/docs/15/>
- Pyptacz, P., & Žukovskis, J. (2023). Implementing robotic process automation in small and medium-sized enterprises — Implications for organisations. *Procedia Computer Science*, 225, 337–346. <https://doi.org/10.1016/j.procs.2023.10.018>
- Ramos De Santis, P. (2024). Satisfacción del cliente en la logística: un análisis de chatbots en las empresas líderes de Colombia, Perú y Ecuador. *Retos. Revista de Ciencias de la Administración y Economía*, 14(27), 115–130.
<https://doi.org/10.17163/ret.n27.2024.08>

- Turban, E., Outland, J., King, D., Lee, J. K., Liang, T.-P., & Turban, D. C. (2018). *Electronic commerce 2018: A managerial and social networks perspective* (9.^a ed.). Springer.
- van der Aalst, W. M. P. (2016). *Process mining: Data science in action* (2.^a ed.). Springer. <https://doi.org/10.1007/978-3-662-49851-4>
- Verhoef, P. C., Kannan, P. K., & Inman, J. J. (2015). From multi-channel retailing to omni-channel retailing: Introduction to the special issue on multi-channel retailing. *Journal of Retailing*, 91(2), 174–181. <https://doi.org/10.1016/j.jretai.2015.02.005>
- WhatsApp LLC. (2025). *WhatsApp Business Platform — Cloud API*. <https://developers.facebook.com/docs/whatsapp/cloud-api>
- Womack, J. P., & Jones, D. T. (2003). *Lean thinking: Banish waste and create wealth in your corporation* (2.^a ed.). Free Press.
- Yin, R. K. (2018). *Case study research and applications: Design and methods* (6.^a ed.). SAGE Publications.
- Zeithaml, V. A., Parasuraman, A., & Berry, L. L. (1990). *Delivering quality service: Balancing customer perceptions and expectations*. The Free Press.
- Zhang, D., Pee, L. G., & Cui, L. (2021). Artificial intelligence in e-commerce fulfillment: A case study of resource orchestration at Alibaba's Smart Warehouse. *International Journal of Information Management*, 57, 102304. <https://doi.org/10.1016/j.ijinfomgt.2020.102304>

CAPÍTULO 9: ANEXOS

Anexo A: Resumen del Schema de Base de Datos

Tabla	Flujo	Columnas principales	Descripción
products	1	id, sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock
orders	1+2	id, order_number, customer_name, customer_email, product_id, quantity, status, received_at, processed_at, notified_at	Órdenes de compra; los timestamps permiten calcular MTTD y MTTR; consultada por chatbot para ESTADO_PEDIDO
order_items	1	id, order_id (FK), product_id (FK), quantity, unit_price, subtotal	Ítems de cada orden: normaliza la relación orden→productos (N por orden)
stock_alerts	1	id, product_id (FK), order_id (FK), sku, stock_actual, stock_min, created_at	Alertas de reposición: se registra cuando el stock queda por debajo de stock_min
interactions	2	id, channel, user_id, message, intent, ai_response, is_urgent, received_at, responded_at	Registro de conversaciones del chatbot
tickets	2	id, interaction_id, order_id, channel, subject, status, priority	Tickets de soporte (intent RECLAMO)
faq_responses	2	id, question, answer, category, enabled	Base de conocimiento para preguntas frecuentes

Script DDL completo. A continuación se transcribe el script de creación del esquema (tablas, restricciones de integridad, índices y vistas de métricas), que corresponde al entregable comprometido en la Tabla 3.1. Los datos de carga inicial se listan por separado en el Anexo C.

```
CREATE TABLE IF NOT EXISTS products (
  id      SERIAL PRIMARY KEY,
  sku     VARCHAR(50) UNIQUE NOT NULL,
```

|

```
name      VARCHAR(200) NOT NULL,
price     DECIMAL(10,2) NOT NULL CHECK (price >= 0),
stock     INTEGER      NOT NULL DEFAULT 0 CHECK (stock >= 0),
stock_min INTEGER      NOT NULL DEFAULT 5,
category  VARCHAR(100),
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE IF NOT EXISTS orders (
  id          SERIAL PRIMARY KEY,
  order_number VARCHAR(50) UNIQUE NOT NULL,
  customer_name VARCHAR(200) NOT NULL,
  customer_email VARCHAR(200) NOT NULL,
  customer_phone VARCHAR(50),
  product_id  INTEGER REFERENCES products(id),
  quantity    INTEGER NOT NULL CHECK (quantity > 0),
  total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0),
  status      VARCHAR(50) DEFAULT 'pending'
    CHECK (status IN (
      'pending',
      'processing',
      'confirmed',
      'shipped',
      'delivered',
      'no_stock',
      'cancelled',
      'error'
    )),
  received_at TIMESTAMPTZ DEFAULT NOW(),
  processed_at TIMESTAMPTZ,
  notified_at TIMESTAMPTZ,
  raw_payload JSONB,
  data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
    CHECK (data_source IN ('measured', 'synthetic', 'e4_manual'))
);
```

-- Ítems de una orden: normaliza la relación orden→productos. orders.product_id y
-- orders.quantity se conservan por compatibilidad con los workflows
-- actuales (que cargan una orden mono-producto); el diseño objetivo es
-- que cada línea de la orden viva acá.

```
CREATE TABLE IF NOT EXISTS order_items (
  id          SERIAL PRIMARY KEY,
  order_id    INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  INTEGER NOT NULL REFERENCES products(id),
```


|

```
quantity    INTEGER    NOT NULL CHECK (quantity > 0),
unit_price  DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
subtotal    DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) STORE
D
);
```

-- Alertas de stock bajo: se registra cada vez que un producto
-- cruza su umbral de reposicion tras descontarse el stock de una orden.

```
CREATE TABLE IF NOT EXISTS stock_alerts (
  id          SERIAL PRIMARY KEY,
  product_id  INTEGER    NOT NULL REFERENCES products(id),
  order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
  sku         VARCHAR(50) NOT NULL,
  stock_actual INTEGER    NOT NULL CHECK (stock_actual >= 0),
  stock_min   INTEGER    NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
              CHECK (data_source IN ('measured', 'synthetic'))
);
```

```
-- #####
-- FLUJO 2 — CHATBOT MULTICANAL CON IA
-- #####
```

```
CREATE TABLE IF NOT EXISTS interactions (
  id          SERIAL PRIMARY KEY,
  channel     VARCHAR(20) NOT NULL
              CHECK (channel IN ('whatsapp', 'telegram', 'email')),
  user_id     VARCHAR(200) NOT NULL,
  message     TEXT        NOT NULL,
  intent      VARCHAR(50) NOT NULL
              CHECK (intent IN ('FAQ', 'ESTADO_PEDIDO', 'RECLAMO', 'GENERAL')),
  ai_response TEXT        NOT NULL,
  order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
  is_urgent   BOOLEAN    DEFAULT FALSE,
  received_at TIMESTAMPTZ DEFAULT NOW(),
  responded_at TIMESTAMPTZ,
  data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
              CHECK (data_source IN ('measured', 'synthetic'))
);
```

```
CREATE TABLE IF NOT EXISTS tickets (
  id          SERIAL PRIMARY KEY,
  interaction_id INTEGER    REFERENCES interactions(id) ON DELETE SET NULL,
```

|

```
order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
channel     VARCHAR(20) NOT NULL
            CHECK (channel IN ('whatsapp', 'telegram', 'email')),
user_id     VARCHAR(200) NOT NULL,
subject     TEXT       NOT NULL,
status      VARCHAR(50) DEFAULT 'open'
            CHECK (status IN ('open', 'in_progress', 'resolved', 'closed')),
priority    VARCHAR(20) DEFAULT 'normal'
            CHECK (priority IN ('low', 'normal', 'high', 'urgent')),
created_at  TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ,
data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
            CHECK (data_source IN ('measured', 'synthetic'))
);
```

```
CREATE TABLE IF NOT EXISTS faq_responses (
  id          SERIAL PRIMARY KEY,
  question    TEXT       NOT NULL,
  answer      TEXT       NOT NULL,
  category    VARCHAR(100),
  enabled     BOOLEAN    DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- #####
-- ÍNDICES DE RENDIMIENTO
-- Cada índice responde a una query real de los flujos o vistas.
-- #####
```

```
CREATE INDEX IF NOT EXISTS idx_orders_status      ON orders (status);
CREATE INDEX IF NOT EXISTS idx_orders_source_received  ON orders (data_source,
received_at);
CREATE INDEX IF NOT EXISTS idx_order_items_order_id  ON order_items (order_id);
CREATE INDEX IF NOT EXISTS idx_order_items_product_id  ON order_items (product
_id);
CREATE INDEX IF NOT EXISTS idx_stock_alerts_product  ON stock_alerts (product_i
d);
CREATE INDEX IF NOT EXISTS idx_stock_alerts_created  ON stock_alerts (created_a
t);
CREATE INDEX IF NOT EXISTS idx_interactions_intent    ON interactions (intent);
CREATE INDEX IF NOT EXISTS idx_interactions_channel_received ON interactions (ch
annel, received_at);
CREATE INDEX IF NOT EXISTS idx_interactions_source    ON interactions (data_sour
ce);
```

|

```
CREATE INDEX IF NOT EXISTS idx_interactions_order_id ON interactions (order_id);
CREATE INDEX IF NOT EXISTS idx_tickets_order_id ON tickets (order_id);
CREATE INDEX IF NOT EXISTS idx_tickets_status ON tickets (status);
```

```
-- #####
-- VISTAS — MÉTRICAS PARA LA TESIS
-- #####
```

```
CREATE OR REPLACE VIEW v_order_processing_time AS
SELECT
  o.id,
  o.order_number,
  o.customer_email,
  o.status,
  o.received_at,
  o.processed_at,
  o.notified_at,
  EXTRACT(EPOCH FROM (o.processed_at - o.received_at)) AS mttdd_seconds,
  EXTRACT(EPOCH FROM (o.notified_at - o.processed_at)) AS mttr_seconds,
  EXTRACT(EPOCH FROM (o.notified_at - o.received_at)) AS total_seconds
FROM orders o
WHERE o.processed_at IS NOT NULL;
```

```
CREATE OR REPLACE VIEW v_daily_order_summary AS
SELECT
  DATE(received_at) AS fecha,
  COUNT(*) AS total_ordenes,
  COUNT(*) FILTER (WHERE status = 'confirmed') AS confirmadas,
  COUNT(*) FILTER (WHERE status = 'shipped') AS enviadas,
  COUNT(*) FILTER (WHERE status = 'delivered') AS entregadas,
  COUNT(*) FILTER (WHERE status = 'no_stock') AS sin_stock,
  COUNT(*) FILTER (WHERE status = 'error') AS errores,
  SUM(total_amount) FILTER (WHERE status IN ('confirmed','shipped','delivered'))
    AS ingresos_del_dia,
  ROUND(AVG(EXTRACT(EPOCH FROM (processed_at - received_at)))::NUMERIC, 2)
    AS avg_mttdd_seg,
  ROUND(AVG(EXTRACT(EPOCH FROM (notified_at - processed_at)))::NUMERIC, 2)
    AS avg_mttr_seg
FROM orders
GROUP BY DATE(received_at)
ORDER BY fecha DESC;
```

```
CREATE OR REPLACE VIEW v_chatbot_response_time AS
SELECT
```

|

```
i.id,  
i.channel,  
i.user_id,  
i.intent,  
i.received_at,  
i.responded_at,  
EXTRACT(EPOCH FROM (i.responded_at - i.received_at)) AS tmr_seconds,  
i.is_urgent  
FROM interactions i  
WHERE i.responded_at IS NOT NULL;
```

```
CREATE OR REPLACE VIEW v_daily_chatbot_summary AS  
SELECT  
    DATE(received_at) AS fecha,  
    COUNT(*) AS total_interacciones,  
    COUNT(*) FILTER (WHERE intent = 'FAQ') AS faq,  
    COUNT(*) FILTER (WHERE intent = 'ESTADO_PEDIDO') AS estado_pedido,  
    COUNT(*) FILTER (WHERE intent = 'RECLAMO') AS reclamos,  
    COUNT(*) FILTER (WHERE intent = 'GENERAL') AS general,  
    COUNT(*) FILTER (WHERE channel = 'whatsapp') AS via_whatsapp,  
    COUNT(*) FILTER (WHERE channel = 'telegram') AS via_telegram,  
    COUNT(*) FILTER (WHERE channel = 'email') AS via_email,  
    ROUND(AVG(EXTRACT(EPOCH FROM (responded_at - received_at))))::NUMERIC, 2)  
        AS avg_tmr_seg,  
    COUNT(*) FILTER (WHERE is_urgent) AS urgentes  
FROM interactions  
GROUP BY DATE(received_at)  
ORDER BY fecha DESC;
```

```
CREATE OR REPLACE VIEW v_metrics_summary AS  
SELECT  
    (SELECT COUNT(*) FROM orders) AS total_orders,  
    (SELECT COUNT(*) FROM orders WHERE status = 'confirmed')  
        AS orders_confirmed,  
    (SELECT ROUND(AVG(EXTRACT(EPOCH FROM (processed_at - received_at))))::NUM  
ERIC, 2)  
        FROM orders WHERE processed_at IS NOT NULL) AS avg_mttdd_seg,  
    (SELECT ROUND(AVG(EXTRACT(EPOCH FROM (notified_at - processed_at))))::NUM  
ERIC, 2)  
        FROM orders WHERE notified_at IS NOT NULL) AS avg_mttr_seg,  
    (SELECT COUNT(*) FROM interactions) AS total_interactions,  
    (SELECT ROUND(AVG(EXTRACT(EPOCH FROM (responded_at - received_at))))::NUM  
ERIC, 2)  
        FROM interactions WHERE responded_at IS NOT NULL) AS avg_tmr_seg,
```

|

```
(SELECT COUNT(*) FROM tickets)          AS total_tickets,
(SELECT COUNT(*) FROM tickets WHERE status = 'resolved')
          AS tickets_resolved;

-- #####

-- =====
-- Vista del corpus evaluado del Flujo 2
-- Aísla los 150 mensajes que tienen etiqueta de referencia, para que
-- las tablas de resultados del Cap. 5 no promedien sobre la totalidad
-- de las interacciones medidas. Se delimita por ventana temporal
-- porque el identificador de usuario se repite entre corridas.
-- =====
CREATE OR REPLACE VIEW v_chatbot_corpus AS
SELECT
  i.id,
  i.channel,
  i.user_id,
  i.intent,          -- intención PREDICHA por el modelo
  i.received_at,
  i.responded_at,
  EXTRACT(EPOCH FROM (i.responded_at - i.received_at)) AS tmr_seconds,
  i.is_urgent
FROM interactions i
WHERE i.data_source = 'measured'
  AND i.responded_at IS NOT NULL
  AND i.received_at >= '2026-08-12 23:00:00'
  AND i.received_at < '2026-08-13 00:00:00';
```

Adicionalmente, la base de datos incluye 6 vistas de métricas. El detalle completo se encuentra en la Sección 4.2 del Capítulo 4.

Anexo B: Configuración Docker Compose

Servicio	Imagen	Puerto(s)	Variables de entorno principales
n8n	n8nio/n8n:2.12.2	5678	DB_TYPE, DB_POSTGRESDB_HOST, N8N_SMTP_HOST, GENERIC_TIMEZONE
PostgreSQL	postgres:15.13-alpine	5433 → 5432 (host → contenedor)	POSTGRES_DB=ecommerce_tesis, POSTGRES_USER, POSTGRES_PASSWORD
Mailpit	axllent/mailpit:v1.29.6	8025 (web), 1025 (SMTP)	—

Grafana	grafana/ grafana: 13.0.7	3000	GF_SECURITY_ADMIN_PASSWORD (definida en .env, no versionada)
----------------	--------------------------------	------	---

Volúmenes: n8n_data, pg_data, grafana_data (persistentes). **Red:** tesis_network (bridge). **Levantar:** docker compose up -d | **Detener:** docker compose down

Anexo C: Catálogo de Productos Seed

SKU	Nombre	Precio	Stock	Stock Mín.	Categoría
PROD-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PROD-002	Mouse Inalámbrico Logitech	\$29.99	50	5	Periféricos
PROD-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PROD-004	Monitor Samsung 24" FHD	\$249.99	8	2	Monitores
PROD-005	Auriculares Sony WH-1000XM5	\$349.99	4	2	Audio
PROD-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PROD-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PROD-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PROD-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PROD-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras

PROD-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PROD-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PROD-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks
PROD-014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores
PROD-015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PROD-016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PROD-017	Disco Rígido Seagate 2TB HDD	\$64.99	20	4	Almacenamiento
PROD-018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamiento
PROD-019	Pad Mouse XL Antideslizante	\$14.99	60	8	Accesorios
PROD-020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

Anexo D: FAQ predefinidas (base de conocimiento completa)

- Se transcribe la base de conocimiento completa: las veintitrés entradas de la tabla `faq_responses`, en el orden en que las cargan los guiones `init_simple.sql` y `seed_expand.sql` del repositorio. La columna de respuesta reproduce el

inicio del texto almacenado; el texto íntegro está en esos guiones. Conviene recordar, para leer este anexo en su justo alcance, lo establecido en la Sección 4.4.3: este contenido está cargado en la base y el flujo lo recupera, pero no llega al prompt del modelo, de modo que no intervino en las respuestas medidas en el Capítulo 5.

Categoría	Pregunta	Respuesta (resumen)
Pagos	¿Cuáles son los métodos de pago?	Aceptamos tarjeta de crédito, débito, transferencia bancaria y MercadoPago. Todos los pagos son procesados de forma segura.
Envíos	¿Cuánto tarda el envío?	Los envíos dentro de Mendoza tardan 1-3 días hábiles. Para el resto del país, entre 3-7 días hábiles.
Devoluciones	¿Cómo puedo devolver un producto?	Tenés 30 días desde la recepción para iniciar una devolución. Escribinos indicando tu número de pedido y el motivo, y te guiamos en...
Garantía	¿Tienen garantía los productos?	Todos nuestros productos tienen garantía oficial del fabricante. La duración varía según el producto (generalmente 12 meses).
Soporte	¿Cómo contacto a soporte?	Podés escribirnos por WhatsApp, Telegram o email. Nuestro sistema de IA te asiste 24/7 y, si es necesario, escala tu caso a un...
Facturación	¿Hacen factura?	Sí, emitimos factura electrónica (AFIP) para todas las compras. La factura se envía automáticamente al email registrado en tu...
Pagos	¿Puedo pagar en cuotas?	Sí, con tarjetas de crédito Visa, Mastercard y American Express podés pagar en hasta 12 cuotas sin interés en productos...
Pagos	¿Es seguro pagar con tarjeta en la web?	Totalmente. Usamos encriptación SSL y procesamos los pagos a través de MercadoPago, que cumple con los estándares PCI-DSS.
Envíos	¿Hacen envíos a todo el país?	Sí, enviamos a todo el territorio argentino. Trabajamos con OCA, Andreani y correo argentino.

Envíos	¿Puedo hacer el seguimiento de mi envío?	Sí. Una vez despachado tu pedido, te enviamos un email con el número de tracking y el enlace directo para seguir el paquete en...
Envíos	¿Hacen envíos internacionales?	Por el momento solo enviamos dentro de Argentina. Estamos trabajando para expandir a países limítrofes próximamente.
Envíos	¿Qué pasa si no estoy en casa cuando llega el pedido?	El transportista deja un aviso y hace un segundo intento al día siguiente.
Devoluciones	¿Cuál es la política de cambios?	Podés cambiar un producto dentro de los 30 días de recibido, siempre que esté en su embalaje original y sin uso.
Devoluciones	¿Cómo inicio una devolución?	Escribinos por este chat o al email devoluciones@techstore.com.ar con tu número de pedido y el motivo.
Garantía	¿Qué cubre la garantía?	La garantía cubre defectos de fabricación. NO cubre daños por mal uso, caídas, líquidos o modificaciones no autorizadas.
Garantía	¿Cómo hago válida la garantía?	Guardá el comprobante de compra (te lo enviamos por email). Si el producto falla, escribinos con foto o video del problema y tu...
Productos	¿Los productos son originales?	Sí, todos nuestros productos son 100% originales con garantía oficial del fabricante.
Productos	¿Tienen stock en físico para retirar?	Por el momento somos una tienda 100% online y no contamos con local a la calle.
Productos	¿Cómo sé si un producto tiene stock?	Si podés agregarlo al carrito, hay stock disponible. Si aparece como "Sin stock", podés anotarte en la lista de espera y te...
Facturación	¿Hacen factura A para empresas?	Sí, emitimos factura A para responsables inscriptos. Durante el checkout elegís el tipo de comprobante e ingresás el CUIT y razón...
Facturación	¿Cuándo recibo la factura?	La factura electrónica se genera automáticamente al confirmar el pago y te llega por email en minutos.

|

Pedidos	¿Puedo cancelar un pedido?	Podés cancelar sin costo dentro de las 2 horas de realizado, siempre que no haya sido despachado.
Pedidos	¿Cómo recibo el comprobante de compra?	Te enviamos un email de confirmación inmediatamente después del pago con todos los detalles del pedido y la factura adjunta.

Anexo E: Comandos de Testing

```
# ----- Levantar el entorno -----
docker compose up -d
docker compose ps

# ----- Verificar la base -----
# El host publica PostgreSQL en 5433 (mapeo 5433:5432) para no colisionar
# con una instalación nativa que ocupe el 5432. Dentro de la red de Docker
# los servicios se ven entre sí como postgres:5432.
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis -c "\dt"

# ----- Flujo 1: orden con stock suficiente -----
curl -X POST http://localhost:5678/webhook/orden-nueva \
  -H "Content-Type: application/json" \
  -d '{"order_number":"ORD-TEST-001","customer_name":"Test","customer_email":"test@example.com","customer_phone":"5492615551234","product_sku":"PROD-018","quantity":1}'

# ----- Flujo 1: rama sin stock -----
# Se piden más unidades de las disponibles para forzar la rama no_stock.
curl -X POST http://localhost:5678/webhook/orden-nueva \
  -H "Content-Type: application/json" \
  -d '{"order_number":"ORD-TEST-002","customer_name":"Test","customer_email":"test@example.com","customer_phone":"5492615551234","product_sku":"PROD-005","quantity":999}'

# ----- Flujo 2: chatbot -----
# El path del webhook es whatsapp-business. El nodo normalizador acepta un
# payload plano con los campos from y message.
curl -X POST http://localhost:5678/webhook/whatsapp-business \
  -H "Content-Type: application/json" \
  -d '{"from":"5492615551234","message":"¿Cuáles son los métodos de pago?"}'

# ----- Corrida de carga secuencial (50 órdenes, espaciadas 2 s) -----
for i in $(seq 1 50); do
```

|

```
curl -s -X POST http://localhost:5678/webhook/orden-nueva \
-H "Content-Type: application/json" \
-d "{\"order_number\":\"ORD-CARGA-$i\", \"customer_name\":\"Carga $i\",
\"customer_email\":\"carga$i@example.com\", \"customer_phone\":\"549261555
1234\", \"product_sku\":\"PROD-018\", \"quantity\":1}"
sleep 2
done

# ----- Prueba de concurrencia (20 solicitudes simultáneas) -----
--
# Contra un producto con stock deliberadamente escaso, para verificar que
no
# se confirmen más órdenes que unidades disponibles.
for i in $(seq 1 20); do
    curl -s -X POST http://localhost:5678/webhook/orden-nueva \
    -H "Content-Type: application/json" \
    -d "{\"order_number\":\"ORD-CONC-$i\", \"customer_name\":\"Conc $i\", \"
customer_email\":\"conc$i@example.com\", \"customer_phone\":\"54926155512
34\", \"product_sku\":\"PROD-005\", \"quantity\":1}" &
done
wait

# ----- Verificar métricas y la invariante de stock -----
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis \
-c "SELECT * FROM v_metrics_summary;" \
-c "SELECT COUNT(*) AS stock_negativo FROM products WHERE stock < 0;"

# ----- Ver correos generados -----
# Abrir http://localhost:8025 en el navegador

# ----- Ver tableros -----
# Abrir http://localhost:3000 (usuario admin; contraseña definida en el a
rchivo .env)
```

Anexo F: Lista de Figuras — Implementación de Desarrollo (SIMPLE)

Se listan aquí únicamente las capturas de pantalla del entorno de desarrollo local, es decir las figuras que registran el sistema tal como quedó ejecutándose. Las restantes figuras del trabajo son diagramas de elaboración propia y se consignan en el Listado de Figuras del frontispicio, que no se repite.

Figura	Descripción	Sección
Figura 4	Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.	4.3.2
Figura 6	Canvas del Flujo 2 en n8n,	4.4.2

	mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por intent y el nodo Router Canal que envía la respuesta por el canal de origen.	
Figura 7	Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1. Los paneles de indicadores y de distribución de estados se restringen a los registros medidos; los de «Órdenes totales» y «Órdenes procesadas por día» no lo hacen, de modo que el primer punto de la serie diaria corresponde a las órdenes de carga inicial y no a una corrida experimental (Sección 4.5).	4.5.1
Figura 8	Bandeja de Mailpit con los correos generados por el Flujo 1 durante las pruebas: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com).	4.6.2
Figura 9	Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la precisión de clasificación y la distribución de intenciones sobre el corpus evaluado.	5.2.3

Todas las capturas corresponden al entorno local de desarrollo. Los datos son simulados y representativos de un escenario típico de e-commerce.

Anexo G: Propuesta de Arquitectura de Producción

Las Figuras 10 y 11 corresponden a los workflows de producción diseñados como extensión del sistema. Están disponibles en el repositorio como archivos JSON importables pero **no fueron ejecutados con datos reales** — representan la hoja de ruta técnica para una implementación productiva.

Figura	Descripción	Archivo en repositorio
Figura 10	Canvas del Flujo 1	workflows/Flujo 1 – Pipeline de

	PRODUCCIÓN: triggers WooCommerce, Shopify y nodo de normalización multi- plataforma	Procesamiento de Órdenes PRODUCCION.json
Figura 11	Canvas del Flujo 2 PRODUCCIÓN: triggers reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API unificados con nodo Merge	workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json

Para importar el workflow del Flujo 1 en su variante de producción (Figura 10) en n8n: Workflows → Import from file.

El workflow del Flujo 2 en su variante de producción (Figura 11) se importa por el mismo procedimiento, y requiere credenciales de Telegram Bot, Gmail y WhatsApp Business API para poder activarse.

Nota: Para reproducir el entorno completo ejecutar `docker compose up -d` en la raíz del repositorio.

Anexo H: Prompt de sistema del clasificador de intenciones

Se transcribe a continuación el texto íntegro del prompt de sistema que recibe el modelo GPT-4o-mini en el nodo de inferencia del Flujo 2, tal como está configurado en el workflow versionado en el repositorio. Es el prompt con el que se obtuvo el accuracy del 92,7 % reportado en la Sección 5.2.3. Se publica porque de él depende íntegramente ese resultado: sin el prompt, la medición de precisión no es auditable ni reproducible.

Sos un asistente virtual de atención al cliente de "TechStore", un e-commerce argentino de tecnología.

IDENTIDAD

- Nombre: Asistente TechStore
- Tono: cálido, humano, profesional — como si fuera una persona real de atención al cliente
- Idioma: respondé SIEMPRE en el mismo idioma que el cliente. Si escribe en inglés, respondé en inglés. Si escribe en español, usá español argentino (voseás: vos, tenés, podés)
- NO sos un chatbot genérico — representás a TechStore con orgullo
- Usá el nombre del cliente si está disponible — hace que la conversación se sienta más personal

|

TU TAREA

Analizá el mensaje del cliente, clasificá su intención, y generá una respuesta genuinamente útil y humana.

Respondé ÚNICAMENTE con un JSON válido. Sin texto antes ni después del JSON. Sin markdown.

FORMATO DE RESPUESTA (JSON estricto)

```
{  
  "intent": "FAQ | ESTADO_PEDIDO | RECLAMO | GENERAL",  
  "order_id": null,  
  "urgente": false,  
  "respuesta": "texto cálido, claro y personalizado para el cliente"  
}
```

Obsérvese que el prompt no contiene ejemplos etiquetados de entrada y salida: el clasificador opera en régimen zero-shot. Tampoco recibe el contenido de la base de FAQ; el nodo que construye ese contexto existe en el flujo pero su salida no se referencia desde este prompt, extremo que se declara en la Sección 4.4.3.

Anexo I: Baseline de atención manual — tiempos cronometrados

Se transcriben los tiempos crudos de las doce órdenes procesadas manualmente según el protocolo de la Sección 3.5.5, incluidas las dos descartadas por interrupción del operador, que se consignan con su marca de descarte y no se eliminan del registro. Los valores están en segundos y provienen sin edición del archivo exportado por el cronómetro. El resultado primario de la Sección 5.1.4 se calcula sobre las diez órdenes válidas.

Tabla I.1: Tiempos cronometrados del procesamiento manual, por orden y por fase.

#	Orden	Rama	T1	T2	T3	Total	Estado
1	ORD-E4-001	—	—	—	—	—	Descartada
2	ORD-E4-002	sin stock	8,033	—	—	—	Descartada
3	ORD-E4-003	con stock	7,530	8,598	53,674	69,801	Válida
4	ORD-E4-004	con stock	11,386	9,089	34,453	54,928	Válida
5	ORD-E4-005	sin stock	6,708	5,598	30,449	42,754	Válida
6	ORD-E4-006	con stock	8,236	4,698	36,577	49,510	Válida
7	ORD-	con	5,506	5,768	38,081	49,354	Válida

	E4-007	stock					
8	ORD-E4-008	con stock	7,958	4,857	31,787	44,602	Válida
9	ORD-E4-009	sin stock	7,695	4,448	34,960	47,104	Válida
10	ORD-E4-010	con stock	5,306	4,447	35,884	45,638	Válida
11	ORD-E4-011	con stock	5,806	5,087	32,009	42,902	Válida
12	ORD-E4-012	sin stock	4,365	5,448	34,863	44,676	Válida

El texto completo de las notificaciones redactadas por el operador durante la medición, así como el archivo original exportado por el cronómetro y el script de análisis, se encuentran versionados en el repositorio del trabajo bajo el directorio del experimento.

Se transcriben además los registros que el operador escribió en la base durante la medición. Las doce órdenes comparten la misma marca de recepción porque fueron creadas por una única sentencia de preparación; en consecuencia la columna de tiempo acumulado mide permanencia en la cola y no latencia individual, mientras que la columna de incremento reproduce el tiempo marginal por otra vía instrumental (Sección 5.1.4).

Tabla I.2: Registros en la base de datos de las órdenes procesadas manualmente.

Orden	Estado	processed_at	notified_at	Acumulado	Incremento
ORD-E4-001	pending	—	—	—	— (descartada)
ORD-E4-002	pending	—	—	—	— (descartada)
ORD-E4-003	confirmed	16:47:12	16:48:06	72,38 s	—
ORD-E4-004	confirmed	16:48:29	16:49:04	130,46 s	58,08 s
ORD-E4-005	no_stock	16:49:33	16:50:04	190,13 s	59,67 s
ORD-E4-006	confirmed	16:50:19	16:50:55	241,80 s	51,67 s

ORD-E4-007	confirmed	16:51:12	16:51:50	297,04 s	55,24 s
ORD-E4-008	confirmed	16:52:06	16:52:38	344,58 s	47,54 s
ORD-E4-009	no_stock	16:52:53	16:53:28	394,29 s	49,71 s
ORD-E4-010	confirmed	16:53:39	16:54:15	441,75 s	47,46 s
ORD-E4-011	confirmed	16:54:28	16:55:01	487,30 s	45,55 s
ORD-E4-012	no_stock	16:55:12	16:55:47	533,94 s	46,64 s
Media	—	—	—	313,37 s	51,28 s
Desvío	—	—	—	154,57 s	5,22 s

Anexo J: Corpus de evaluación y procedimiento de cálculo estadístico

El corpus de 150 mensajes utilizado para evaluar la clasificación, sus etiquetas de referencia, las predicciones del modelo y las etiquetas del evaluador independiente se encuentran versionados en el repositorio del trabajo en formato separado por comas, junto con los tiempos de anotación registrados por el instrumento de etiquetado. El corpus fue incorporado al repositorio antes de producirse el etiquetado, de modo que el sello temporal de esa incorporación acredita el carácter ciego del procedimiento descrito en la Sección 3.5.3.

Se detalla a continuación el procedimiento de cálculo del intervalo de confianza de Wilson reportado para el accuracy global, a fin de que el valor publicado sea verificable.

Datos: $x = 139$ aciertos, $n = 150$ mensajes, $\hat{p} = x/n = 0,926667$

Nivel: 95 % $\rightarrow z = 1,959964$

$$\text{centro} = \frac{\hat{p} + z^2/(2n)}{1 + z^2/n}; \quad \text{semiancho} = \frac{z}{n} \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{1 + z^2/n} + \frac{z^2}{4n^2}}$$

$$z^2 = 3,841459 \quad z^2/n = 0,025610 \quad z^2/(2n) = 0,012805$$

|

$$\text{centro} = (0,926667 + 0,012805) / 1,025610 = 0,916019$$

$$\text{semiancho} = (1,959964 / 1,025610) \cdot \sqrt{(0,000453 + 0,0000427)} = 0,042548$$

$$\text{IC 95 \% de Wilson} = [0,873471 ; 0,958567] = [87,3 \% ; 95,9 \%]$$

Comprobación con otros métodos, a título informativo:

Wald (normal, sin corrección) [88,5 % ; 96,8 %]

El límite inferior supera el umbral del 85 % en cualquiera de los métodos, por lo que la conclusión sobre H2b no depende de esta elección.

El coeficiente κ de Cohen reportado en la Sección 5.2.3 se calculó sobre la submuestra de 50 mensajes etiquetada por el evaluador independiente, con un acuerdo observado $P_o = 0,940$ (47 coincidencias sobre 50) y un acuerdo esperado por azar $P_e = 0,259$ derivado de las distribuciones marginales de ambos anotadores, de donde $\kappa = (P_o - P_e) / (1 - P_e) = 0,919$. La interpretación se realiza según la escala de Landis y Koch (1977).